



モデルを使ってソフトウェアを開発しよう

UML初学者向けチュートリアル（C言語版）

株式会社チェンジビジョン

バージョン pdf 0596, 2026-04-01

目次

はじめに	1
この文書の目的	1
対象・目標	1
アイコンの説明	2
注意事項	3
著作権表示	3
諸注意	3
商標等について	3
第1章 準備	5
1.1 モデリングツールを用意する	5
1.2 実装環境を用意する	5
1.3 演習用のプロジェクトを用意する	7
1.4 ボウリングのスコアのつけ方	10
第2章 スコアシートの構造を調べる	15
2.1 スコアシートの例	15
2.2 スコアシートの構造をオブジェクト図で表す	16
2.3 スコアシートの構造をクラス図で表す	34
2.4 まとめ	42
第3章 モデルとコードの対応づけ	43
3.1 なぜ対応づけの検討が必要なのか	43
3.2 対応づけ検討用モデルの作成	44
3.3 構造のモデルとコードを対応づける (1)	46
3.4 振る舞いのモデルを作成する	49
3.5 振る舞いのモデルにコードを対応づける	61
3.6 ステートマシン図に対応するコードを作成する	62
3.7 追加したアクションをクラス図に反映する	68
3.8 変換ルールを使ったコードを確認する	69
3.9 まとめ	77
第4章 スコアやフレームの状態を調べる	79
4.1 フレームの状態について検討する	79
4.2 フレームの状態をステートマシン図で表す	82
4.3 サービスフレームの扱いについて検討する	89
4.4 スコアの状態をステートマシン図で表す	91
4.5 ゲームの進行について検討する	99
4.6 まとめ	103
第5章 できあがったプログラムを試す	105
5.1 プログラムを完成させる	105
5.2 プログラムをテストする	114
5.3 まとめ	116
第6章 まとめ	117
第7章 他の図、他の機能など	119

付録A: モデルやプログラムの作成例.....	121
参考文献.....	123

はじめに

この文書の目的

この文書は、ソフトウェアの開発にモデリングツールを使う方法などについて独習するためのチュートリアルです。

このチュートリアルは、ソフトウェアの開発に、UMLを使って描いたモデルが役に立つことを実感する機会を提供することを目的としています。主に、これからソフトウェア開発にモデル図を使うことについて学ぶみなさんを対象にしています。また、モデリングツールを使うことにも慣れてもらえるよう、「**astah* Professional**」使って演習します。

対象・目標

チュートリアルの対象者

このチュートリアルは、次のような方を対象者と想定しています。

チュートリアルの対象者

- ・モデリングやUMLについてこれから学ぼうとしているみなさん
- ・ソフトウェア開発工程は知っているが、まだモデル図を活用していないみなさん
- ・分析・設計・テストなどソフトウェアの開発に携わっているみなさん

チュートリアルの目標

みなさんがチュートリアル終了後に次のような人になっていることを、このチュートリアルの目標としています。

チュートリアルの目標（人物像）

- ・開発工程のどこでどのモデル図を使えばよいか知っている人になる。
- ・「開発にはモデルがあったほうがいい」と考えている。
- ・自分だけでなく他の人にもモデリングを勧めたいと考えている。

アイコンの説明

本文中では、次のようなアイコンを用いています。

NOTE

告知。みなさんが覚えておくといふ追加情報など。

TIP

ティップス。覚えておくといふ便利なちょっとしたヒントやコツ。

IMPORTANT

重要。間違えたり見落としたりすると期待通りの結果が得られない設定や操作など。

CAUTION

注意。気をつけないといふ問題の発生につながるようなことがら。

WARNING

警告。守らないといふ破損や怪我などにつながる可能性のあることがら。

注意事項

著作権表示

©2025 by Change Vision Inc.

「モデルを使ってソフトウェアを開発しよう: UML初学者向けチュートリアル (C言語版)」

本文書ならびに関連する成果物の著作権は、株式会社チェンジビジョンが保有します。



本文書ならびに関連する成果物は、下記のクリエイティブ・コモンズ ライセンスに則り提供します。

クリエイティブ・コモンズ<表示-非営利-改変禁止 4.0ライセンス>

<https://creativecommons.org/licenses/by-nc-nd/4.0/legalcode.ja>

諸注意

- ・ 本文書は、株式会社チェンジビジョン（作成者）が編集したもので、本文書に関する権利、責任は作成者が保有します。
- ・ 本文書に記載されている情報は、本文書を更新した時点のものであり、利用時にはURLなどの各種の情報が変更されている可能性があります。
- ・ 本文書は演習用テキストとしての使用を想定し、内容等は予告なしに変更することがあります。また、作成者がその内容を保証するものではありません。
- ・ 本文書の内容に誤りや不正確な記述がある場合も、作成者は一切の責任を負いません。
- ・ 本文書に記載の内容や実施結果からいかなる損害が生じても、作成者は責任を負いかねますので、あらかじめご了承ください。
- ・ 本文書の複製、保存については、作成者の同意を得てください。

商標等について

- ・ astahは、株式会社チェンジビジョンの日本、米国、欧州における登録商標です。
- ・ Windows 並びに同社の各製品名は米国マイクロソフトコーポレーションの米国およびその他の国における登録商標です。
- ・ Apple、iCloud、iPad、iPhone、Mac、Macintosh、macOSは、米国およびその他の国々で登録されたApple Inc.の商標です。
- ・ Linuxは Linus Torvalds氏、米国及びその他の国における登録商標 あるいは商標です。
- ・ UNIXは、X/Open Company Limitedが独占的にライセンスしている米国ならびに他の国にお

ける登録商標です。

- その他、本書に記載されている社名、製品名、ブランド名、システム名などは、一般に商標または登録商標でそれぞれ帰属者の所有物です。
- 本文中では ©、®、™、は表示していません。

第1章 準備

このチュートリアルでは、モデリングツールとして **astah* Professional** を、実装にはC言語を使います。最初に、演習に必要な環境を用意しましょう。

1.1 モデリングツールを用意する

このチュートリアルで使用するモデリングツール **astah* Professional** を用意しましょう。

購入前に試用したい場合は、次のページを参照してください。

astah*製品の評価手順

<https://astah.change-vision.com/ja/shopping/evaluate.html>

購入する場合は、次のページを参照して、自分に合ったライセンスを選択してください。

最適なライセンスを見つける

<https://astah.change-vision.com/ja/shopping/price.html>

いずれかの方法で、**astah* Professional** を入手できたら、次のページを参照して、自分に合ったライセンスを登録してください。

ライセンス登録方法

<https://astah.change-vision.com/ja/registry.html>

1.2 実装環境を用意する

このチュートリアルでは、実装に C言語 を使います。Windows、Mac、Linuxで動作するプログラムになるよう、C99標準に準拠したできるだけポータブルなコードを作成しようと思います。

すでにC言語の利用環境が用意できている場合は、その環境を使ってみましょう。また、まだ用意できていない場合のために、それぞれの環境でのC言語用コンパイラのセットアップ方法を紹介しておきます。

1.2.1 Windows環境

Windowsでは、以下のいずれかの方法でCコンパイラを用意します。

MinGW-w64を利用する場合

1. MinGW-w64のインストーラーをダウンロード
 - <https://www.mingw-w64.org/>
2. インストール後、環境変数 **PATH** にMinGW-w64の **bin** ディレクトリを追加

i NOTE

ほぼ同じ環境として、MSYS2をインストールする方法あります。

Visual Studio Build Toolsを利用する場合

1. Visual Studio Community（無料版）をインストール
 - <https://visualstudio.microsoft.com/ja/>
2. インストール時に「C++によるデスクトップ開発」を選択

WSL2とUbuntuを利用する場合

1. PowerShellを管理者で実行
2. `wsl --install` を実行後、PCを再起動する
3. Ubuntuのターミナルを起動する
4. `sudo apt update` と `sudo apt upgrade` を実行
5. `apt install gcc` を実行

1.2.2 macOS環境

macOSでは、Xcodeのコマンドラインツールをインストールします（リスト 1.1）。

リスト 1.1 xcodeのコマンドラインツールをインストールする

[CONSOLE]

```
xcode-select --install
```

これにより、gccコマンド（実際はclangへのエイリアス）が使用可能になります。

1.2.3 Linux環境

多くのLinuxディストリビューションでは、gccがデフォルトでインストールされています。もし、インストールされていない場合は、ディストリビューションに応じたパッケージマネージャーを使ってインストールします（リスト 1.2、リスト 1.3）。

リスト 1.2 Ubuntu/Debianの場合

[CONSOLE]

```
sudo apt-get update
sudo apt-get install build-essential
```

リスト 1.3 Fedora/RHELの場合

[CONSOLE]

```
sudo dnf install gcc make
```

1.2.4 コンパイラの動作確認

正しくインストールされたか確認するには、次のコマンドを実行します（リスト 1.4、リスト 1.5）。

リスト 1.4 gccのバージョン番号を確認する

[CONSOLE]

```
gcc --version
```

または

リスト 1.5 ccのバージョン番号を確認する

[CONSOLE]

```
cc --version
```

バージョン情報が表示されれば、正常にインストールされています。

1.2.5 C言語の基礎知識

このチュートリアルでは、C言語の高度な機能は使いませんが、基本的な知識は必要です。C言語になじみがない方は、以下のような参考資料を確認することをお勧めします。

C言語の基礎を学ぶ参考文献

- ・『作って身につくC言語入門』（久保秋 真 著）[kuboaki01]
- ・『新・明解C言語 入門編』（柴田 望洋 著）[sibata01]
- ・オンライン学習サイト: <https://www.learn-c.org/> [learnc]

このチュートリアルで使用する主なC言語の機能はつぎのようなものです。

このチュートリアルで使うC言語の機能

- ・構造体 (struct)
- ・列挙型 (enum)
- ・関数とポインタ
- ・標準ライブラリ (stdio.h, stdlib.h, string.h, time.h, stdbool.h)

1.3 演習用のプロジェクトを用意する

業務やシステムを分析（あるいは設計）するときにUMLのモデル図を使う場合、モデリングの対象のいくつかの側面について、それぞれに向いた図を作成することが一般的です。

複数の図で表すのは、大きな問題ならば分割して小さな問題として捉えるほうが整理しやすいからです。たとえば、システム全体の構造と、その内部のある構成要素の内部の構造を別の図に分けることができるでしょう。また、関心事によって表すもの（モデルに残すものや削るもの）が変わるからです。たとえば、あるシステムを設計するのに、構造を表すためにクラス図を使い、振る舞いを表すのにシーケンス図やステートマシン図使うといったことです。

astah*でも、モデリングの対象に関連した複数の図をまとめて扱えるよう、ひとまとまりのファイルに保存します。astah*では、この複数の図のまとまりのことを「プロジェクト」と呼んでいます。

1.3.1 astah* のプロジェクトを作成する

それでは、演習用に astah* のプロジェクトを用意しましょう。

astah* のプロジェクトを作成する手順

1. astah* を起動する。
2. 「ファイル」メニューから「プロジェクトの新規作成」を選択する（図 1.1）。

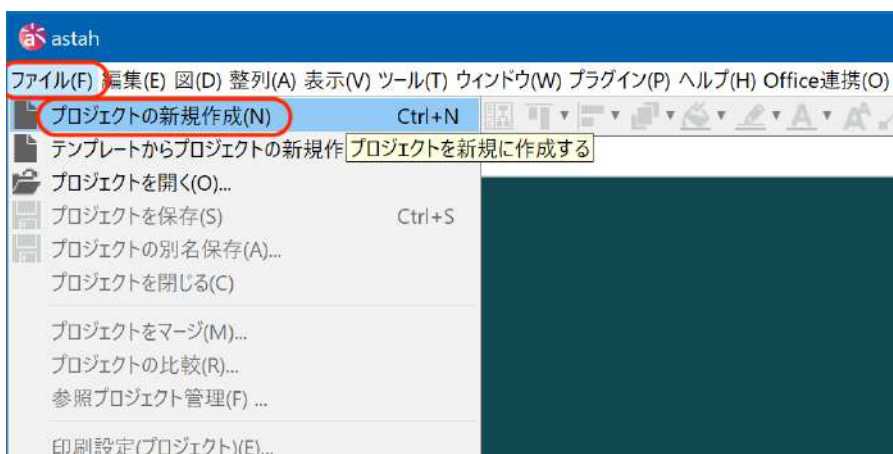


図 1.1 新しいプロジェクトを作成する

3. 「no_title」という名前で新しいプロジェクトが作成される（図 1.2）。

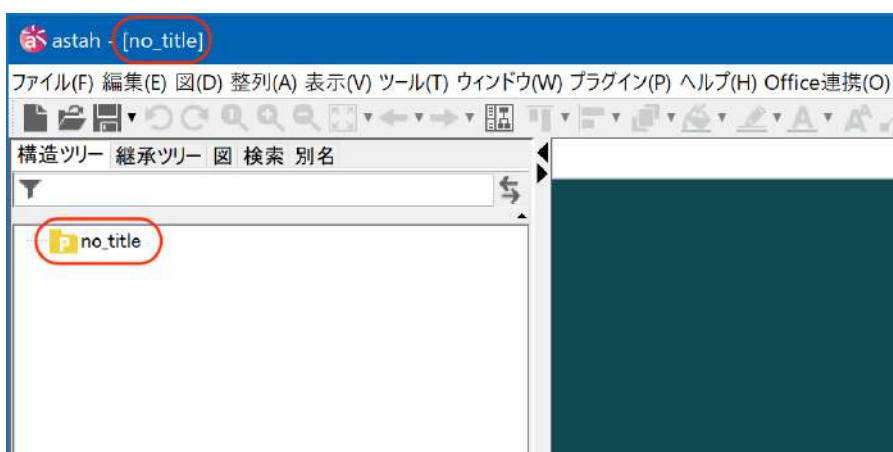


図 1.2 新しいプロジェクトが作成された

1.3.2 作成したプロジェクトを保存する

新規プロジェクトを作成したら、最初に保存しておき、それからモデルの作成に入るようにするとよいでしょう。

astah* のプロジェクトを保存する手順

1. 「ファイル」メニューから「プロジェクトを保存」を選択する（図 1.3）。

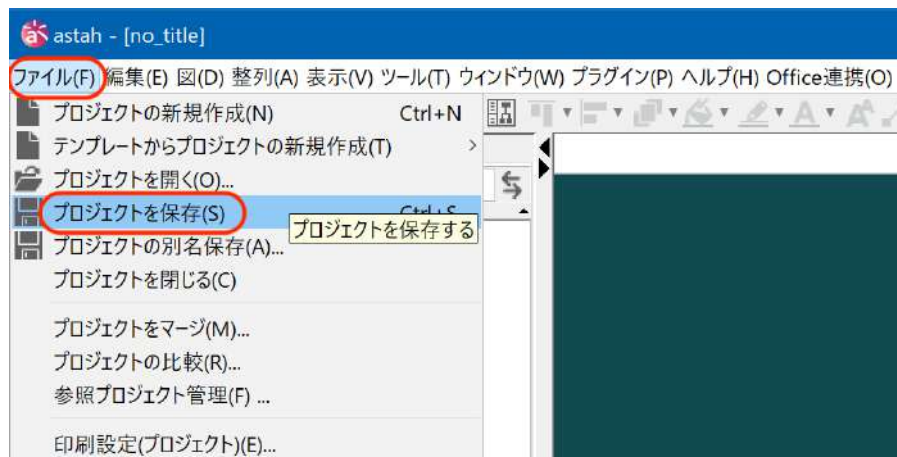


図 1.3 新しいプロジェクトを保存する

2. 「保存」ダイアログが開くので、プロジェクトの保存場所と保存するファイル名を指定する。
 - ここでは、保存場所として「デスクトップ」に「BowlingScore」フォルダーを作成した。
 - ファイル名に「bowling_score」を指定して保存した（図 1.4）。

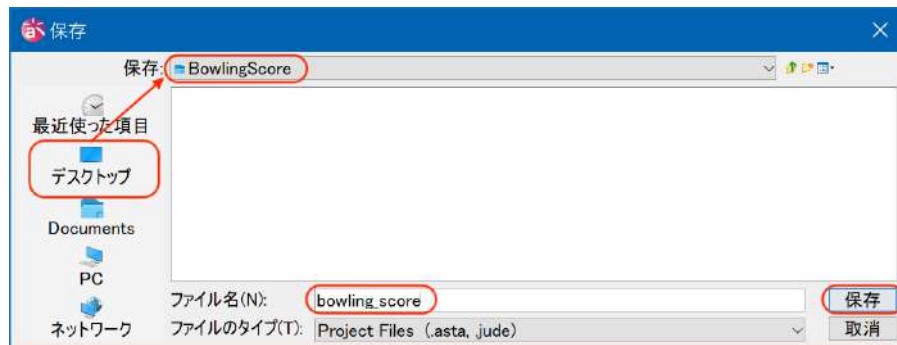


図 1.4 プロジェクトを保存する場所とプロジェクトファイル名を指定する

3. 保存すると、プロジェクトファイルにつけた名前が「構造ツリー」に反映される（図 1.5）。

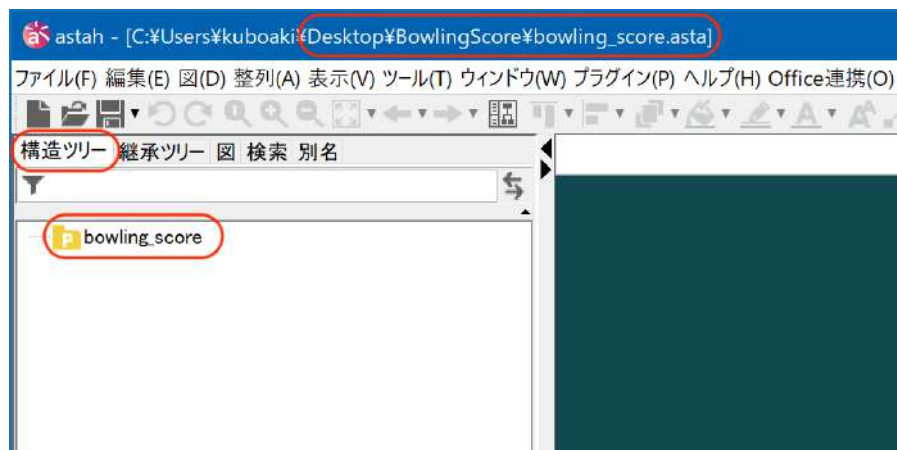


図 1.5 ファイル名がプロジェクト名に反映された

NOTE

astah* のプロジェクトの作り方についての詳しい説明は、機能ガイドの「プロジェクトの作成と利用」を参考にしてください。

astah* 機能ガイド プロジェクトの作成と利用

<https://astah.change-vision.com/ja/manual/278-project-file.html>

1.4 ボウリングのスコアのつけ方

ボウリング場では、だいたい前（1980年代の終わりくらい）から、スコアを自動計算してディスプレイに表示するシステムが使われるようになっていきます。自動化される前は、受付でスコアシートをもらって、自分たちでスコアを計算し、手書きでスコアをつけていました（図 1.6）。



図 1.6 手でスコアを記入している様子

ボウリングの規則とスコアのつけ方について、公益財団法人「全日本ボウリング協会」の「ボウリング競技規則」を参考に、まとめてみました。

公益財団法人「全日本ボウリング協会」のボウリング競技規則

<https://jbc-iwate.com/service/message/201012101.pdf>

1.4.1 ボウリングスコアに使う記号

ボウリングスコアに使う記号と意味を表 1.1 に示します。

表 1.1 ボウリングスコアに使う記号と意味

記号	名称	意味
	ストライク	1投目で10本全てを倒した場合。ストライクをだしたフレームの得点は、次の2投分を加算する。2回続いたら3フレーム目の1投目までを加算する。スコア欄は、それまでは空欄にしておく。ストライクが続くことを「ダブル」「トリプル（ターキー）」と呼ぶ。
	スペア	1投目で残ったピンを、2投目で全部倒した場合。次の1投分を加算する。スコア欄は、それまでは空欄にしておく。
	ミス	フレームの2投目でピンを1本も倒せなかった場合。2投目でガターに落ちても「G」の記号は使わずにこの記号を使う。そのフレームの得点は、1投目で倒したピン数になる。なお、1つのフレームで2回投球し、10本のピンを全部倒すことができなかった場合をエラーという。

記号	名称	意味
G	ガター	1投目でレーンの両脇にある溝（ガター）にボールが落ちた場合。2投目はガターに落ちた場合も「ミス」として扱う。
	スプリット	1投目で1番ピン（ヘッドピンともいう）と他のいくつかのピンが倒れ、2本以上のピンが次のような状態に残った場合。スプリットの場合は、1投目のピン数を○で囲む。（残っているピンの中間のピンが少なくとも1本倒れたとき。例えば7と9あるいは3と10。残っているピンのすぐ前のピンが少なくとも1本倒れたとき。例えば5と6。）
F	ファウル	ファウルラインを越えて投球されたという記号。1投目でファウルした場合、ピンを倒しても1投目のピン数は0になる。2投目は全ピンを立て直す。2投目でファウルした場合、ピンを倒しても2投目のピン数は0になる。第10フレームの3投目がファウルの場合、ピンを倒しても3投目のピン数は0になる。

1.4.2 クラッシュスコアリング

古くから使われていて、広く普及しているスコア計算方式です。

クラッシュスコアリングにおけるスコア計算の規定

- ボウリングの1ゲームは、10個のフレームをもって構成する。
- 競技者は、ストライクの場合を除き、それぞれのフレームで2回ずつ投球する。
 - ただし、第10フレームがストライクまたはスピアの場合、サービスフレームが追加される。
 - ストライクの場合は、サービスフレームで2回投球できる。
 - スピアの場合は、サービスフレームで1回投球できる。
- ゲームの成績は、10個のフレームの合計点によってこれを表す。
 - 適正な投球によって倒されたピンの数をもって計算する。
 - 適正に投球されたボールとは、競技者の持っているボールが、手から放れファウルラインを越えたものをいう。

名前	1	2	3	4	5	6	7	8	9	10	Total
くぼあき	6 3 9	9 — 18	G 3 21	8 38	7 58	 78	9 96	⑧ F 104	 130	6 150	150/150

図 1.7 スコアの例（クラッシュスコアリングの場合）

1.4.3 カレントフレームスコアリング

2018年アジア競技大会で採用された新しいスコア計算方式です。

クラシックスコアリングと異なる点

- 1投目によって10ピンすべてを倒した場合はストライクとなる。ストライクのとき、そのフレームの得点は30となる。
- 続けて2回ストライクの場合は、それぞれのストライクの得点は30となる。
- 1投目の後、残ったピンを2投目によって全部倒した場合は、スペアとなる。スペアのとき、そのフレームの得点は、1投目のピン数に10を加えた数となる。
- 第10フレームでストライクあるいはスペアをとった場合も、3投目はない。

名前	1	2	3	4	5	6	7	8	9	10	Total
うえはら					F 8	9 F		8 -	G 9		202/202
	30	60	90	108	116	125	155	163	172	202	

図 1.8 スコアの例（カレントフレームスコアリングの場合）

NOTE

ボーリングのハンディキャップは、参加者が普段プレーしているときのアベレージ（平均スコア）を基に、力量に応じて設定するそうです。このチュートリアルでは、ハンディキャップは取り扱わないでおきます。

【コラム1】 ボウリングの名前の由来

「ボウリングをすること」などを意味する「bowl」という英語は、ラテン語で「泡」や「瘤」を意味する「bulla」に由来する。一方、同じ綴りで食器や容器（ボウル）を意味する「bowl」や「球」を意味する「ball」は、ゲルマン語に由来し、本質的に異なる。

— Wikipedia

1.4.4 スコアの付け方をもう少し詳しく

このチュートリアルで作成するプログラムで実現する処理でもありますので、ボウリングのスコアの付け方についてももう少し詳しく確認しておきましょう。

ボウリングの基本ルール

ボウリングの1ゲームは、10フレームで構成されています。プレイヤーは1フレームに最大2回投球できます。

- 1投目で10本すべてのピンを倒した場合は「ストライク」（そのフレームは終了）。

- ・ 2投で10本すべてのピンを倒した場合は「スペア」。
- ・ 2投しても10本倒せなかった場合は「ミス」。

スコアの計算方法

ストライクやスペアの場合は、倒したピン数だけでなく、ボーナス点が加算されます。

ストライクの場合

- ・ 10点 + 次の2投で倒したピン数がボーナス点として加算される。
 - 例: ストライク (10点) + 次の1投目 (7点) + 次の2投目 (2点) = 19点

スペアの場合

- ・ 10点 + 次の1投で倒したピン数がボーナス点として加算される。
 - 例: スペア (5+5=10点) + 次の1投目 (7点) = 17点

ミスの場合

- ・ 倒したピン数の合計がそのフレームのスコアになる。
 - ボーナス点はなし。

10フレーム目の特別ルール

10フレーム目は特別なルールがあります。

- ・ ストライクまたはスペアの場合、追加で1投または2投できる。
- ・ これにより、10フレーム目は最大3投できる可能性がある。

スコア計算の例

表 1.2 にスコアの計算例を示します。

表 1.2 スコア計算例

フレーム	投球結果	フレームスコア	累計スコア
1	7, 0	7	7
2	5, 5 (スペア)	10 + 次の1投(10) = 20	27
3	10 (ストライク)	10 + 次の2投(10+5) = 25	52
4	10 (ストライク)	10 + 次の2投(5+4) = 19	71
5	5, 4	9	80

このように、ボウリングのスコア計算は、単純な足し算ではなく、「ストライクとスペアでは異なる方法で次のフレームの結果を待つ」という状態管理が必要になります。

この計算ルールをUMLでモデル化し、C言語で実装するのが、このチュートリアル目標です。

第2章 スコアシートの構造を調べる

ボウリングのゲームスコアを記録するスコアシートは、どのような要素で構成されているでしょうか。また、それらの構成要素の間にはどのような関係があるのでしょうか。構成要素や構成要素の関係を表すには「構造のモデル」を使います。構造のモデルを使って、スコアシートの構成要素や、構成要素間の関係を検討してみましょう。

NOTE

このチュートリアルでは、クラシックスコアリング（1.4.2）の場合について考えることにします。

2.1 スコアシートの例

ゲーム中のスコアシートの例を図 2.1 に示します。この例では、2人でプレーしています。いまは、2ゲーム目のプレー中で、1人目の6フレーム目の第1投目までゲームが進んでいます。また、彼らは1フレームずつ交代で投球していることがわかります。もし、彼らがもう1ゲーム分プレーする場合には、このスコアの下にもう一度2人分のスコアが追加されるでしょう。

スコアシートのサンプル (クラシックスコアリング)

プレイ開始: 2022年02月04日 16時03分 ← プレイ開始日時

名前	1	2	3	4	5	6	7	8	9	10	Total				
くぼあき	7	—	5			5	4		7	5	4	155			
	7	27	52	71	80	100	115	124	141	155					
うえはら	6	3	9	—	G	3	8		7		(8)	—	6	3	132
	9	18	21	38	58	78	96	104	123	132					
名前	1	2	3	4	5	6	7	8	9	10	Total				
くぼあき	6	3	9	—	G	3	8		7		3				51
	9	18	21	38	51										
うえはら			8		7	2									46
	20	37													

プレイヤー

フレーム

ターン

スコアシート

ゲーム

図 2.1 スコアシートの例

NOTE

ボウリングでは、「ゲーム」が、各自が10フレーム分プレーした結果を指す場合と、複数名で10フレーム分プレーしたひと組の結果を指す場合があります。これら呼び分ける方法がわからなかったため、このチュートリアルでは、前者を「スコア」、後者を「ゲーム」と呼ぶことにします。

実際のボウリング競技において、これらの呼び分方をご存じの方は、その呼び分け方を使うとよいでしょう。

スコアシートを観察してわかることをまとめました。

スコアシートを観察してわかること（決めたこと）

- スコアシートには、プレー開始日時が記載されている。
- スコアシートには、プレーヤー名とその人の10フレーム分の投球を記録する欄がある。
 - これを「**スコア**」と呼ぶことにする。
- 1つのゲームの進行中は、1フレームごとにプレーヤーが交代して投球する。
 - この方式は「**ベーカー方式**」と呼ばれている。
- スコアシートには、複数人のスコアが記録されている。
 - 同時に進行している複数名のスコアのまとまりを「**ゲーム**」と呼ぶことにする。
- プレーヤーの組を単位として複数回ゲームをプレーできる。

これらを元に、スコアシートの構成要素や構成要素の間の関係をモデル図で表してみましょう。

2.2 スコアシートの構造をオブジェクト図で表す

スコアシートの構造を調べるには、まず実際のスコアシートがどうなっているかを調べるのがよいでしょう。スコアシートに記入されている具体的なデータを使って図を作成するには、オブジェクト図が向いています。

それでは、スコアシートの例を参照しながら、スコアシートのオブジェクト図を作成してみましょう。

2.2.1 プロジェクトに設計モデルを追加する

まず、astah* で作成したプロジェクトに「設計モデル」を追加しましょう。

NOTE

「設計モデル」ということばが表すものは、開発に使用する手法や分野によって少しずつ異なります。このチュートリアルでは、次の2つの側面について表したものを「設計モデル」と呼ぶことにします。

- 対象とする業務やサービスに必要となる構成要素やその関係を「**構造のモデル**」（静的モデル）として表したもの。
- 業務やサービスを実現するために必要となる動作を「**振る舞いのモデル**」（動的モデル）として表したもの。

TIP

設計に関わる構成要素は、対象とする業務やサービスを分析して得られることもあれば、開発に採用する資源や方式から得られることもあります。

プロジェクトに設計モデルを追加する

- ・ 構造ツリー上で、プロジェクトを選択する。
- ・ 右クリックしてポップアップメニューを開き、「モデルの追加>モデル」を選択する（図 2.2）。



図 2.2 プロジェクトにモデルを追加した

追加したモデルに名前をつける

- ・ 構造ツリー上で、追加したモデルを選択した状態で、プロパティの「ベース」タブを選択する。
- ・ 「名前」を編集して「設計モデル」とする（図 2.3）。
- ・ 入力が確定すると、構造ツリーの表示にも反映される。

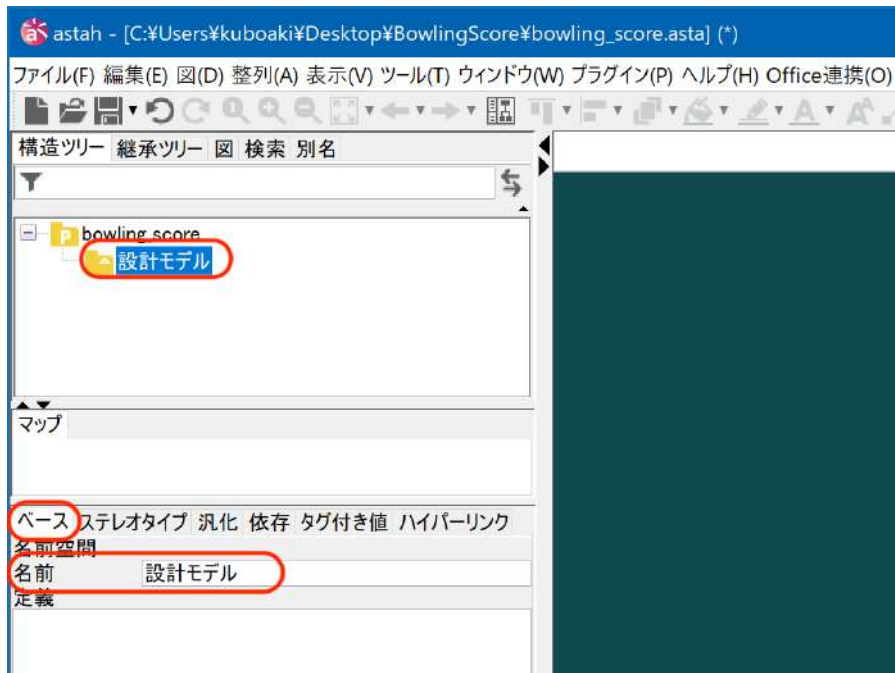


図 2.3 モデルに設計用のモデルとして名前をつけた

2.2.2 設計モデルにオブジェクト図を追加する

「オブジェクト図（インスタンス図と呼ぶこともあります）」を使ってスコアシートに登場するオブジェクトを表してみましょう。astah*でオブジェクト図を作成するときは、「クラス図」を使います。

それでは、プロジェクトにクラス図を追加して、オブジェクト図を描いてみましょう。

設計モデルにオブジェクト図を追加する

まず、プロジェクトにオブジェクト図を描くのに使うクラス図を追加します。

1. 構造ツリーから設計モデルを選択し、右クリックしてポップアップメニューを開く（図 2.4）。
2. 「図の追加>クラス図」でクラス図が追加される。

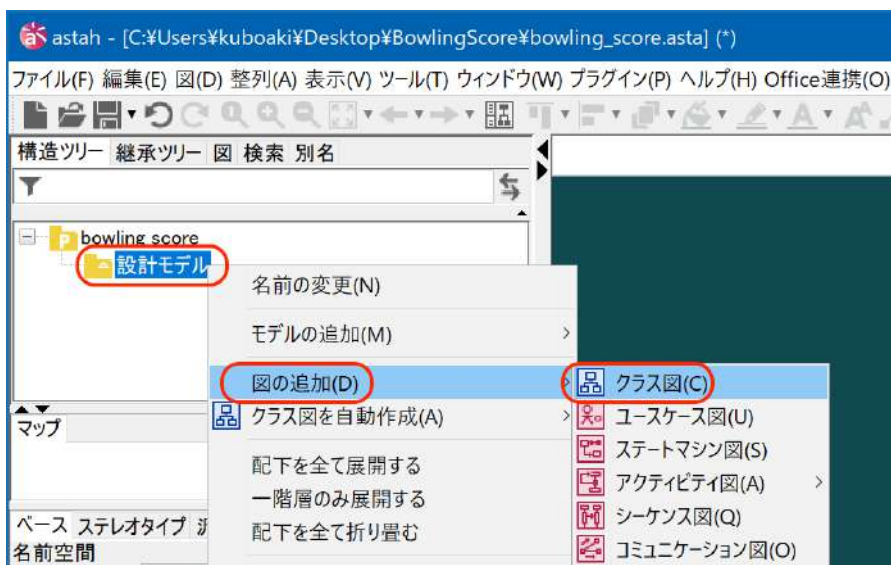


図 2.4 モデルにクラス図を追加する

オブジェクト図に名前をつける

追加したオブジェクト図に、図の名前を設定します。描こうとしているのはオブジェクト図ですので、それがわかるような名前をつけます。

- ・追加したクラス図のプロパティの「ベース」タブを開く。
- ・名前を編集して「ゲームスコアのオブジェクト図」とする（図 2.5）。
- ・ダイアグラムエディタのタイトルやタブにも反映される。

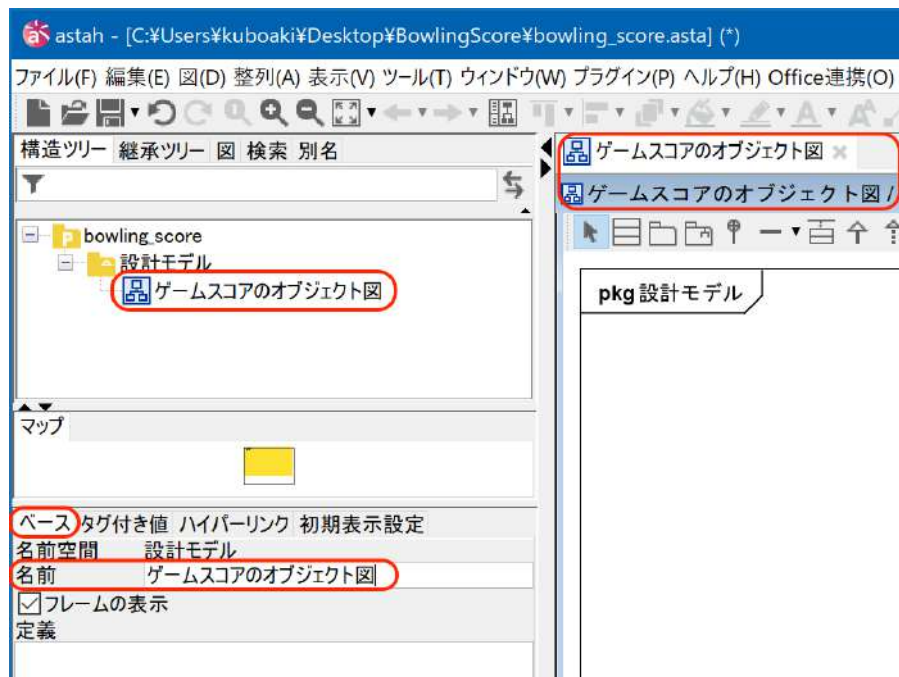


図 2.5 図の名前を「ゲームスコアのオブジェクト図」にする

2.2.3 オブジェクト図にスコアシートを追加する

図 2.1 や「スコアシートを観察してわかること（決めたこと）」を観察しながら、スコアシートに記載されている要素をオブジェクト図に追加してみましょう。まず、「スコアシート」オブジェクトを追加します。

スコアシートを表すオブジェクトを追加する

こんどは、図にスコアシートを表すオブジェクトを追加してみましょう。

- ・パレットから「インスタンス仕様」を選択して、図に配置する（図 2.6）。
- ・「インスタンス仕様0」という名前のオブジェクトが配置される（末尾の数字は作るたびに新しいものになる）。

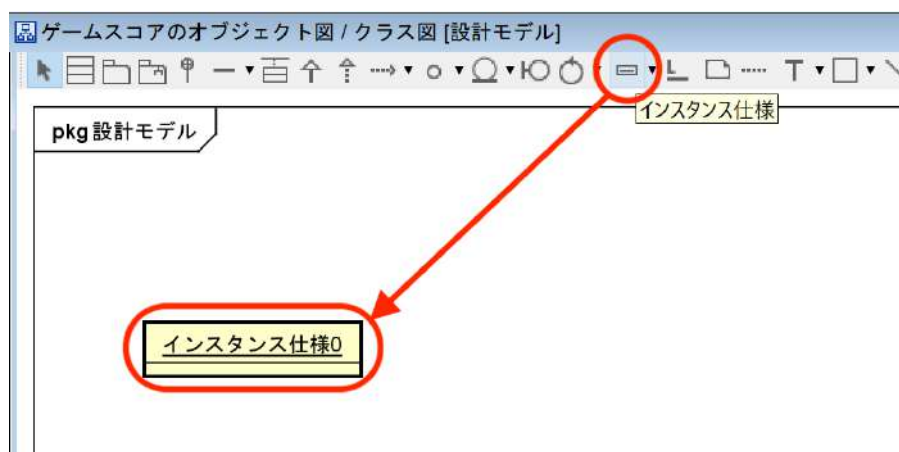


図 2.6 インスタンス仕様を追加する

オブジェクト名を個別のスコアシート名に変える

実際のスコアシートでは、個々のスコアシートを区別するようなはっきりとした名前はないこともあるでしょう。スコアシート番号みたいな通番をつけている場合もあるかもしれません。

ここでは、追加したオブジェクトが特定のスコアシートを表すことがわかるよう、他のシートと区別できるような名前（オブジェクト名）をつけましょう。

- 追加したオブジェクトを選択し、プロパティから名前の編集欄を選択する（図 2.7）。
- オブジェクトの名前に、図 2.1 のスコアシートを示す固有の入力する。ここでは「scoresheet01」とした。
- オブジェクトの名前の入力を確定すると、ダイアグラムエディタ中のオブジェクトの名前にも反映される。

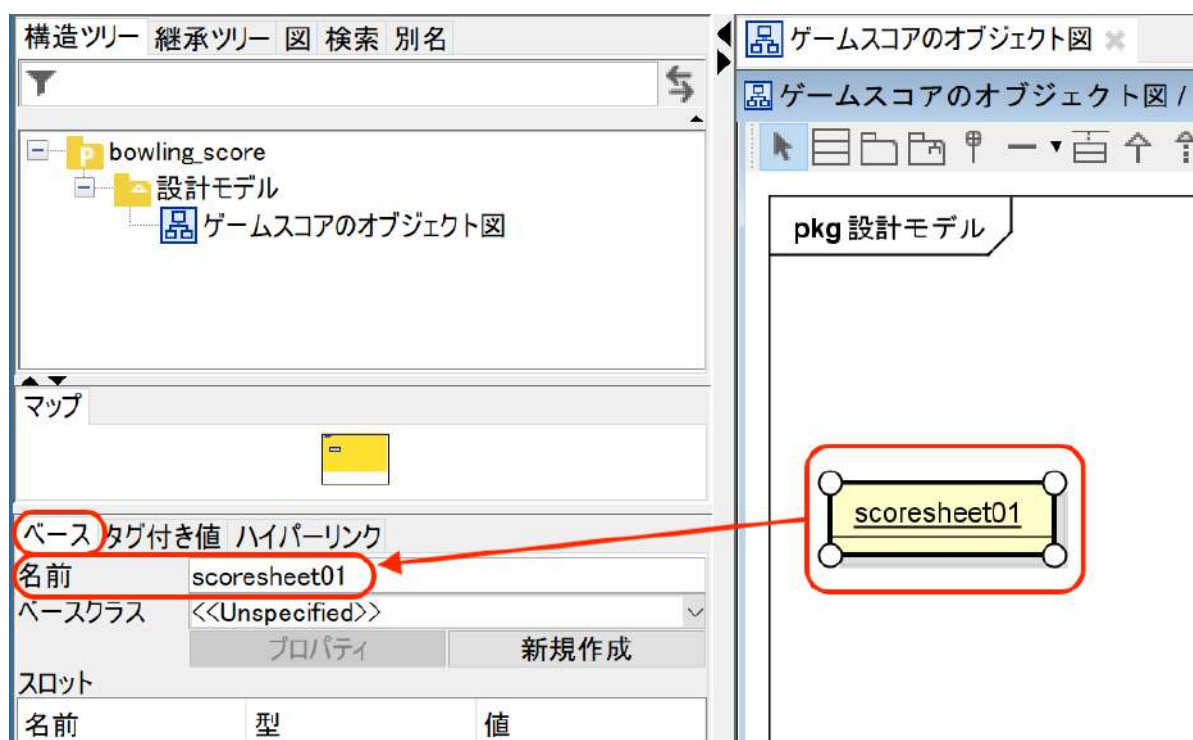


図 2.7 名前を「scoresheet01」に変更する

TIP

個々のオブジェクトを識別するためにつける名前のことを、オブジェクト名（またはインスタンス名）と呼びます。

2.2.4 スコアシートオブジェクトにクラスを割り当てる

追加したオブジェクトは、名前はつけたものの、たくさんあるスコアシートの仲間であるとみなす方法がありません。それは、このオブジェクトがどんなクラスに属するか決まっていないからです。それが分かるよう「スコアシート」を表すクラスを定義して割り当てておきましょう。

スコアシートクラスを追加してオブジェクトに割り当てる

まず、図中のオブジェクトに割り当てるクラスを用意します。

- ・オブジェクト「scoresheet01」を選択した状態で、プロパティから「新規作成」ボタンをクリックする（図 2.8）。

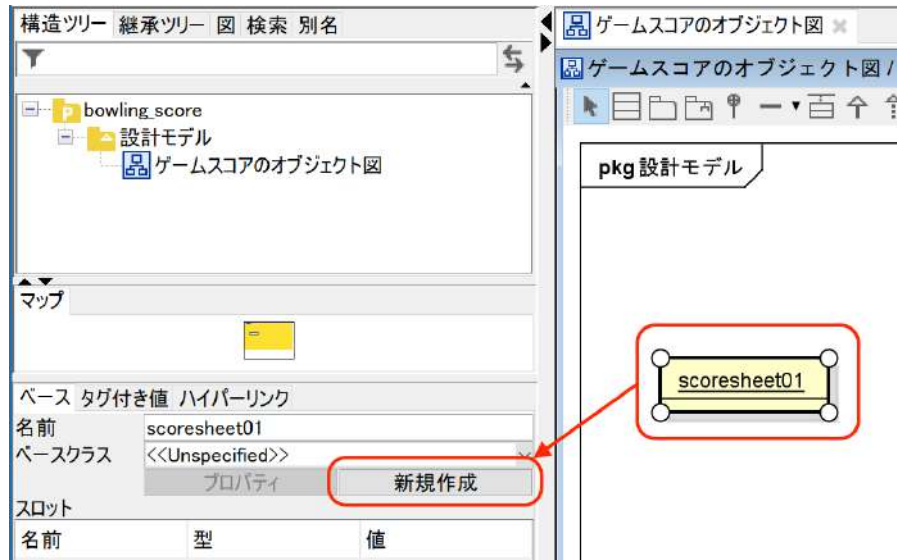


図 2.8 オブジェクトを選択してクラスを「新規作成」する

- ・クラスを定義するダイアログが表示される（図 2.9）。
- ・「ベース」タブを選択し、名前に「ScoreSheet」を入力する。

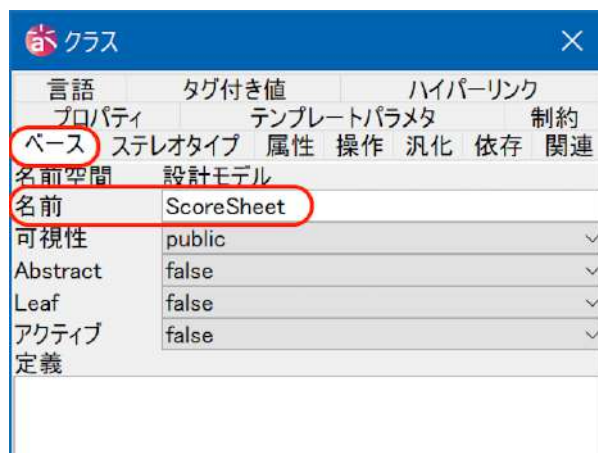


図 2.9 クラス「ScoreSheet」を定義する

スコアシートに属性を追加する

スコアシートクラスがどのような情報を保持するのかはまだ検討が必要ですが、ひとまずプレー開始日時を保持することは必要そうです。そこで、プレー開始日時を表す属性を追加しましょう。ここでは、プレー開始日時を表す属性の名前を「play_date」とします。

- ・スコアシートクラスの「属性」タブを選択する。
- ・「+」ボタンを押すと属性が追加されるので、名前に「play_date」を入力する（図 2.10）。

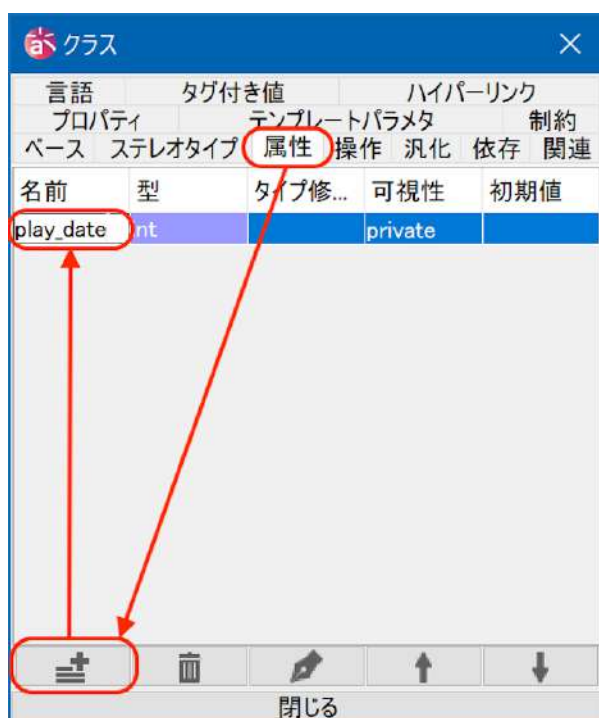


図 2.10 属性「play_date」を追加する

追加した属性の型を定義する

追加した属性も、どんな型なのかを設定しておきましょう。プレー開始日時を表す属性の型の名前を「Time」とします。

NOTE

「Time」は、日付と時刻を操作するためのクラスです。第3章において、このクラスを、`time.h`を通じて利用できるC言語のライブラリに対応づけする見込みで割当てています。

- ・追加した属性の「型」欄を選択する。
- ・「型」欄を編集状態にして「Time」を入力すると、ダイアログに「型になるTimeを新規作成しますか?」というメッセージが表示される（図 2.11）。
- ・「はい」をクリックしてダイアログを閉じる。
- ・クラス「Time」が作成される。（図 2.12）。
- ・構造ツリーにも追加されたことを確認する。

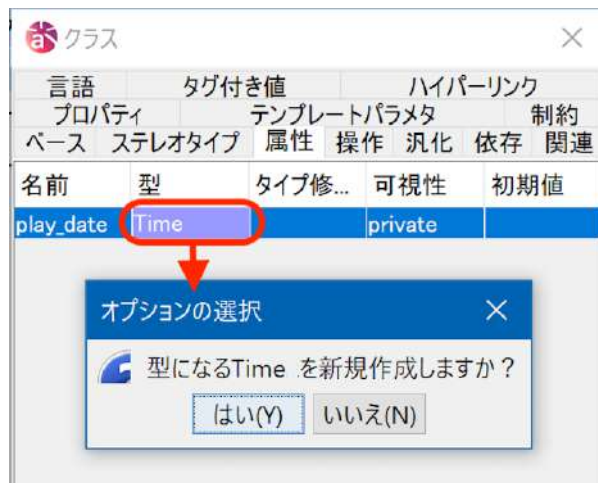


図 2.11 クラス「Time」の追加を促すダイアログ



図 2.12 クラス「Time」が追加された

オブジェクト図や構造ツリーを確認する

オブジェクト図においても、構造ツリーにおいても、追加したオブジェクトの名前が変わっているを確認します。

- ・「閉じる」をクリックして、クラス定義のダイアログを閉じる。
- ・オブジェクトの表示が「scoresheet01: ScoreSheet」に変わっている。
- ・属性「play_date」の属性値を保持する欄（スロットと呼ぶ）も追加されている。
- ・構造ツリーにもTimeクラスが追加されている（図 2.13）。

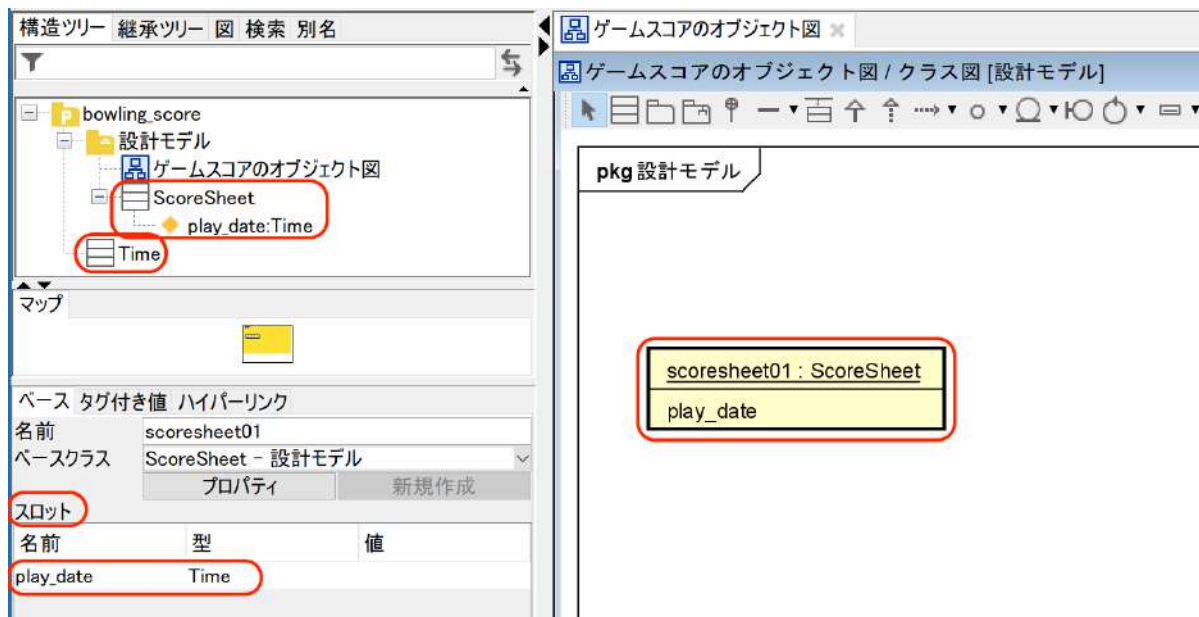


図 2.13 オブジェクトにクラスやスロットが割当てられた

追加したスロットに属性値を設定する

追加した属性「play_date」のインスタンス（スロットと呼びます）に、属性値として具体的な日付データを設定します。

- ・ オブジェクト「scoresheet01」を選択して、プロパティから「ベース」タブを開く。
- ・ 属性「play_date」の値に日時、たとえば「2022/02/04 16:18」などを入力する（図 2.14）。

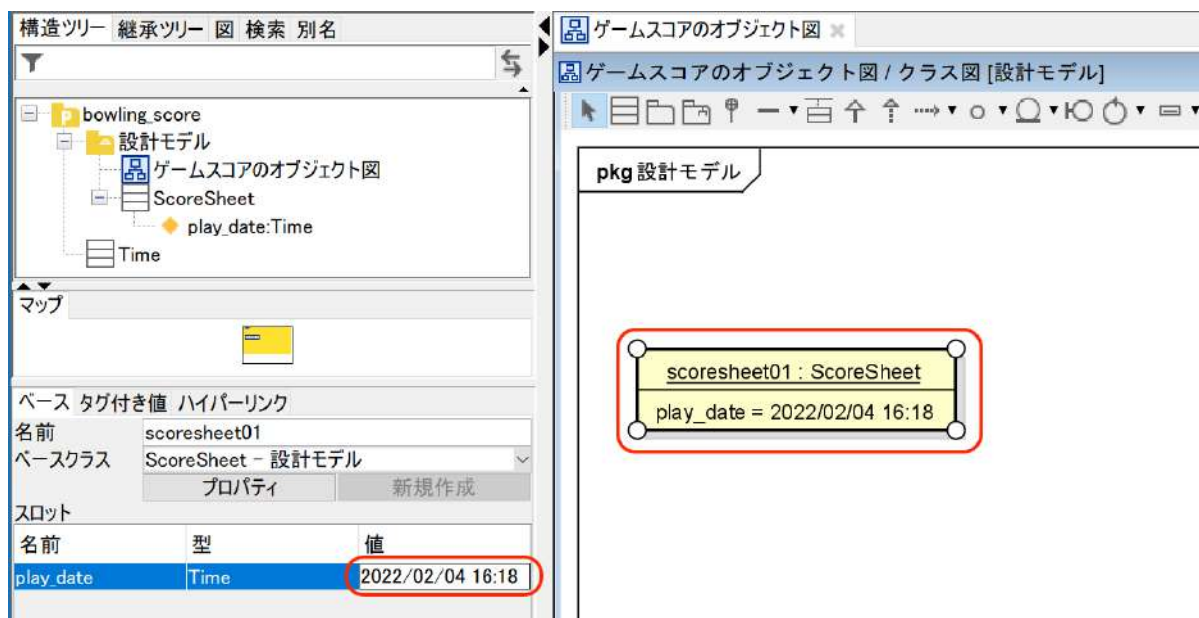


図 2.14 オブジェクトのスロットに属性値が追加された

2.2.5 オブジェクト図にゲームを追加する

次に、「スコアシート」を追加したのと同じ手順で「ゲーム」オブジェクトを追加します。図 2.1 の場合ゲームが2組あるので、ゲームのオブジェクトを「game01」、「game02」としましょう。

クラス名は「Game」としましょう。

ゲームを表すオブジェクトを追加する

最初の（1組目の）ゲームを追加してみましょう。

- ・パレットから「インスタンス仕様」を選択して図に配置する。
- ・オブジェクト名を「game01」とする。
- ・クラスを追加して「Game」とする（図 2.15）。



図 2.15 ゲームを表すオブジェクト「game01」とクラス「Game」を追加した

次のゲームのオブジェクトを追加する

次のゲーム（つまり、2組目）のオブジェクトも追加してみましょう。

- ・同様の手順で「game02」を追加する。
- ・プロパティから既存のクラスをプルダウンし、「Game」クラスを選択する（図 2.16）。

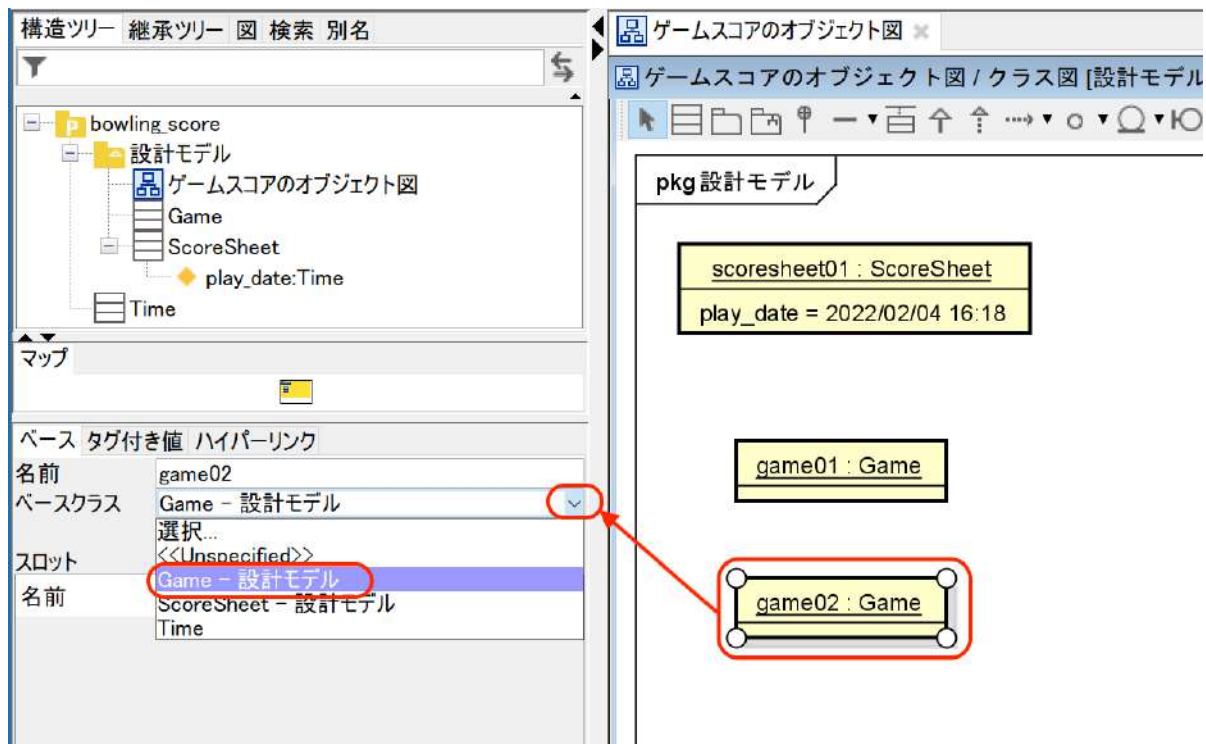


図 2.16 ゲームを表すオブジェクトを追加し、既存のクラスを割当てた

2.2.6 オブジェクト図にスコアを追加する

こんどは、プレイヤーひとり分のスコアを記録しているスコア部分を追加しましょう。図 2.1 の場合、スコアは4つあります。

このチュートリアルでは、プレイヤー名はスコアの属性と考えることにしておきます。（もちろん、プレイヤーを独立したクラスと考え、複数のスコアとを関連づけた方がもっとよいでしょう）

それぞれのプレイヤーのスコアを表すオブジェクトを追加する

それぞれのプレイヤーごとにスコアがありますので、まず、プレイヤーごとのスコアのオブジェクトを追加します。

- パレットから「インスタンス仕様」を選択して図に配置して「score01」とする。
- 「Score」クラスを追加して、「score01」に割当てて（図 2.17）。



図 2.17 スコアを表すオブジェクト「score01」を追加した

- ・「Score」クラスに属性「player」を追加し、「String」クラスを追加して割当てする（図 2.18）。

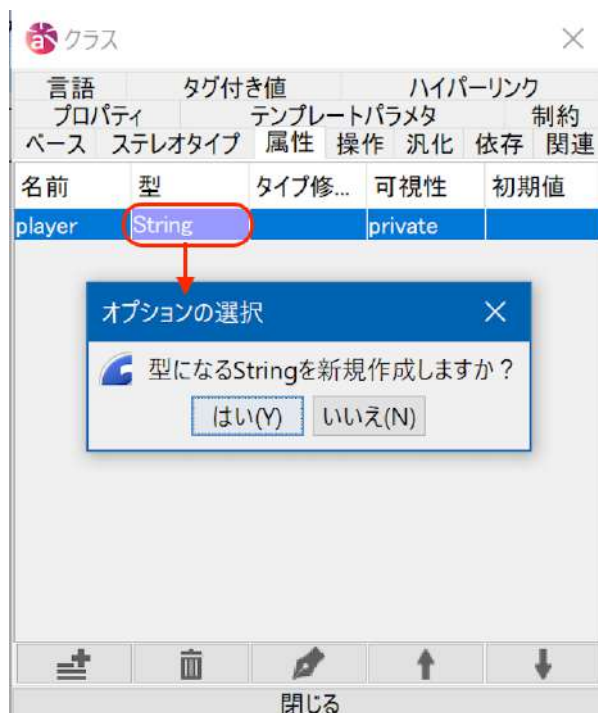


図 2.18 属性「Player」を追加し、クラス「String」を追加して割当てする。

追加したオブジェクトのスロットに値を設定する

追加したオブジェクトの「player」スロット（スコアクラスの属性playerの、このオブジェクトにおける属性値）を設定します。

- ・「score01」のスロット「player」の値に「くぼあき」を設定する（図 2.19）。



図 2.19 「player」の Slots の値にプレイヤー名を設定した

- ほかのスコアのオブジェクトも作成する（図 2.20）

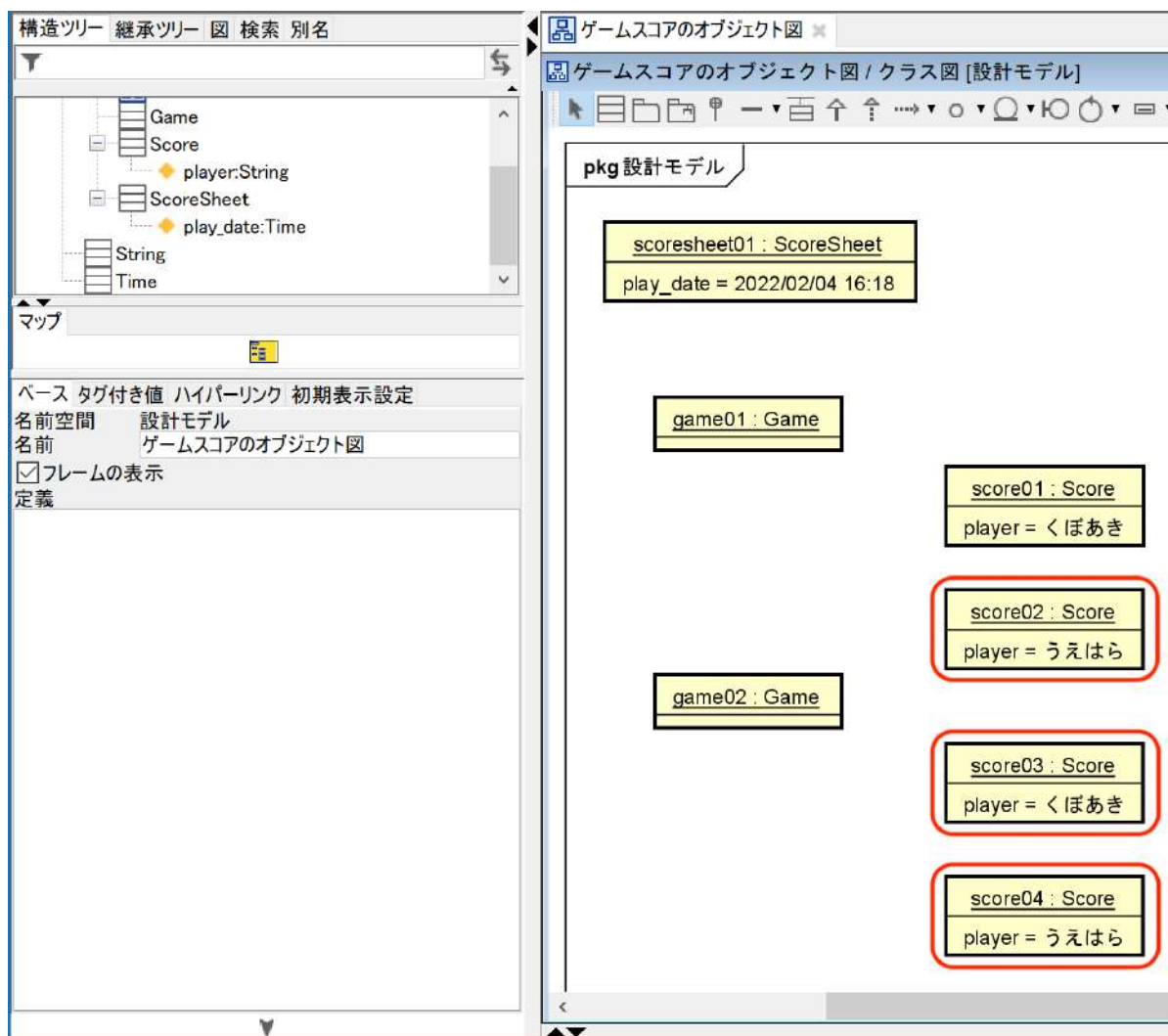


図 2.20 残りのスコアのオブジェクトを作成した

2.2.7 オブジェクト図にフレームを追加する

次に、各スコアに記載されているフレームを追加しましょう。やはり、図 2.1 や「スコアシートを観察してわかること（決めたこと）」を観察しながら、作成します。ですが、フレームのオブジェクトの数が多いので、この場で作成してみるのはいずれにします。

それぞれのスコアのフレームを表すオブジェクトを追加する

それぞれのスコアは、フレームを持っていますので、これを表すオブジェクトを追加します。

- ・パレットから「インスタンス仕様」を選択して図に配置して「frame0101」とする。
- ・「Frame」クラスを追加して、「frame0101」に割当て（図 2.21）。
- ・「Frame」クラスの属性に「frame_no」、「first」、「second」、「spare_bonus」、「strike_bonus」、「total」を追加する。

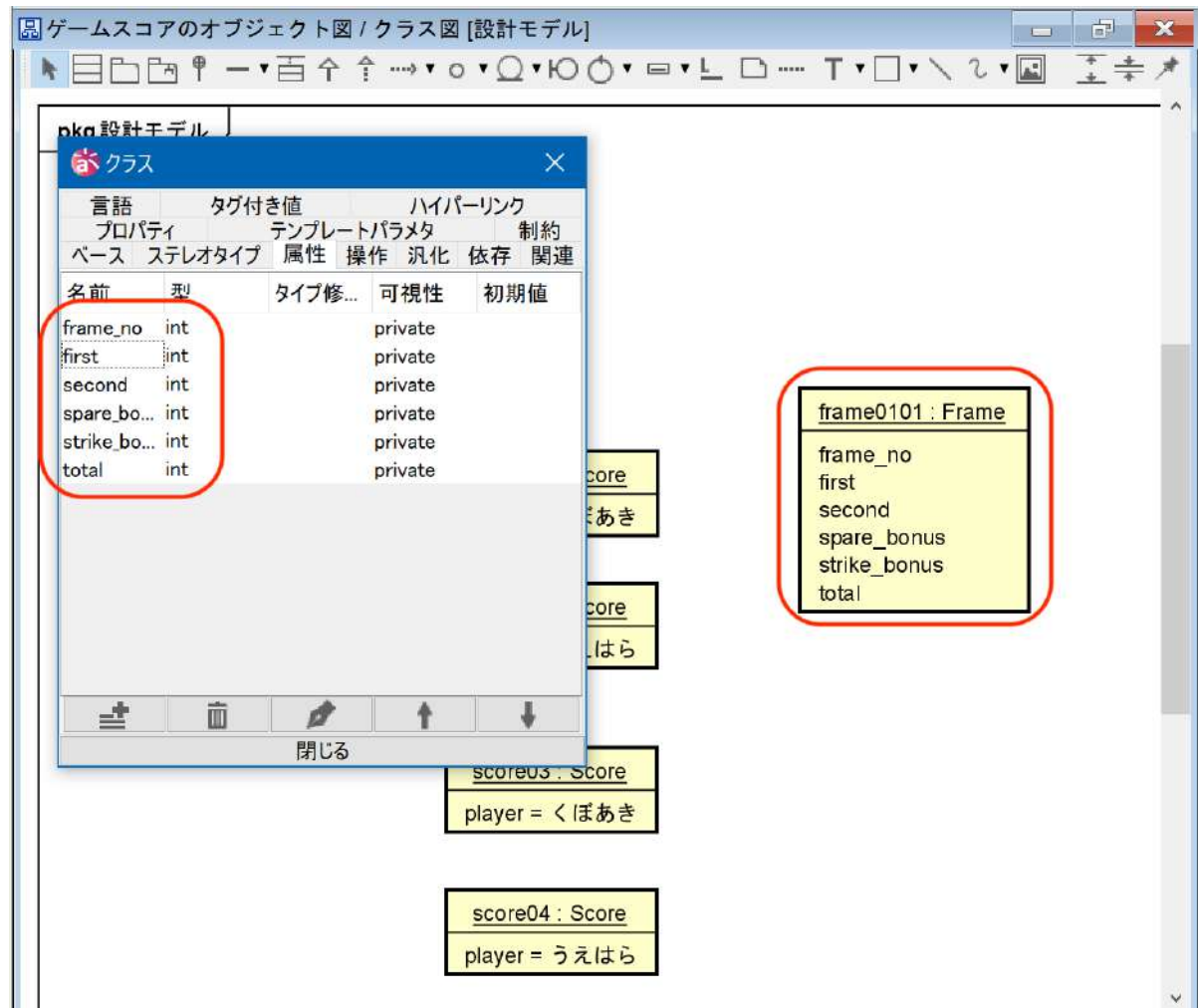


図 2.21 フレームを表すオブジェクト「frame0101」を追加した

追加したオブジェクトのスロットに値を設定する

オブジェクト「frame0101」の各スロットに値を設定します。

- ・1組目のゲームの「くぼあき」さんの第1フレームは、1投目7ピン、2投目ミス（0ピン）。
- ・第1フレームのトータルは7ピン。ボーナスはなし（図 2.22）。

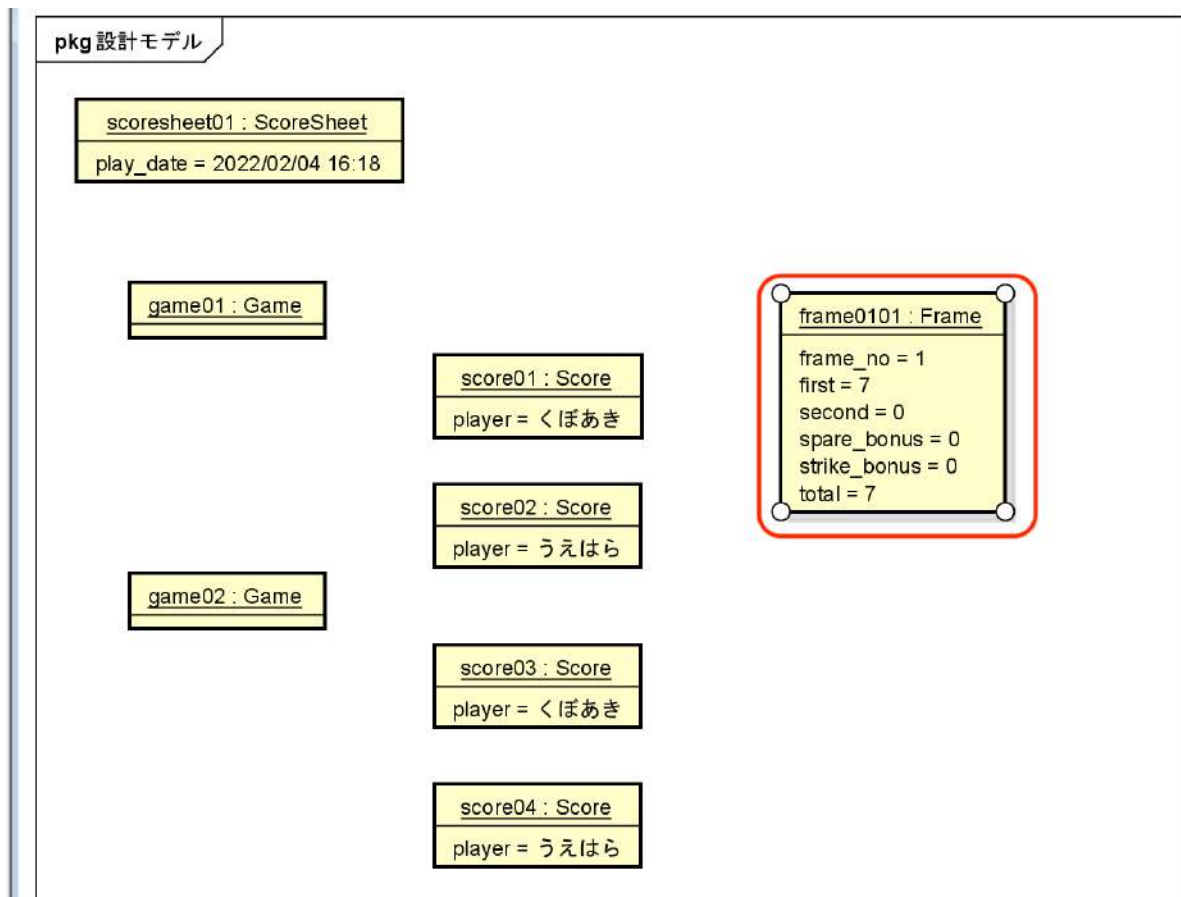


図 2.22 オブジェクト「frame0101」の各スロットの値を設定した

- ・ 同様にして、ほかのフレームのオブジェクトも作成する（図 2.23）。

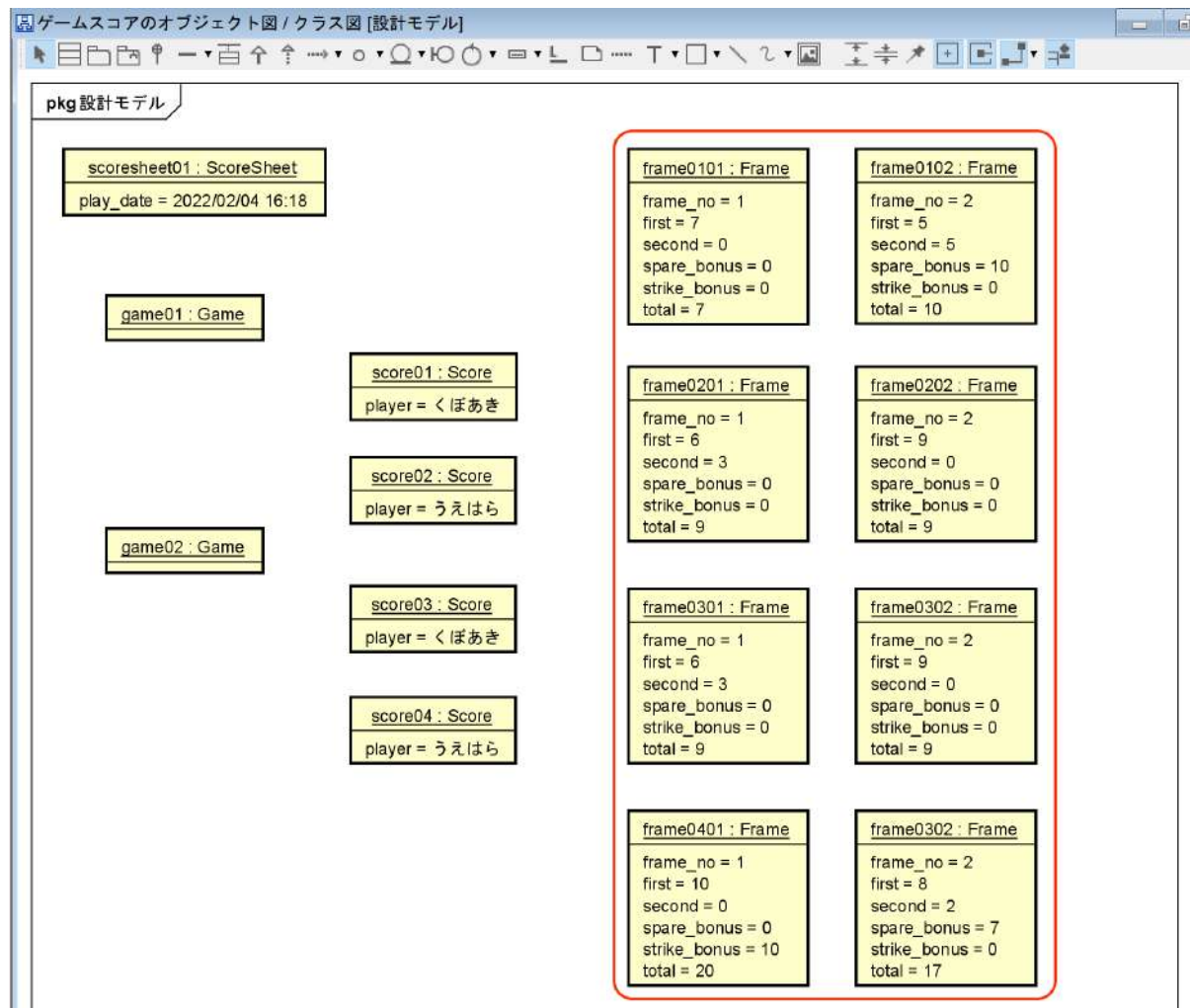


図 2.23 残りのフレームのオブジェクトを作成した（一部だけ）

2.2.8 オブジェクトのつながりを整理する

オブジェクト図に、スコアシート、ゲーム、スコアのオブジェクトが追加できました。図 2.1 を見ながら、これらの間にはどのようなつながりがあるか考えてみましょう。

オブジェクト同士のつながりを考える

- スコアシートは、複数のゲームを記録できるので、スコアシートとゲームにはつながりがありそうです。
- ゲームには、複数のプレイヤーのスコアを記録するので、ゲームとスコアにはつながりがありそうです。
- スコアには、フレームごとのピン数などを記録するので、スコアとフレームにはつながりがありそうです。

このようなオブジェクトの間のつながりを、目に見えるようにしてみましょう。

オブジェクト図では、オブジェクト間のつながりは、「リンク」と呼ぶ線でつないで表します。これは、クラス図におけるクラス間のつながりを表す「関連」のインスタンスにあたるものです（クラス図は、オブジェクト図を元に作成します）。

まず、スコアシートとゲームの間にリンクを引きましょう。

作成したオブジェクトの間にリンクを引く

まず、「scoresheet01」から「game01」へリンクを引いてみましょう。

- ・パレットから「リンク」を選択する。
- ・「scoresheet01」の内部へマウスカーソルを移動し、青枠が表示されるのを待つ（図 2.24）

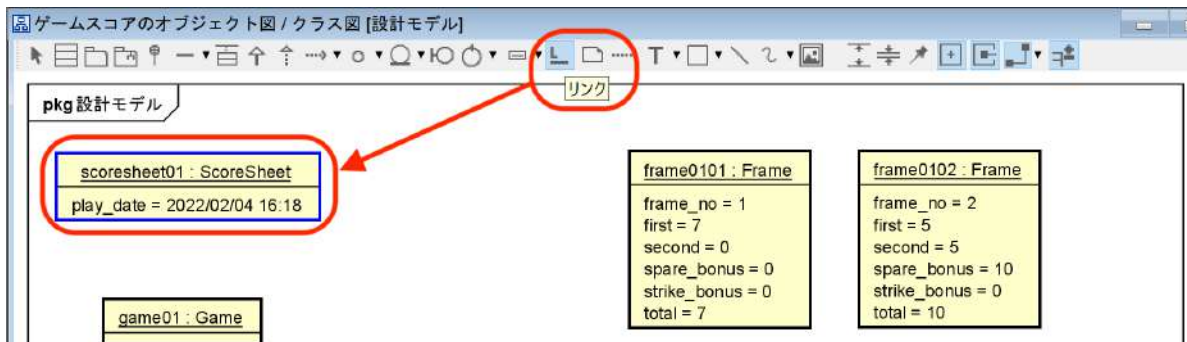


図 2.24 リンクの引き始めのオブジェクトで青枠を表示させる

- ・青枠が表示されたら、マウスのボタンを押したまま「game01」の内部へマウスカーソルを移動する（図 2.25）。

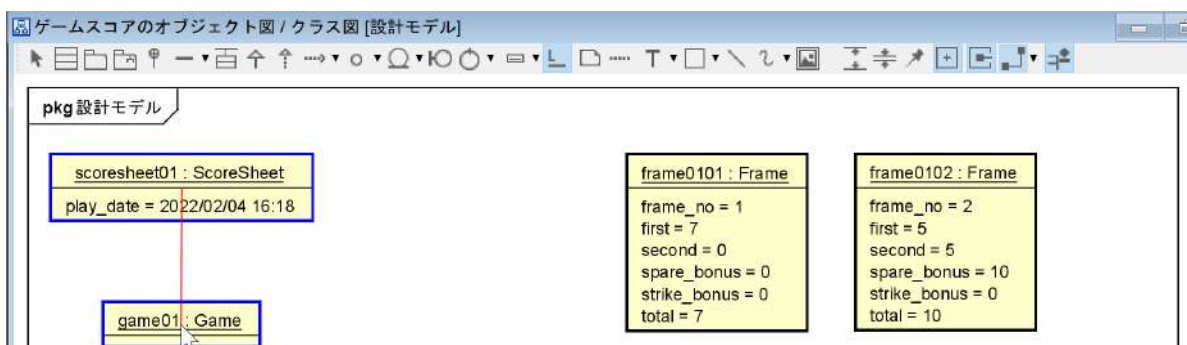


図 2.25 青枠が表示されたらマウスのボタンを押したままマウスカーソルをドラッグする

- ・「game01」にも青枠が表示されたら、マウスのボタンを離すとリンクが引かれる（図 2.26）。

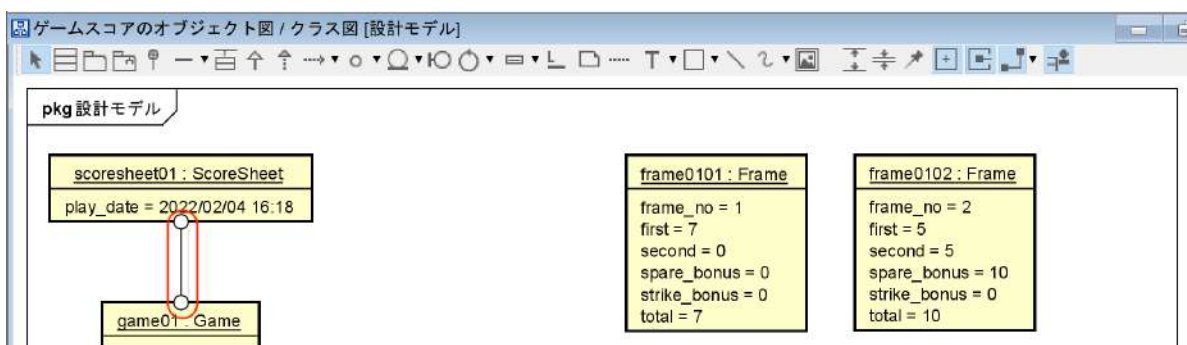


図 2.26 リンク先のオブジェクトにも青枠が表示されたらマウスのボタンを離す

ほかのオブジェクトの間にもリンクを引く

「game02」へも 2.2.8.1 と同様の手順でリンクを引きましょう（図 2.27）。

- 1ゲーム目のスコアは、1つ目のゲームにリンクを引く。
- 2ゲーム目のスコアは、1つ目のゲームにリンクを引く。

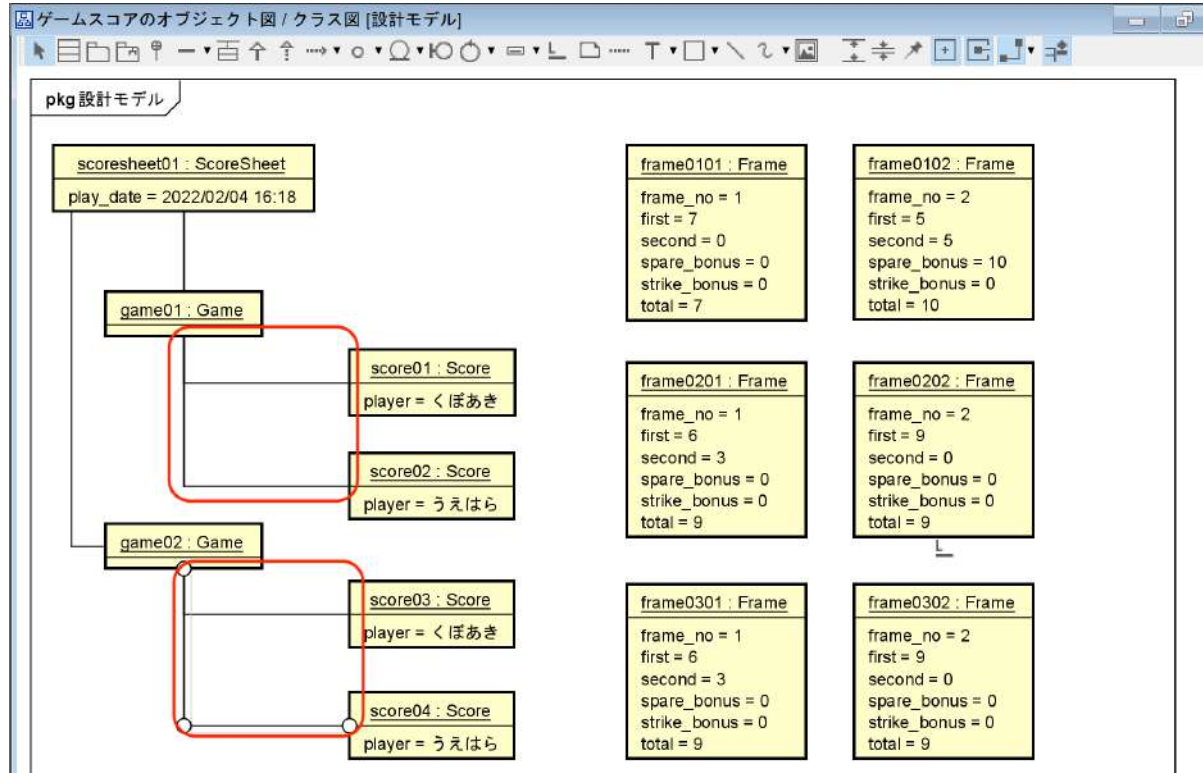


図 2.27 ゲームからスコアへリンクを引く

- スコアからフレームへもリンクを引きます（図 2.28）。

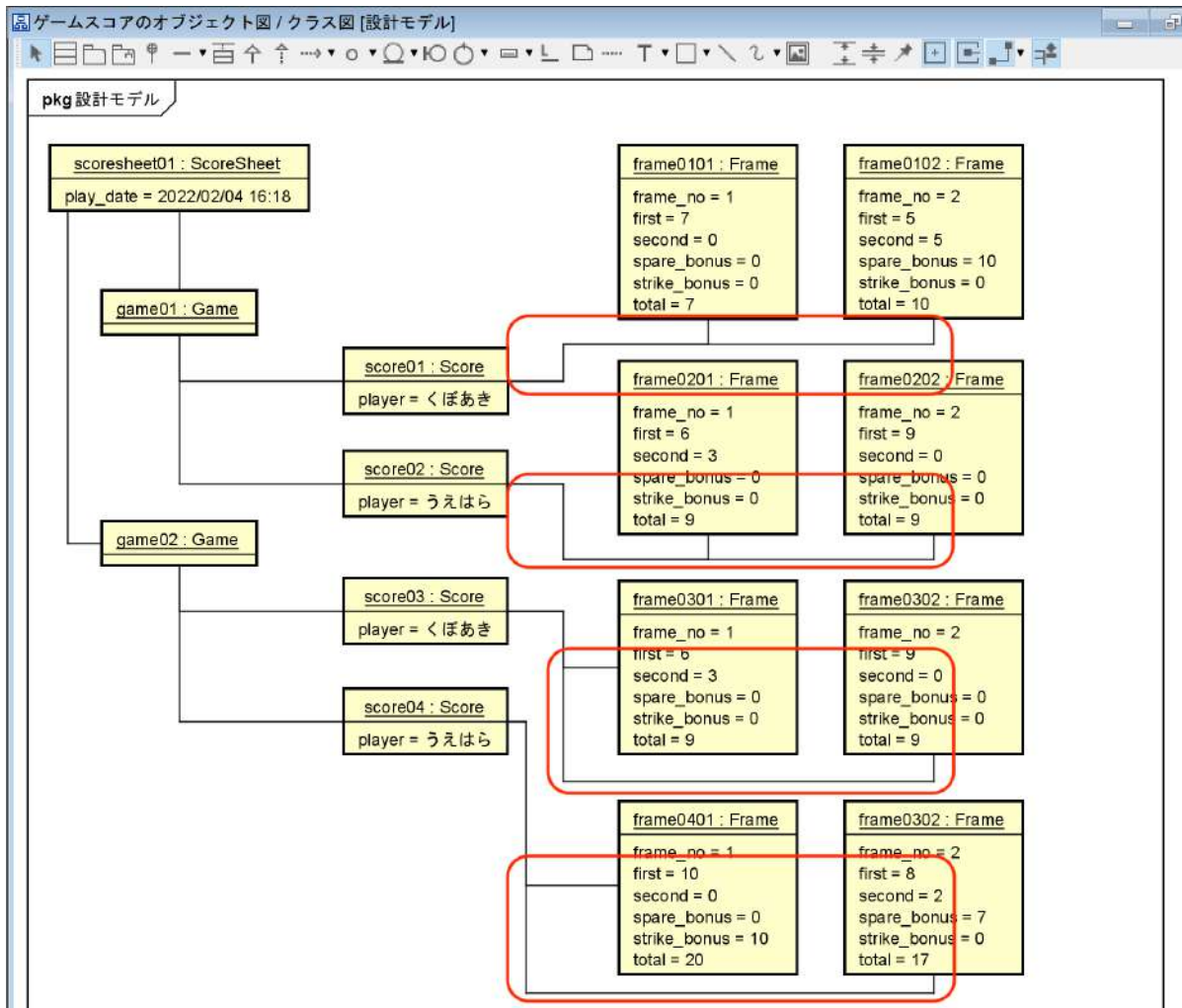


図 2.28 残りのつながりについてリンクを引く

これで、図 2.1 の要素を反映したオブジェクト図が作成できました。

2.3 スコアシートの構造をクラス図で表す

「2.2」では、実際のスコアシートがどうなっているかを調べるために、スコアシートに記載された内容を使ってオブジェクト図を作成しました。しかし、特定のスコアを記載した図では、プログラムで扱うような、より一般的なスコアシートを扱えません。代わりに、オブジェクト図の作成時に考えたクラスを使ったクラス図を考えます。そこで、作成したオブジェクト図 図 2.28 を元に、ゲームスコアのクラス図を作成してみましょう。

2.3.1 クラス図とスコアシートクラスを追加する

設計モデルに、「ゲームスコアのクラス図」と「スコアシートクラス」を追加しましょう。

スコアシートのクラス図を追加する

まず、「ゲームスコアのクラス図」を追加します。

- ・ 構造ツリーで、「設計モデル」でポップアップメニューを開き、「図の追加>クラス図」でモデルにクラス図を追加する（図 2.29）。

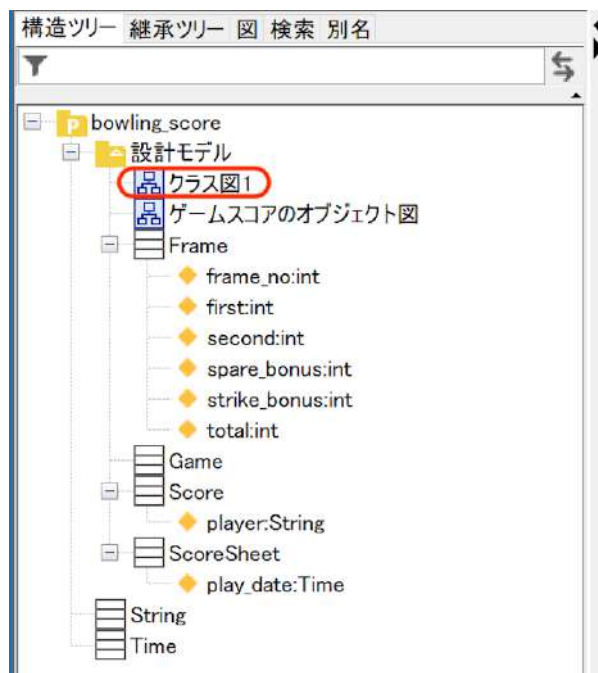


図 2.29 モデルにクラス図を追加する

- ・ 追加した図を「ゲームスコアのクラス図」とする（図 2.30）。

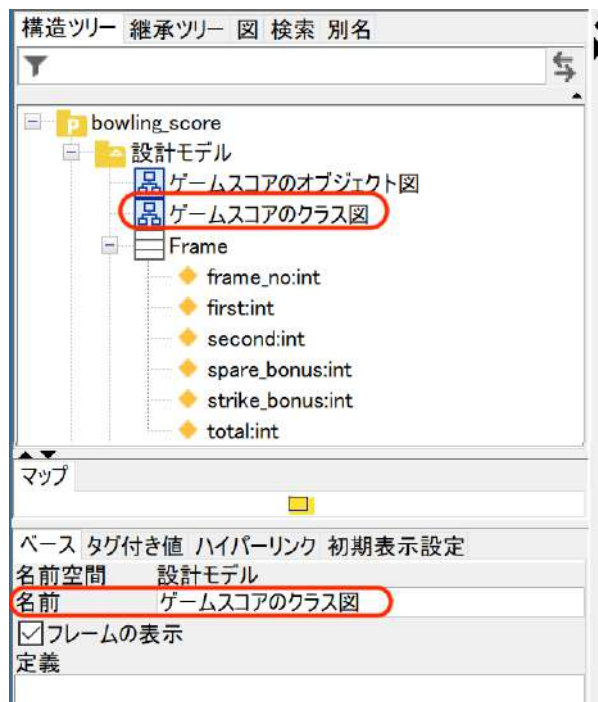


図 2.30 追加した図を「ゲームスコアのクラス図」とした

クラス図にクラスを追加する

追加したクラス図に、ゲームスコアクラスを追加します。

- ・ 構造ツリーから、オブジェクト図を作成したとき登録した「ScoreSheet」クラスをさがし、選

択する。

- ・ 選択したクラスをドラッグ&ドロップして、クラス図に追加する（図 2.31）。
- ・ 構造ツリーから、「Game」クラスを選択する。
- ・ 選択したクラスをドラッグ&ドロップして、クラス図に追加する（図 2.31）。

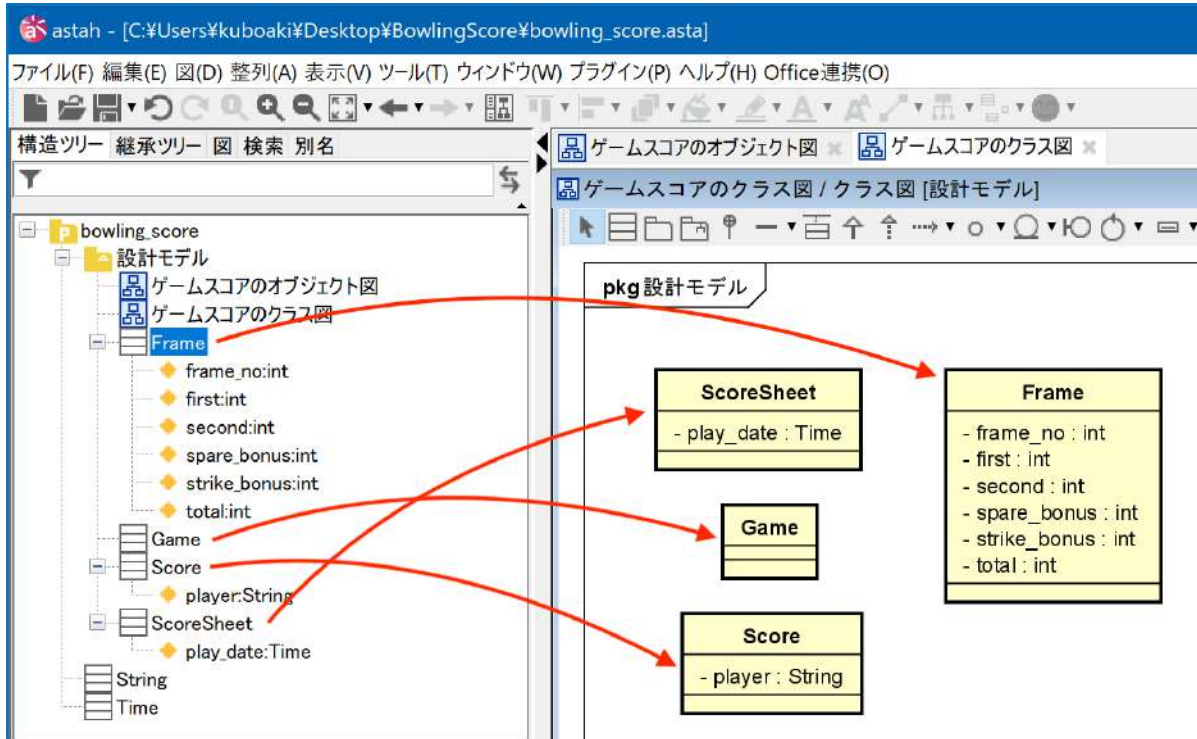


図 2.31 既存のクラスを構造ツリーからクラス図に追加する

2.3.2 クラス間の関連を追加する（1）

作成したオブジェクト図（図 2.27）に記載されているリンクを参照して、リンクでつながっているオブジェクトが属するクラスの間に関連を引きます。

スコアシートとゲームの間に関連を追加する

まず、「ScoreSheet」クラスから「Game」クラスへ関連を引きます。

- ・ パレットから矢印付きの関連を選択する。
- ・ 「ScoreSheet」クラスから「Game」クラスへ向かって関連を引く（図 2.32）。

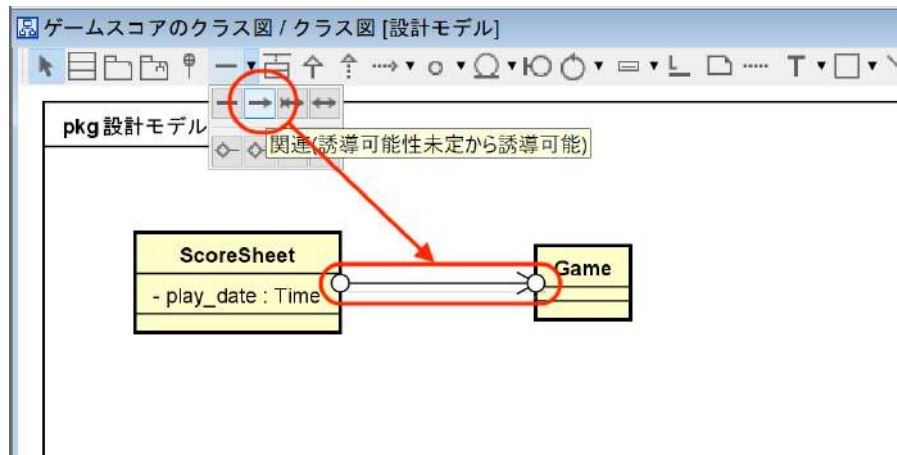


図 2.32 「ScoreSheet」クラスから「Game」クラスへ関連を引いた

スコアシートからみたゲームの多重度を設定する

スコアシートには1ゲーム以上の複数のゲームを記録できます。クラス間の関連において、このことを表には多重度を使います。

- ・ 図 2.32 で引いた関連を選択した状態で、プロパティからターゲットが「Game」の関連端のタブを開く。
- ・ 「多重度」を「1..*」に設定する（図 2.33）。



図 2.33 「ScoreSheet」からみた「Game」の多重度を「1..*」に設定した

スコアシートからみたゲームの関連端名を設定する

スコアシートがゲームを参照するときに使う名前を決めるために、関連端名を設定します。

- ・ 図 2.32 で引いた関連を選択した状態で、プロパティからターゲットが「Game」の関連端のタブを開く。
- ・ ゲームを複数回記録できることを反映して、「名前」を「games」に設定する（図 2.34）。

ベース	ステレオタイプ	制約	関連端 A
関連端 A 制約	関連端 B	関連端 B 制約	タグ付き値
ターゲット	Game		
タイプ修飾子			
名前	games		
誘導可能	navigable		
集約	none		
初期値			
可視性	private		
Static	false		
Leaf	false		
多重度	1..*		
派生	false		
定義			

図 2.34 「ScoreSheet」 からみた 「Game」 の関連端名を 「games」 に設定した

スコアシートとゲームの集約関係を設定する

スコアシートには、複数のゲームを追加できます。このことをクラス図に反映する方法がほしいですね。

まず、スコアシートが複数のゲームをとりまとめていること表すのに、「スコアシートには複数のゲームを集約する」役割があるというように捉えてみましょう。このような役割を表すために、クラスの関連端には「集約」を表すための記法が用意されています。

さらに、同じスコアシートで追加でゲームをやりたいときは、新しいゲームを始めた時点で新しいゲームの記録を始めます。つまり、スコアシートが作成される時点と、あるゲームが開始される時点が同時とは限らないわけです。このように、集約する側（ここではスコアシート）のインスタンスと、集約される側（ここではゲーム）のインスタンスが作成される時点が同じではないことを「スコアシートとゲームはライフサイクルが異なる」といいます。

一方で、集約する側とされる側が同時に作成され、同時に破棄されるなら、「スコアシートとゲームのライフサイクルは同じ」と考えます。

これらの違いを表せるように、集約の記法には、ライフサイクルが異なる場合の「aggregate」、ライフサイクルが同じ場合の「composit」を表すオプションが用意されています。

NOTE

コンポジション (Composition) を「合成集約」と呼ぶこともあります。

ここでは、スコアシートが複数のゲームをとりまとめていて、スコアシートとゲームのライフサイクルが異なることを反映して、スコアシート側の関連端に「aggregate」オプションを指定した集約を設定しましょう。

関連端に「aggregate」オプションを設定する

- ・ 図 2.32 で引いた関連を選択した状態で、プロパティからターゲットが「ScoreSheet」の関連端のタブを開く（描画時の順序によって該当する関連端がAの場合とBの場合がある）。
- ・ 「集約」のプルダウンメニューから「aggregate」を選択する（図 2.35）。



図 2.35 「ScoreSheet」が「Game」を集約していることを示した

- ・「ScoreSheet」から「Game」への関連が引けた（図 2.36）。

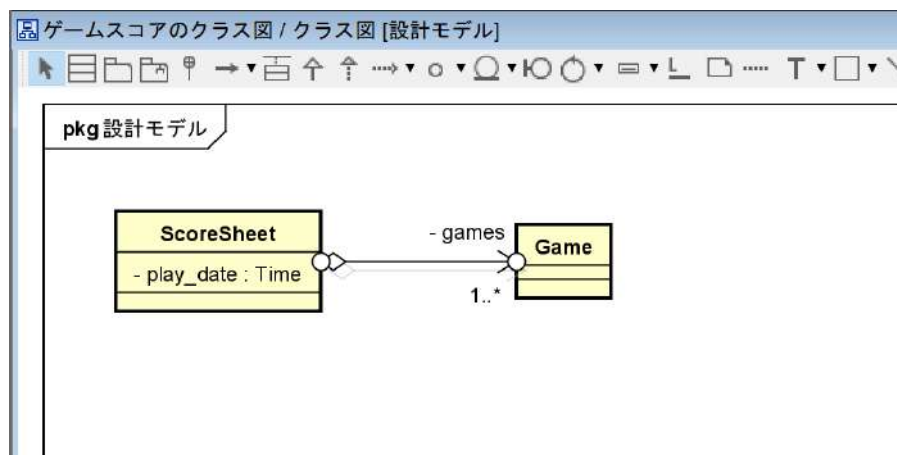


図 2.36 「ScoreSheet」から「Game」への関連が引けた

2.3.3 クラス間の関連を追加する（2）

ゲームとスコア、スコアとフレームの間にも関連を引いて、関連端も設定してみましょう。

ゲームとスコアの間に関連を追加する

こんどは、「Game」クラスから「Score」クラスへ向かって関連を引きましょう。1組のゲームには、複数プレイヤーのスコアが記録できます。つまり、ここにも、スコアシートとゲームの間と同じような関連が引けそうですね。

「Game」クラスから「Score」クラスへ関連を追加する

- ・「Score」クラスと「Game」クラスの間に関連を引く。
- ・「Score」クラス側の関連の多重度を「1..*」に設定する。
- ・「Score」クラス側の関連端には関連端名として「scores」を設定する（複数のスコアが記録できることを反映した）。
- ・「Game」クラス側の「集約」のプルダウンメニューから「aggregate」を選択する（図 2.37）。

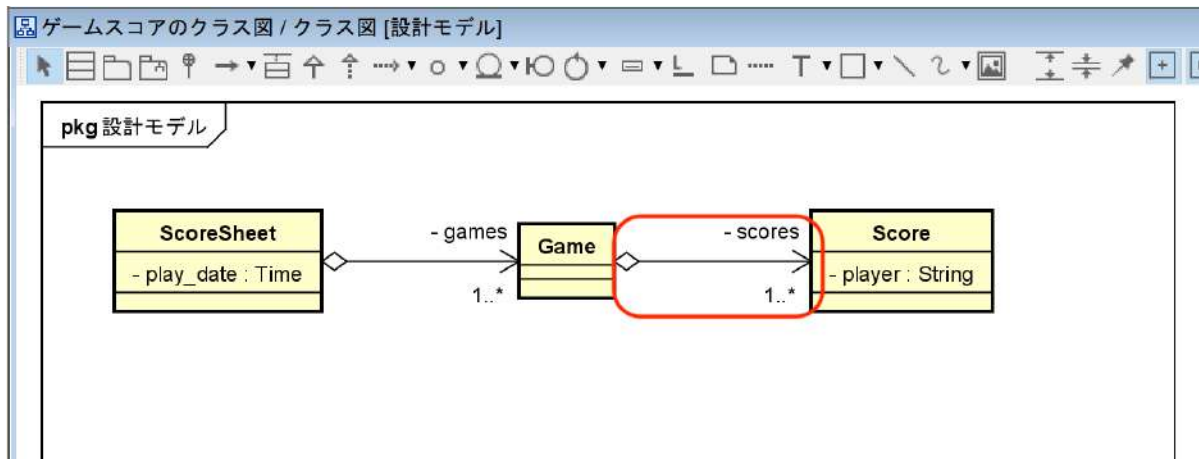


図 2.37 ゲームとスコアの間に関連を引いた

スコアとフレームの間に関連を追加する

最後に、「Score」クラスから「Frame」クラスへ向かって関連を引きましょう。フレームのオブジェクトをいつ用意するのかには、いくつか選択肢があります。たとえば、ゲームを用意したときに全フレーム分用意すると決めることもできるでしょう。あるいは、ゲームの進行に合わせて、その都度フレームを追加すると決めることもできるでしょう。いくつかの選択肢があるとき、それらからいずれを選択するのかを決定するのは、多くの場合設計時の決定事項です（このことを、設計事項、設計項目、設計事由などと呼ぶ人たちもいます）。

このチュートリアルでは、スコアのオブジェクトを用意したときは、常に10フレーム分のフレームのオブジェクトも一緒に用意することとします。この考えを選択したのは、表示や出力の際に、未投球のフレームについても、それらのフレームのオブジェクトを参照すれば、未投球の処理で特別な扱いが減られると考えたからです。この場合、フレームとスコアは同時に作成される一ライフサイクルが同じなので、集約の設定もコンポジションにしておきます。

- ・「Game」クラスと「Frame」クラスの間に関連を引く。
- ・「Frame」クラス側の関連端の多重度を「10」に設定する。
- ・「Frame」クラス側の関連端には関連端名として「frames」を設定する（複数のフレームが記録できることを意識した名前にした）。
 - 集約のプルダウンメニューの中から「composite」を選択する（図 2.38）。
 - 「Score」クラス側の関連端が、黒塗りのダイヤモンド（コンポジションのシンボル）に変わる。

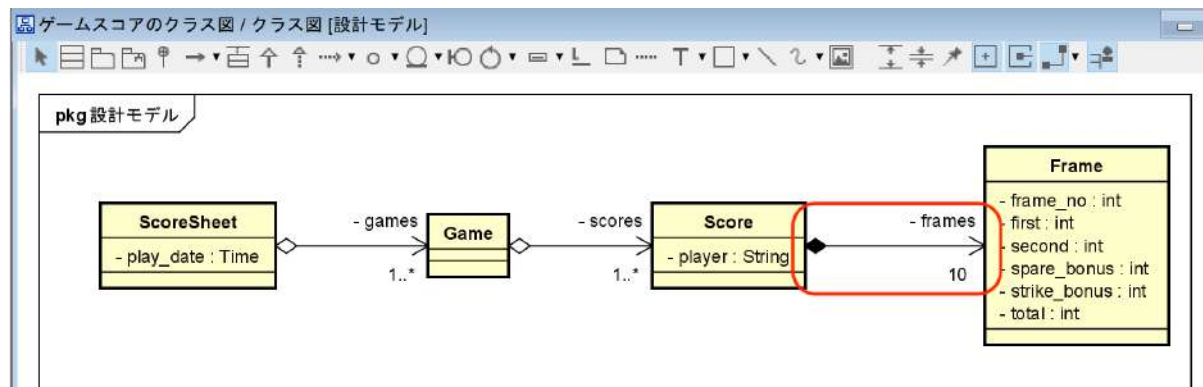


図 2.38 スコアとフレームの間に関連を引いた

これで、いったん、ボウリングのゲームスコアを表した構造のモデルができました。

2.4 まとめ

ボウリングのゲームスコアを記録するスコアシートの構造を検討しました。

2.4.1 構造のモデルを作成した手順

構造のモデルに登場する構成要素や要素間の関連を見つけるために、次のような手順を使いました。

スコアシートの構造のモデルを作成した手順

1. スコアシートを観察して、どのような要素で構成されているか洗い出した。
2. スコアシートの実例をそのまま使ってオブジェクト図で表した。
3. オブジェクト図の要素や要素間のつながりを観察して、クラス図を作成した。

まず、オブジェクト図を作りました。オブジェクトを洗い出し、リンクを使ってオブジェクト同士のつながりを表しました。そして、オブジェクト図を参照しながら、クラス図を作成しました。クラス間の関連の関連端名や多重度を検討する際は、オブジェクト図におけるリンクの数などを判断材料にしました。

2.4.2 この段階の構造のモデルはまだ未完成

ここで作成したクラス図には、検討の余地が残っています。たとえば、いずれのクラスにも操作が定義できていません。これは、プレーするのに従ってスコアをつけるときの動作（振る舞い）を検討していないからです。また、動作を検討する中で、属性や関連についても追加や修正が必要になる場合があります。

そこで、続いて振る舞いのモデルを検討します。振る舞いのモデルの作成が進むにつれて、構造のモデルも追加・修正されることになるでしょう。

第3章 モデルとコードの対応づけ

プログラムがどのように動作するのか検討するときには、振る舞いのモデルを使います。振る舞いのモデルを作成する前に、構造のモデルと振る舞いのモデルがどのようなコードとして実現されるのかについて、このチュートリアルの開発で用いる方式を決めておきましょう。このことを「モデルとコードの対応づけ」または「実装方式の設計」と呼ぶことにします。

TIP

このような対応づけを「モデル変換ルール」と呼ぶこともあります。また、開発プロセスのなかでこの対応づけを検討する工程を「方式設計」と呼ぶ人もいます。

【この章の役割についてもう少し】

この章で検討したいことは、ボウリングのスコアに関してどのようなモデルを作るのか、という話題から少し離れています。モデルをどのように実装するのかということは、モデルの表し方とコードの表し方の間の関係を定めるものであって、ボウリングのスコアをどのようなモデルとして表すかということとは独立している話題だからです。

もし、この章の役割がピンとこないとか、実装についての議論にはあまり関心がないのであれば、最初はこの章を読み飛ばしてみてもかまいません。後の章へ進めば、実装する人がモデルを使う際には、モデルとコードを対応づける必要があり、事前に実装方式を決めておかなければ、実装者が実装方法について考える（実装時に「どのように作るのか」を設計する）必要があることに気づくでしょう。そのときは、この章に戻って、どうして実装方式の設計が必要なのか考えてもらえたらと思います。

3.1 なぜ対応づけの検討が必要なのか

ソフトウェア開発でよく目にする事態として、設計と実装が乖離してしまい、頼れるのは動いているコードだけというものがあります。このとき、多く人が「設計書がないから」「設計書が更新できていないから」と説明します。このことについて、もう少し詳しく考えてみましょう。

ソフトウェアの開発に使う設計書には次のような2つの役割があります。

設計書の役割

- ・「何を作るか」について、構造と振る舞いをもって示す。
- ・「どのようにして作るか」について、設計書と実装の対応づけをもって示す。

「何を作るか」の設計書は、どんな機能を提供し、それがどのような動作・振る舞いとなっているかを示す設計書です。例えば、「機能的な要求の発見・獲得→要求の分析→外部設計→内部設計」という開発プロセスがあるとき、「要求分析のユースケース図」、「外部設計のクラス図・シーケンス図」、「内部設計のクラス図・ステートマシン図」などを作成します。これらが「何を作るか」を示すための設計書です。

では、「どのようにして作るか」の設計書とはどんなものなのでしょうか。これは、例えば、設計書にクラスがあったらC言語の構造体やRubyのクラスに対応づけるとか、状態遷移は2次元配列を使っ

て遷移表として実装しようとか…そういった、設計書に記載した要素に対して、実装するときにコードのどの部分にどのように反映するかを示した設計書です。

先に挙げた「何を作るか」の設計書には、「どのような実装方法で実装するのか」は書いてありません。しかし、「どのようにして作るか」の設計書がないと、その側面は実装者がコードを作成しながら考えることになります。そうすると、「どのようにして作るか」は、実装者ごとに考えることになり、また、同じ実装社でも実装する毎に異なってきます。しかも、その考えはコードに埋め込まれ、明文化されません。設計書として明文化されていないならば、実装に不具合があって設計書に戻ったとしても、検討すべき情報がそこにはないということになります。かくして、設計と実装の乖離ができていくというわけです。

i NOTE

「モデル駆動開発（MDD）」という開発手法に、「モデル・タイプマッピング」と呼ばれる、2つのモデルの間の対応づけについての考え方があります。モデルとコードの対応づけは、タイプマッピングの一種とみなすこともできます。

3.2 対応づけ検討用モデルの作成

このチュートリアルでは、実装にC言語を使うことにしました。そのために、対応づけの検討においても、設計モデルとC言語プログラムの対応づけを考えてみることにします。

i NOTE

同じモデルからRubyで実装するチュートリアルもあります。

モデルを使ってソフトウェアを開発しよう

https://github.com/ChangeVision/tut_uml_modeling_bs/

3.2.1 対応づけ検討用プロジェクトを用意する

「モデルとコードの対応づけ」の検討用に astah* で別のプロジェクトを作成しましょう。

対応づけ検討用プロジェクトを作成する

- astah* で新規プロジェクトを作成する。
- 作成したプロジェクトをいったん保存する。
- モデルファイルの名前は「stm_sample.asta」とする。

プロジェクトの設定

実装にC言語を使うので、プロジェクトで使う言語情報を「cxx」に変更しておきます。

- 構造ツリーで、プロジェクト名をクリックして選択する。
- プロパティから「プロジェクトの設定」タブを開く。
- プログラミング言語のチェックボックスから「cxx」をチェックする。

- ・ 確認ダイアログ「プロジェクトに言語情報を適用しますか？」で「はい」をクリックする。
- ・ 「Java」「C#」など他の言語の項目のチェックは外しておく。



図 3.1 プロジェクトの設定で言語情報を設定する

検討用プロジェクトにクラス図を追加する

プロジェクトが保存できたら、プロジェクトにクラス図を追加しましょう。

- ・ 構造ツリーで、プロジェクト名をクリックしてポップアップメニューを開く。
- ・ 「図の追加>クラス図」でクラス図を追加する。
- ・ プロパティから、クラス図の名前を「クラス図」に変更する。

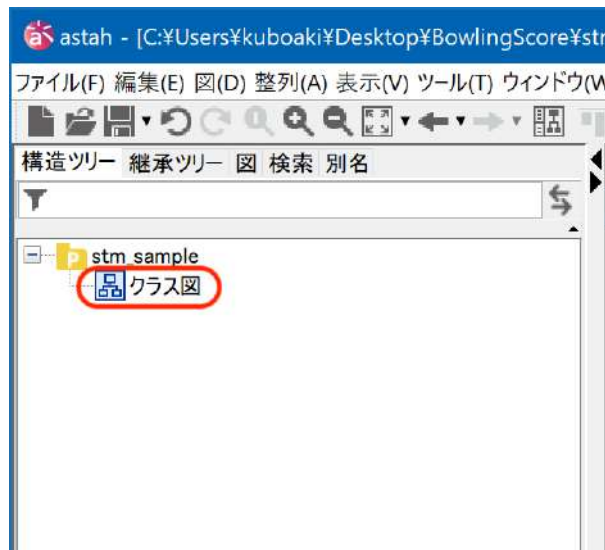


図 3.2 対応づけの説明に使うプロジェクトにクラス図を追加した

3.3 構造のモデルとコードを対応づける (1)

手始めに、UMLのクラスとC言語の構造体を対応づけてみましょう。いきなり細かいことまですべて取り扱うのは大変なので、基本的な構成要素だけ対応づけてみます。

3.3.1 クラス図に「Sample」クラスを追加する

対応づけの説明用に、簡単なクラス「Sample」の定義から始めましょう。このクラスは、操作「Play」と属性「attr_a」「attr_b」を持つクラスとします。これをUMLのクラスとして表してみましょう。

- パレットからクラスを選択し、クラス図に追加する。
- クラス名を「Sample」にする（図 3.3）。
- クラスに操作「play」を追加する。
- クラスに属性「attr_a」「attr_b」を追加する。

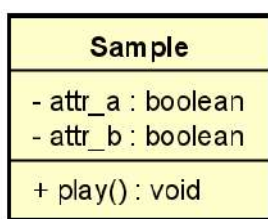


図 3.3 クラス図に「Sample」クラスを追加した

3.3.2 Sampleクラスの最初の対応づけ

対応づけルールの1つ目として、次のように書きたいと決めることにします。操作や属性は公開範囲（スコープ）を設定できますが、ここでは演習の簡単のために、属性はプライベート、操作はパブリックということにします。

UMLのクラスとC言語のクラスの対応づけ (1)

- UMLのクラス名を、C言語の `typedef` で構造体型として定義する。
- UMLのクラスの属性を、クラスを割り当てた構造体の中のメンバ変数とする。
- UMLのクラスの操作を、最初の引数がクラスを割り当てた構造体型へのポインタになっている関数とする。
- クラスのインスタンスを作成するための初期化関数（コンストラクタ）として `init` の付く関数を用意する。

【コラム2】C言語でのクラスの表現

C言語はオブジェクト指向言語ではありませんが、構造体(struct)を使ってクラス概念を表現できます。

このチュートリアルが採用した実装方式では、次のような対応づけになっています。

- **クラス** → 構造体(struct)
- **属性** → 構造体のメンバー変数
- **メソッド** → 構造体へのポインタを第1引数に取る関数
- **コンストラクタ** → `xxx_init()` 関数
- **カプセル化** → ヘッダーファイルとソースファイルの分離、static関数

たとえば、シングルインスタンス・モジュール（[コラム 3](#)で紹介）では、次のような対応づけを使うこともできます。

- **クラス** → ファイルセット（クラス名のわかる `.c` と `.h`）
- **属性** → そのファイルの static なグローバル変数
- **メソッド** → そのファイルで定義している関数
 - クラス名のわかるプレフィックスをつける
 - パブリックなメソッドは、ヘッダーファイルに関数のプロトタイプ宣言を追加
- **コンストラクタ** → `クラス名_init()` 関数を定義
- **カプセル化** → staticなグローバル変数、パブリックなプロトタイプ宣言の活用

3.3.3 Sampleクラスの最初のサンプルコード

対応づけルールの1つ目として、[図 3.3](#)を表したCのヘッダーファイル（`.h` ファイル）と実装ファイル（`.c` ファイル）のセットで書きたいと決めることにします。

まず、ヘッダーファイル `sample.h` については、[リスト 3.1](#) のように対応づけます。

リスト 3.1 `sample.h`: 「Sample」クラスを表すC言語のヘッダーファイル

[C]

```
#ifndef SAMPLE_H
#define SAMPLE_H

#include <stdio.h>
#include <stdbool.h>

typedef struct { ①
    bool attr_a; ②
    bool attr_b; ②
    /* 状態変数は後で追加 */
} Sample;

/* メソッドにあたる関数のプロトタイプ宣言 */
void sample_init(Sample *sample); ③
void sample_play(Sample *sample); ④

#endif /* SAMPLE_H */
```

- ① 構造体「Sample」の定義（UMLのクラスに対応）。
- ② このクラスのメンバー変数（UMLの属性に対応）。

- ③ 初期化関数（コンストラクタに相当する関数）。
- ④ 操作「play」に対応する関数。

そして、実装ファイル `sample.c` は リスト 3.2 のようになります。

リスト 3.2 `sample.c`: 「Sample」クラスの実装（.cファイル）

[C]

```
#include "sample.h" ①

/* Sample構造体の初期化（インスタンスの作成） */
void sample_init(Sample *sample) { ②
    sample->attr_a = true; ③
    sample->attr_b = true; ③
    /* 状態の初期化は後で追加 */
}

/* playメソッドの実装 */
void sample_play(Sample *sample) { ④
    /* 振る舞いは後で実装 */
    printf("play called attr_a:%d\n", sample->attr_a);
}
```

- ① 実装するクラスのヘッダーファイルのインクルード。
- ② 初期化関数の実装。
- ③ メンバー変数の初期化。
- ④ playメソッドの実装。

Sampleクラスを利用する場合には、Sampleクラスのインスタンスを作り、その操作（メソッド）を呼び出します（リスト 3.3）。

リスト 3.3 Sampleクラスを使うコード例

[C]

```
#include "sample.h" ①

int main(void) {
    Sample smp101; ②
    sample_init(&smp101); ③
    sample_play(&smp101); ④
}
```

- ① 使用するクラスのヘッダーファイルのインクルード。
- ② Sampleクラスのインスタンスに使う変数 `smp101` を定義。
- ③ `smp101` の初期化。
- ④ `smp101` のメソッド `play` の呼び出し。

 TIP

インスタンスに使う変数は、より一般的には `malloc` など動的に確保する方法が使われます。この場合、メソッドとなる関数には確保した領域のポインタを渡すことや、使用後に `free` を使って領域を開放することに注意します。

 NOTE

この段階のサンプルコードは、`codes/sample01` ディレクトリに用意してあります。

【コラム3】実装方式の策定も設計の一部

C言語に限ったことではありませんが、モデルとコードの対応づけ（実装方式）は、決まったものがあるわけではありません。重要なのは、自分たちがチームで決まった方式を決めておき、モデルの作成（設計時）にもその実装方式を前提に設計し、実装時にもその実装方式に従って実装することです。

たとえば、「テスト駆動開発による組み込みプログラミング」[embtdd]では、「SOLIDの原則をCモジュールに提要した設計モデル」として次のようなモデルを紹介しています。

シングルスインスタンスモジュール（Single-instance module）

モジュールの内部構造をカプセル化するが、モジュールのインスタンスがひとつであることを前提とする。

マルチインスタンスモジュール（Multi-instance module）

モジュールの内部構造をカプセル化して、モジュールのインスタンスを複数作れるようにする。

動的インターフェイス（Dynamic interface）

モジュールのインターフェイス関数を実行時に割り当てられるようにする。

型による動的インターフェイス（Pre-type dynamic module）

同じインターフェイスを持つモジュールの型が複数あり、それぞれが独自のインターフェイス関数を持てるようにする。

この分類からみると、このチュートリアルが採用した実装方式は、「マルチインスタンスモジュール」に相当することがわかるでしょう。

さらに、クラス、シーケンス、状態遷移といった設計上の概念を、実装時にどのように実現するのかを決めておくこともまた重要な設計上の決定になります。設計の方針ともなるので、「実装方式の決定はアーキテクチャ設計である」と捉える場合もあります。

3.4 振る舞いのモデルを作成する

このサンプルにおいては、いま追加した操作「play」が、「Sample」クラスの振る舞いを提供しているとします。しかし、クラス図に描いた「Sample」クラスに操作「play」を追加しただけでは、その処理内容はわかりません。どこかに「play」の処理内容を表した図が必要です。

そこで、「Sample」クラスに対して、操作「play」の振る舞いを表すステートマシン図を追加します。ここで、図を追加する対象を「Sample」クラスにしているのは、このステートマシン図が「Sample」クラスの振る舞いのモデル図であるとわかるようにするためです。

3.4.1 クラスにステートマシン図を追加する

「Sample」クラスにステートマシン図を追加します。

- ・ 構造ツリーから「Sample」クラスを選択する。
- ・ 右クリックしてポップアップメニューを表示する（図 3.4）。
- ・ 「図の追加＞ステートマシン図」を選択すると、ステートマシン図が追加される。

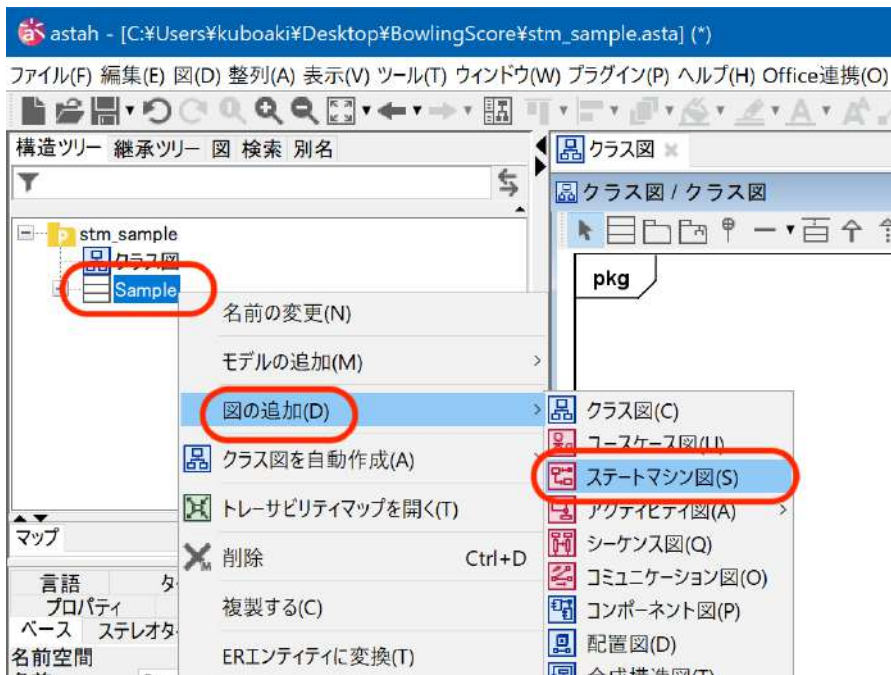


図 3.4 「Sample」クラスにステートマシン図を追加する

- ・ プロパティから図の名前を編集し、「Sampleのplayのステートマシン図」とする（図 3.5）。
- ・ ダイアグラムエディタのタイトルやタブにも図の名前が反映される。

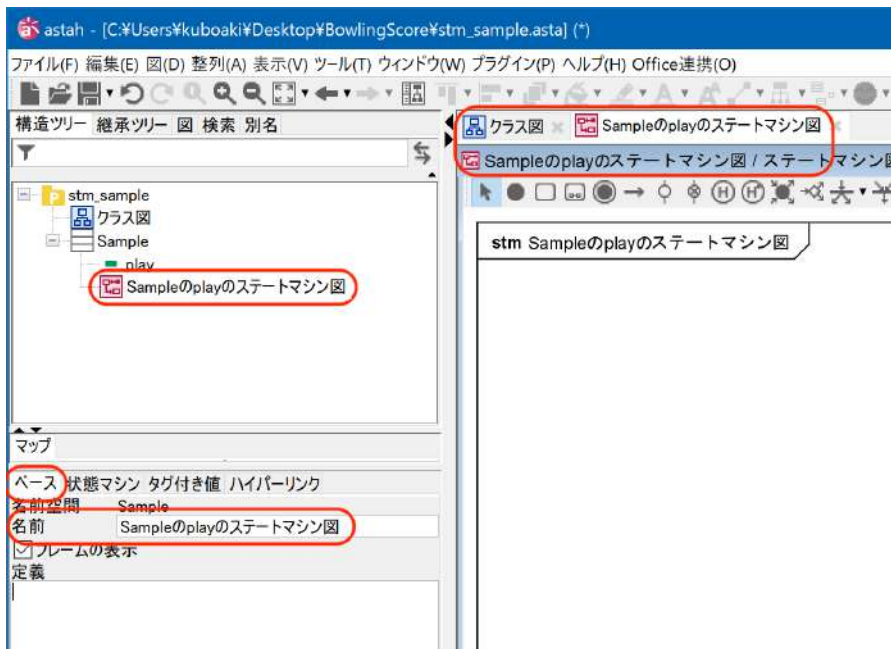


図 3.5 追加したステートマシン図に名前をつける

TIP

図 3.4 では、Sampleクラスの振る舞いを表すステートマシン図とわかるようにと考えて、クラ

スに対してステートマシン図を割り当てました。この他、astah*では、メソッドに対してステートマシン図を割り当てることもできます。任意のメソッドの処理を示すステートマシン図の割り当てとしては、後者のほうが好ましいでしょう。ここでは、クラスの振る舞いということを意識して、クラス図に割り当ててみてください。どちらの方法を使うにしても、クラスの振る舞いを担当するメソッドを対応づけすることは必要になるでしょう。

【コラム 4】ステートマシン図を学ぶチュートリアル【参考】

ここでは、モデルとコードの対応づけを説明するために、振る舞いのモデルの一種であるステートマシン図を作成しています。ステートマシン図を使って振る舞いを設計する方法については、次のチュートリアルも参考にしてみてください。

ステートマシン図 & 状態遷移表チュートリアル

<https://www.changevision.co/tutorial-statemachine-japanese.html>

3.4.2 ステートマシン図に状態を追加する

次に、追加したステートマシン図に、状態を追加します。

ステートマシン図に状態を追加する

1. パレットから「状態」を選択し、ステートマシン図に状態を追加する。
 - 。追加した状態の状態名は「状態0」（数字は追加の都度変わる）となっている（図 3.6）。

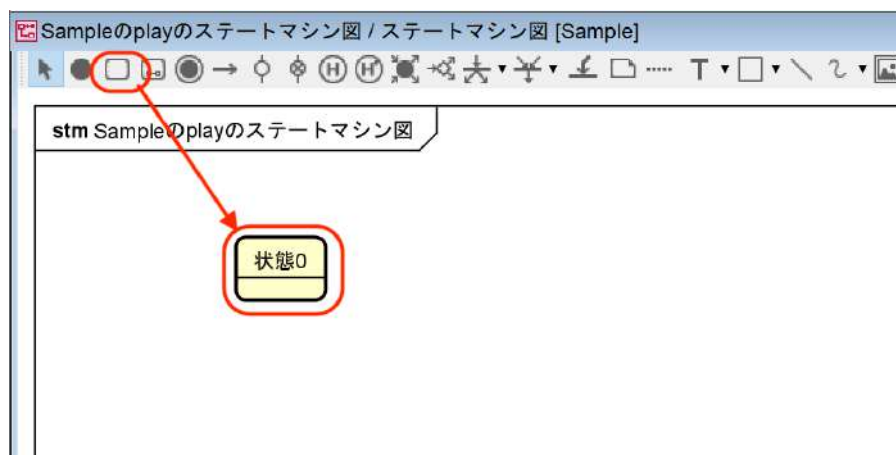


図 3.6 新しい状態を追加した

2. 「状態0」を選択し、プロパティの「ベース」タブの「名前」を編集して、状態の名前を「ST0」にする（図 3.7）。
 - 。状態「ST0」は、このサンプルの例示用に使う状態名の1つ。

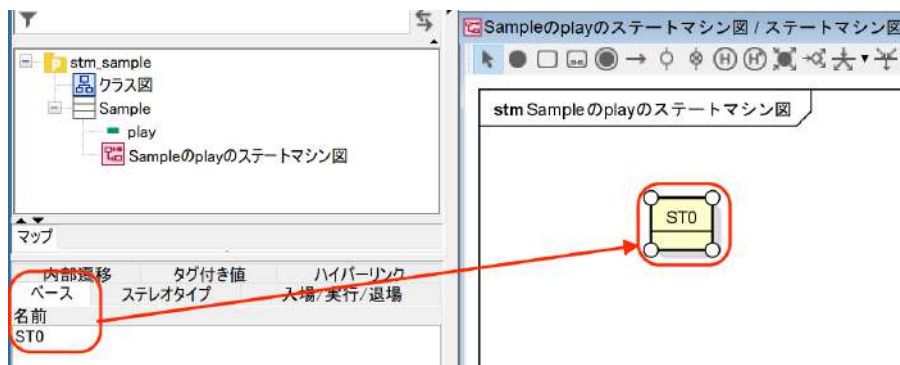


図 3.7 新しい状態に状態名をつけた

3. このサンプルでは3つの状態を使いたいので、状態をもう2つ追加する（図 3.8）。

- 。「ST1」、「ST2」を追加した。

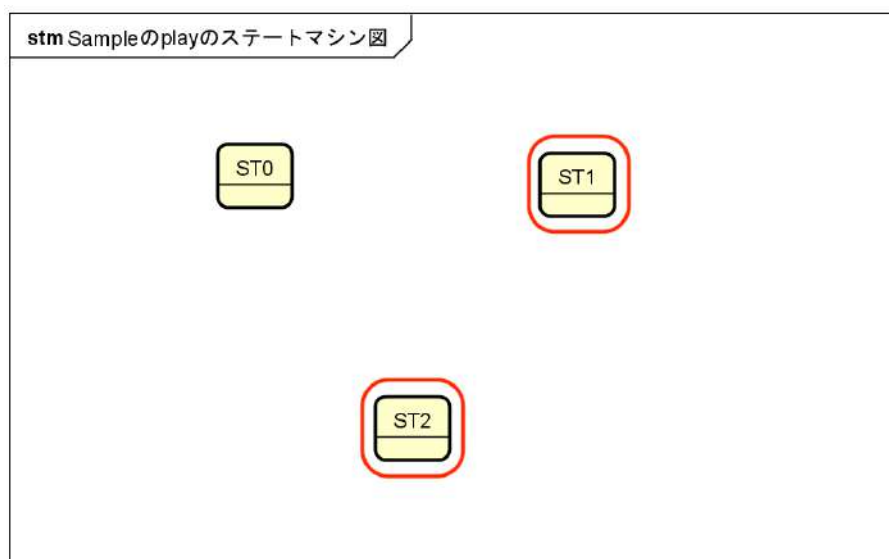


図 3.8 新しい状態に状態名をつけた

3. 開始疑似状態、終了疑似状態を追加した（図 3.9）。

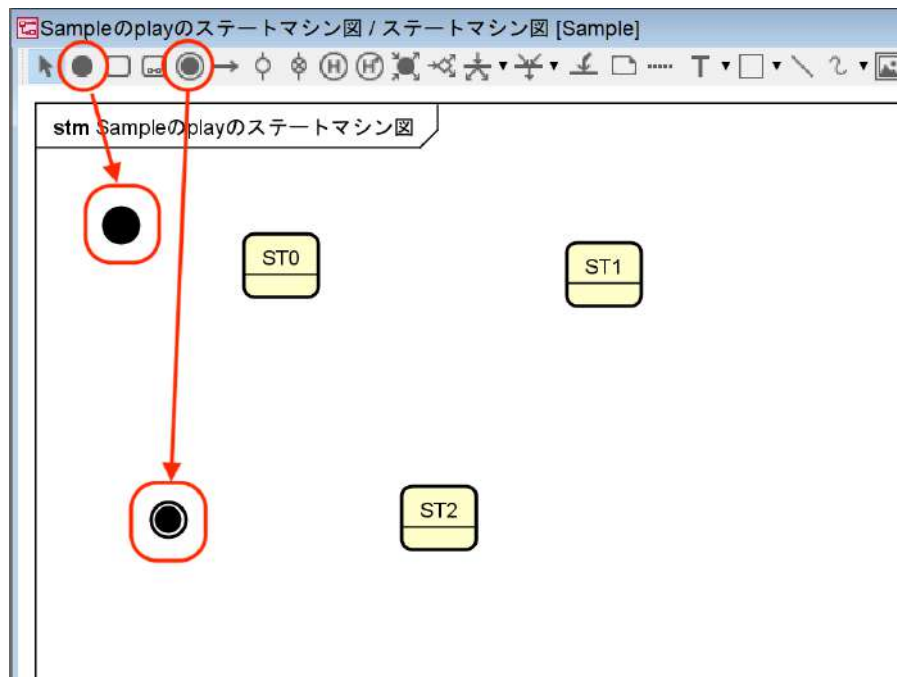


図 3.9 開始疑似状態、終了疑似状態を追加した

NOTE

「開始疑似状態」は、それ自体は状態ではありません。開始疑似状態から遷移する先の状態がその図の中の最初の状態ということを示すラベルと考えるとよいでしょう。「終了疑似状態」も、終了疑似状態へ遷移する状態が最後の状態（のひとつ）ということを示すラベルと考えるとよいでしょう。

3.4.3 ステートマシン図に状態遷移を追加する

状態が追加できたので、こんどは状態遷移を追加しましょう。ステートマシン図には、通常の状態のほかにいくつかの「疑似状態」が登場します。たとえば、「開始疑似状態」は、この図における最初の状態を示すのに使います。「終了疑似状態」は、その図における最後の状態（1つとは限りません）を示すのに使います。

状態遷移を追加する

1. パレットから「遷移」を選択して、「開始疑似状態」の内部へマウスカーソルを移動し、青枠が表示されるのを待つ（図 3.10）。

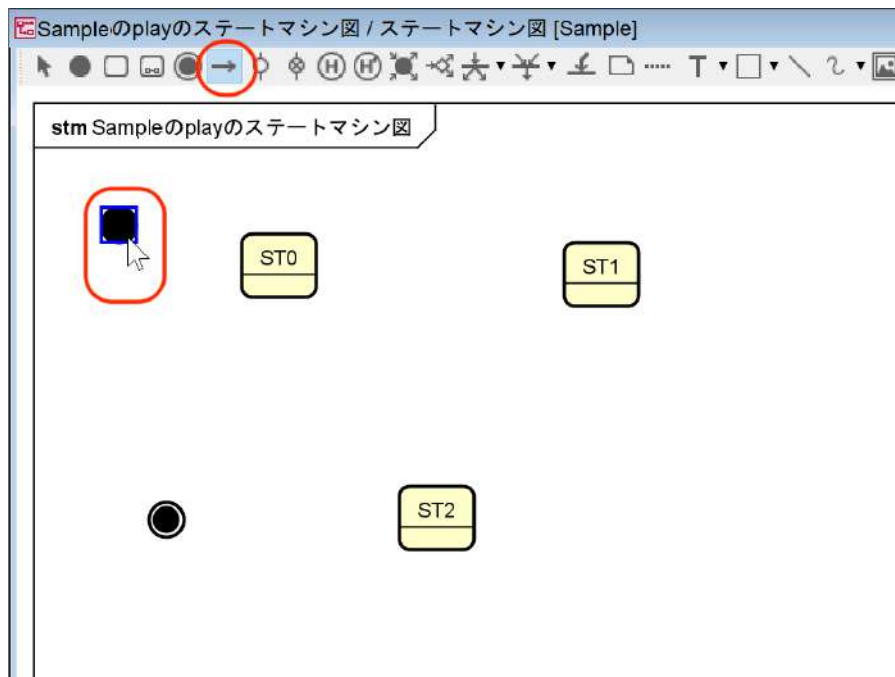


図 3.10 開始疑似状態の内側で青枠を表示させる

2. 青枠が表示されたらマウスカーソルをドラッグする（マウスのボタンを押したままマウスカーソルを移動する）。
3. マウスカーソルを「ST0」ヘドラッグして青枠が表示されるのを待つ（図 3.11）。

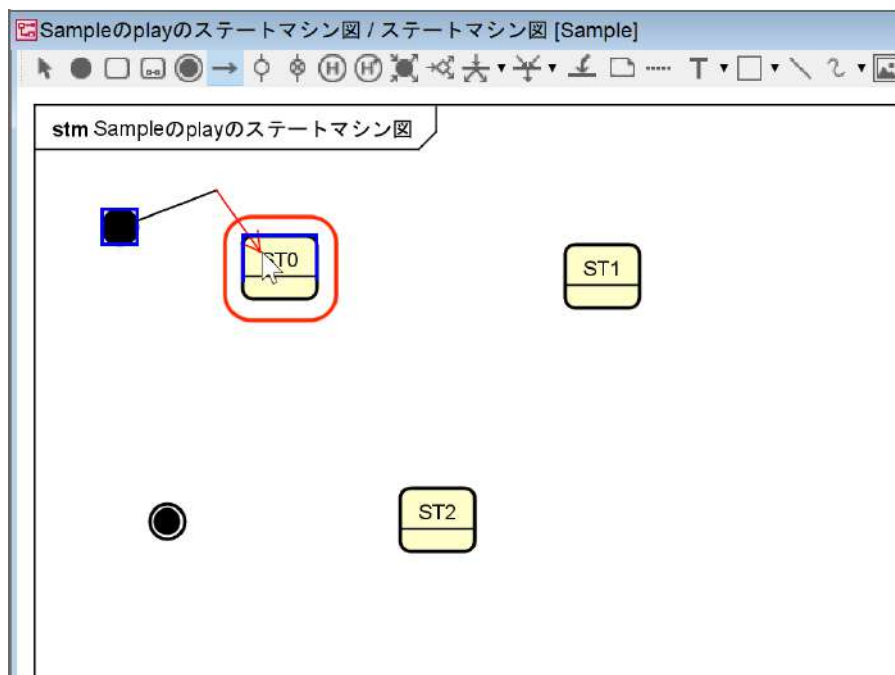


図 3.11 「ST0」ヘマウスカーソルをドラッグして青枠が表示されるのを待つ

4. 「ST0」でも青枠が表示されたらマウスのボタンを離すと状態遷移が引かれる（図 3.12）。

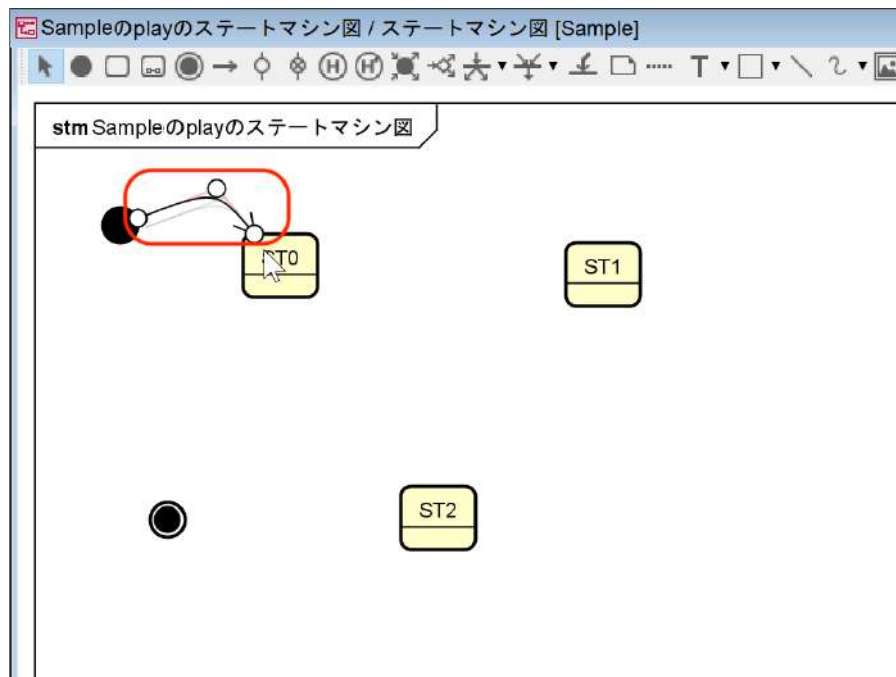


図 3.12 「ST0」でも青枠が表示されたらマウスのボタンを離すと状態遷移が引かれる

4. ほかの状態についても 図 3.13 と同じように状態遷移を追加する。

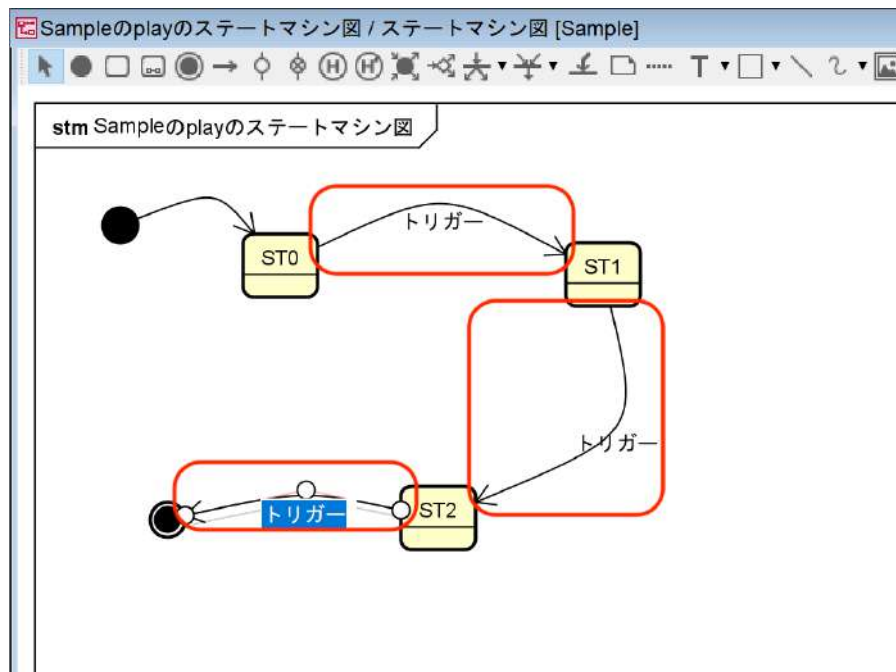


図 3.13 ほかの状態遷移も追加する

3.4.4 状態・イベント・状態遷移・アクション

ステートマシン図で処理を表す時に登場する要素について、簡単に整理しておきましょう。

状態とイベント

ステートマシン図において、状態は「イベント」の発生を待っているところです。多くの場合、イベントは、そのクラス自身の操作では処理を先に進められなくなるようなできごとです。たとえ

ば、外部からの入力（操作待ちや受信待ちなど）や、一定の時間経過などがイベントの候補になります。

状態遷移

状態遷移は、ある状態において待っていたイベントが発生して、別の状態へ移ることを表します。遷移先が2つ以上ある状態では、各々の状態遷移で待っているイベントは異なっている必要があります。それは、同じイベントで別の状態へ遷移する状態遷移が複数あると、どちらへ遷移すべきかをイベントの違いだけでは判定できなくなるからです。そのような状態遷移を「あいまいな状態遷移」と呼びます。「選択疑似状態」を使うと、イベントの発生時に状況によって遷移先を振り分けることもできます。

状態遷移には、イベントが発生したとき、実際に状態を遷移するか判定する条件を追加できます。この条件のことを「ガード条件」と呼びます。ガード条件がある状態遷移では、イベントが発生したときにガード条件を評価します。ガード条件の評価が真のときは、状態が遷移します。ガード条件の評価が偽のときは、状態は遷移しませんが、その場合もイベントは起きたとことになります（「イベントが消費された」といいます）。

アクション

状態や状態遷移には、イベントが発生したときに実行したい処理を割り当てられます。この処理のことを「アクション（またはエフェクト）」と呼びます。

状態遷移に割り当てられたアクションの場合、イベントが発生すると状態遷移に伴うアクションが実行されます。状態が遷移するのは、アクションの実行が完了してからです。ガード条件付きの状態遷移の場合は、イベントの発生後さらにガード条件も真なら、アクションが実行され、次の状態へ遷移します。

状態にアクションを割り当てる場合、次の3つの内部処理を記載できます。

状態内のアクション（内部動作）の種類

entry / 入場時アクション

その状態に入った直後に一度だけ実行される処理。

do / Doアクティビティ

状態に滞在している間、継続的に（または繰り返し）実行される処理。

exit / 退場時アクション

イベントが発生してその状態から遷移して抜けるとき、その直前に実行される処理。

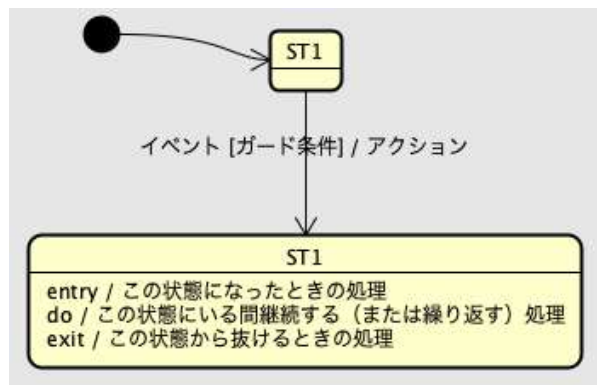


図 3.14 アクションはイベントや状態に割り当てられる

3.4.5 ステートマシン図にイベントやアクションを追加する

ここでは、イベントにアクションを割り当ててみましょう。

まず、「ST0」から「ST1」への状態遷移を、イベントが「ev1」でガード条件は「gd（が真）」のとき、アクション「act1」を実行するよう編集してみましょう。

状態遷移にイベントやアクションを追加する（1）

1. 「ST0」から「ST1」への状態遷移を選択し、この遷移のプロパティを表示し、ベースタブを開く。
2. プロパティを編集する（図 3.15）。
 - 「トリガー」にイベント名を設定する。ここでは「ev1」というイベントを設定する。
 - 「ガード」にガード条件を設定する。ここでは「gd1（が真）」を設定しする。
 - 「アクション」にアクションを設定する。ここでは「act1」という処理があるとして、これを設定する。
 - act1は、2つの引数（イベントとパラメーター）を持つメソッドと想定する。

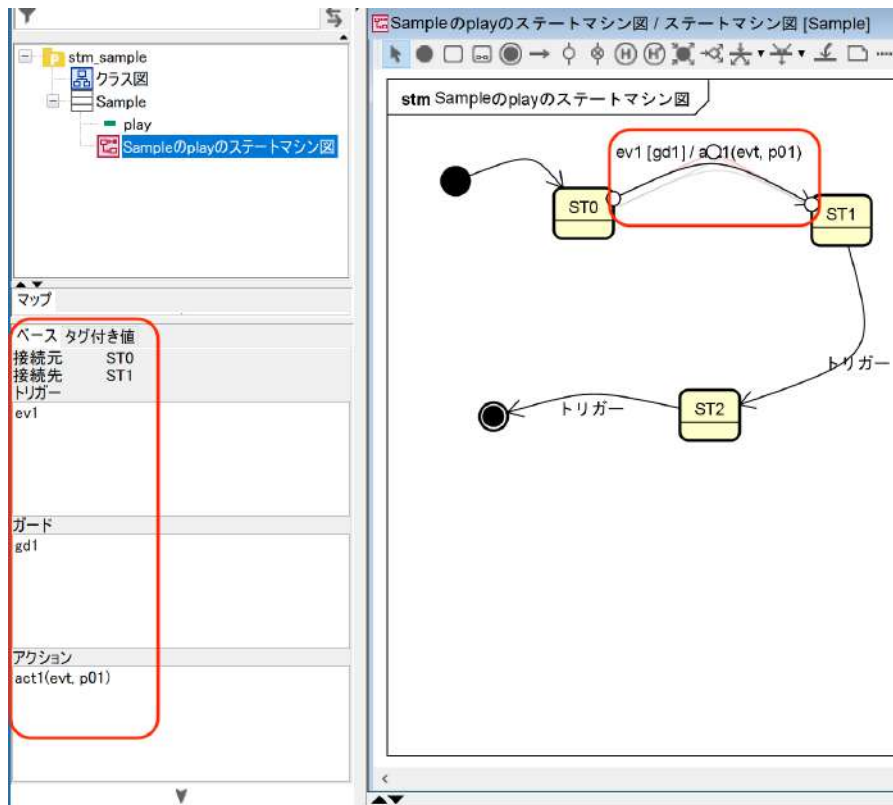


図 3.15 「ST0」から「ST1」への状態遷移のイベントとアクションを編集する

次に、「ST1」からの「ST2」への遷移です。この遷移に割り当てるイベントは、「ev2」、アクションを「act2」とします。そして、このアクションの実行後、ガード条件「gd2」が真なら「ST2」へ、偽ならアクション「act3」を実行してから「ST1」へ遷移するように変更してみます。

遷移先が2つあるなら、状態遷移を別々に引けばよさそうです。ところが、同じイベントを待っている遷移先が2つ以上あると、どちらの状態へ遷移するのか決定できなくなります（あいまいな状態遷移と呼びます）。

そこで、1つのイベントによる遷移先が状況によって変わる場合には「選択疑似状態」を使います。

状態遷移イベントやアクションを追加する (2)

1. パレットから「選択疑似状態」を選択し、状態マシン図に追加する（図 3.16）。

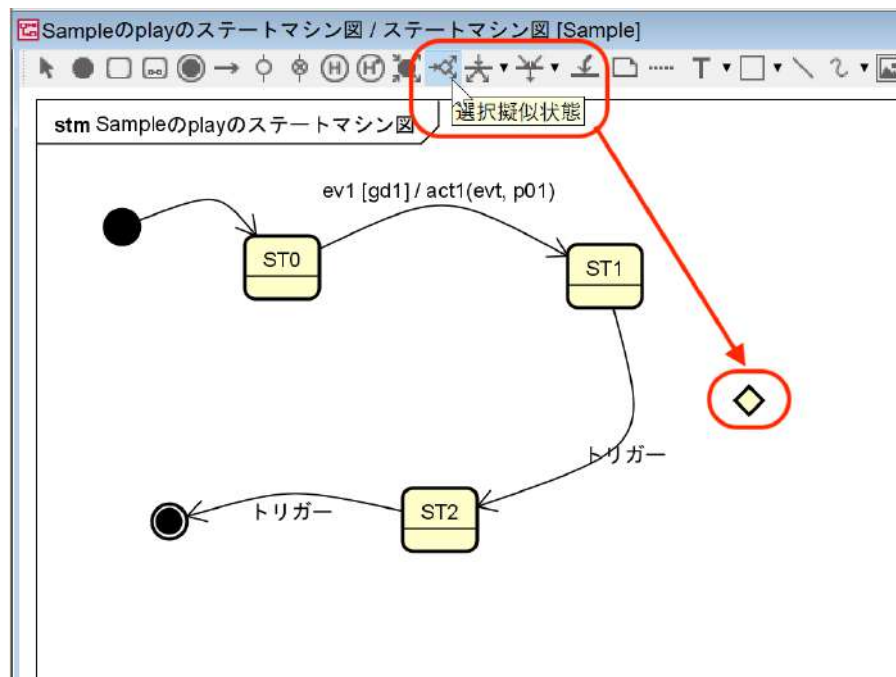


図 3.16 「選択疑似状態」をステートマシン図に追加する

2. 「ST1」から「ST2」への状態遷移を選択し、「ST2」側のハンドル（丸印）をマウスでつまみ、「選択疑似状態」へつなぎ直す（図 3.17）。

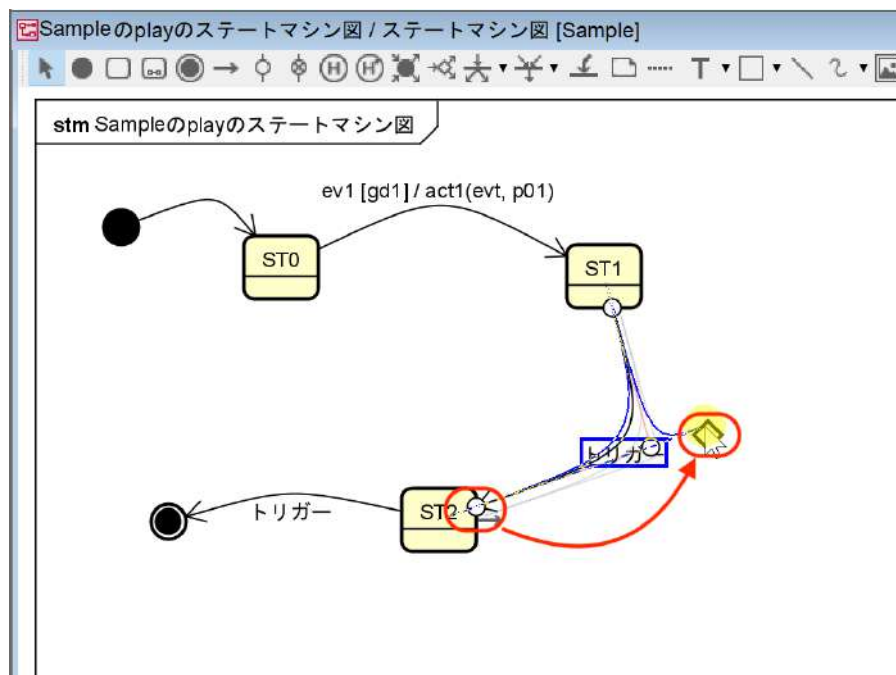


図 3.17 既存の状態遷移を追加した「選択疑似状態」へつなぎ直す

3. 「選択疑似状態」から「ST0」と「ST2」への状態遷移を追加する（図 3.18）。

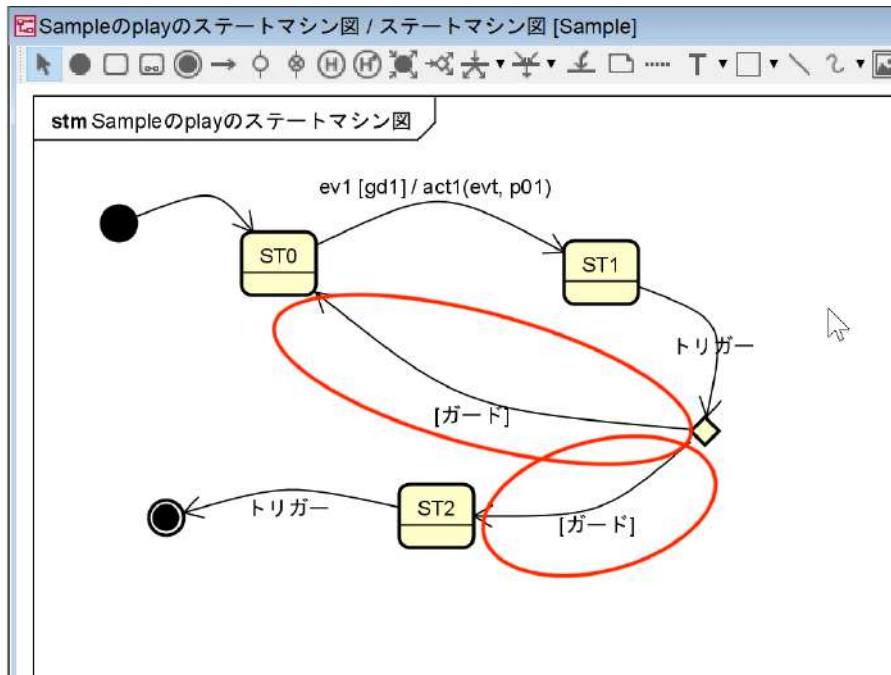


図 3.18 「選択疑似状態」から「ST0」と「ST2」への状態遷移を追加する

4. それぞれの遷移のプロパティを編集する（図 3.19）。

- トリガーに「ev2」、ガード条件はなし、アクションを「act2」に設定する。
- ガードにガード条件「!gd2 (gd2ではない)」、アクションを「act3」に設定する。
- ガードにガード条件「gd2」に設定する。

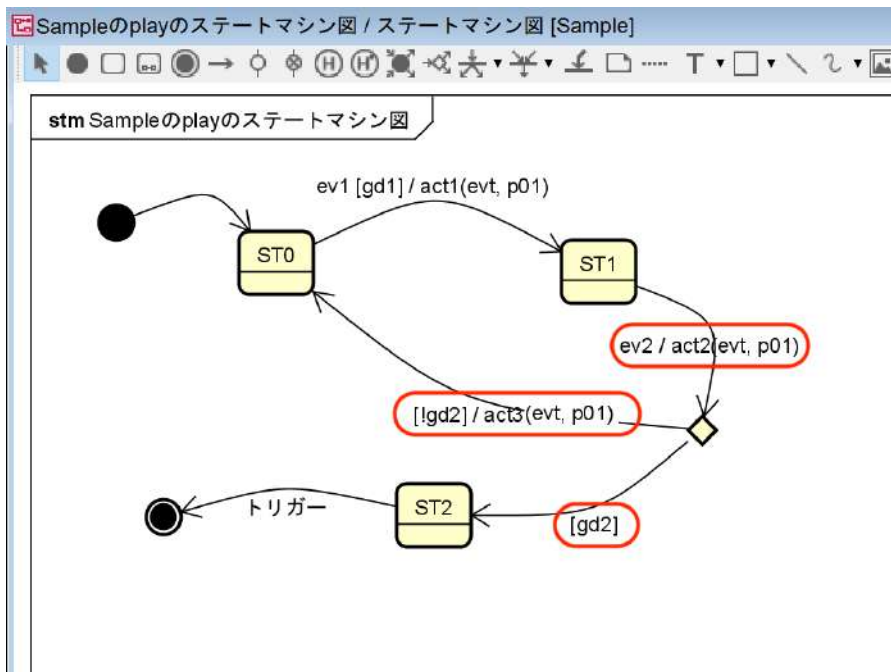


図 3.19 状態遷移のイベントとアクションを編集する

5. 同様にして、もう2つほど状態遷移を追加する（図 3.20）。

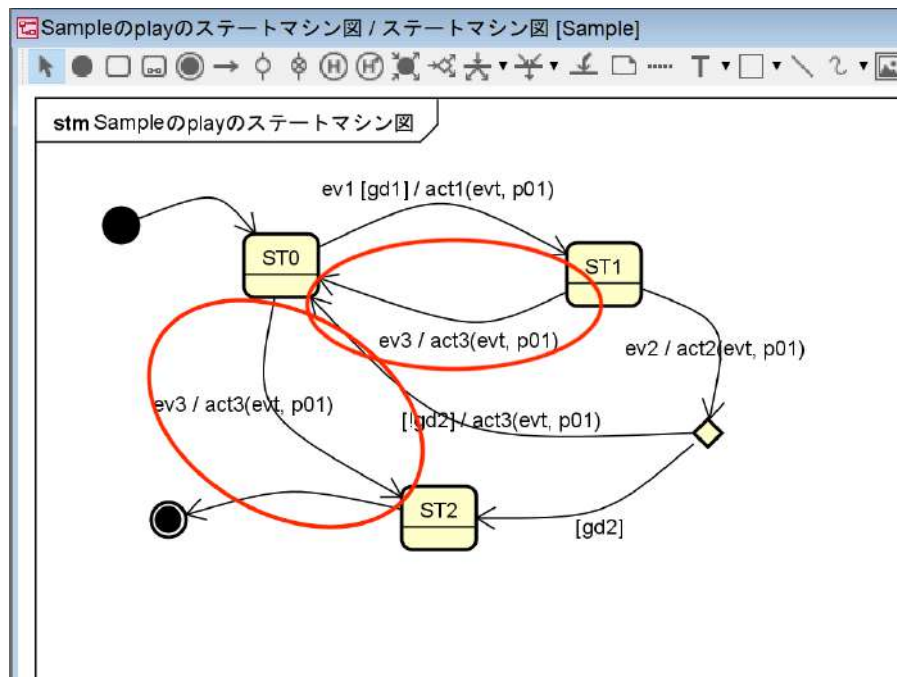


図 3.20 さらに状態遷移とイベントとアクションを追加した

3.5 振る舞いのモデルにコードを対応づける

細かい説明の前に、このチュートリアルで使うC言語のコードとステートマシン図の要素の対応づけを図で表してみましょう（図 3.21）。

NOTE

この段階のサンプルコードは、[codes/sample02](#) ディレクトリに用意してあります。

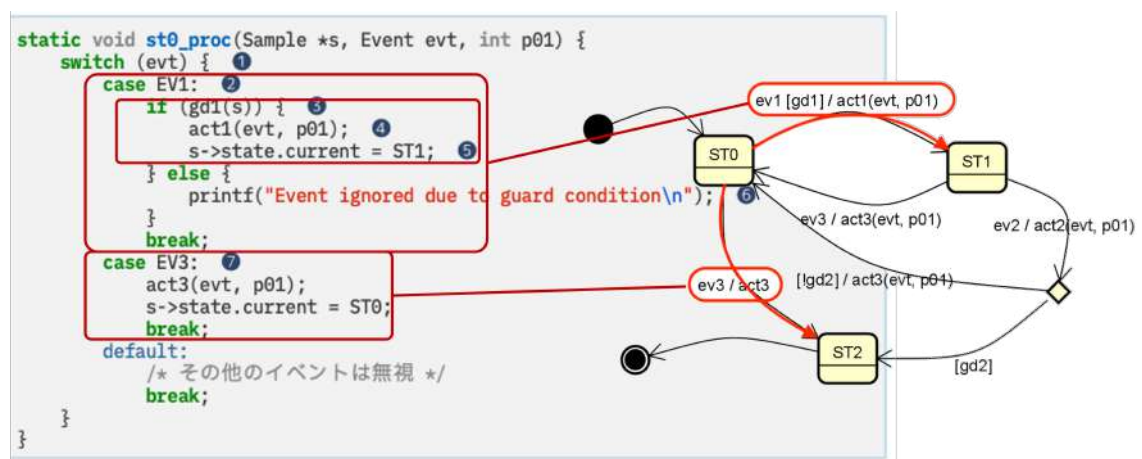


図 3.21 C言語のコードとステートマシン図の対応関係（1）

図 3.21 は、状態「ST0」のイベントとアクションを処理する `st0_proc` 関数とステートマシン図の関係を示しています。

図 3.21 に示した対応関係は、次のようなルールとして説明できるでしょう。

ガード条件のある状態遷移をC言語のコードで表すルール

1. 遷移元の状態ごとに場合分けする。
2. 状態の中で待っているイベントごとに場合分けする。
3. イベントごとの処理の中でアクションと状態遷移を処理する。
 - ガード条件なしのときは（アクションがあればそれを呼び出した後）、次の状態へ遷移する。
 - ガード条件があるときは、ガード条件を評価する。
 - ガード条件が真のときは、アクションを実行して次の状態へ遷移する。
 - ガード条件が偽のときは、アクションは実行されず、状態も遷移しない。

3.6 ステートマシン図に対応するコードを作成する

ステートマシン図と対応づけのルールができたので、この図に合うようなC言語のコードを作成してみましょう。このコードの作成方法も、このチュートリアルで使う対応づけのルールになります。

3.6.1 状態を表す定数を追加する

ステートマシン図の「状態」をC言語で表現するために、`enum`（列挙型）を使いましょう（リスト 3.4）。

リスト 3.4 `sample.h`: 状態の定義

[C]

```
1 #ifndef SAMPLE_H
2 #define SAMPLE_H
3
4 #include <stdbool.h>
5
6 typedef enum {
7     ST0, ❶
8     ST1,
9     ST2,
10    ST_INVALID
11 } State; ❷
12
13 typedef struct {
14     State current;
15 } SampleState; ❸
16
17 void sample_state_init(SampleState *ss); ❹
```

- ❶ 各状態を列挙型の値として定義。
- ❷ 状態を表す型名。
- ❸ 状態を保持する変数をメンバーに持つ構造体。
- ❹ 状態を保持する構造体変数を初期化する関数。

3.6.2 イベントを表す定数を追加する

ステートマシン図の「イベント」も `enum` で定義します（リスト 3.5）。

リスト 3.5 `sample.h`: イベントの定義

[C]

```
19 typedef enum {
20     EV1, ①
21     EV2,
22     EV3,
23     EV_UNKNOWN
24 } Event; ②
```

① 各イベントを列挙型の値として定義。

② イベントを表す型名。

3.6.3 Sample構造体に状態変数を追加する

`Sample`構造体に現在の状態を保持する変数を追加します（リスト 3.6）。

リスト 3.6 `sample.h`: 状態変数を持つ`Sample`構造体

[C]

```
26 typedef struct {
27     bool attr_a;
28     bool attr_b;
29     SampleState state;
30 } Sample; ①
```

① `Sample`構造体に現在の状態を保持する変数 `state` を追加した。

3.6.4 公開メソッドの宣言を追加する

`Sample`クラスの操作を公開関数として定義するため、ヘッダーファイルにプロトタイプ宣言を追加します（リスト 3.7）。

リスト 3.7 `sample.h`: 公開関数のプロトタイプ宣言

[C]

```
32 void sample_init(Sample *s);
33 void sample_play(Sample *s, Event evt, int p01); ①
```

① イベントとイベントの引数を追加したので、`sample01` のときと宣言が変わっていることに注意。

3.6.5 インスタンス作成処理の実装

インスタンスの作成（実際には与えられたインスタンス用変数の初期化）の処理は、`sample_init` 関数として実装します（リスト 3.8）。

リスト 3.8 `sample.c`: インスタンス作成処理（「init」関数）の実装

[C]

```
1 #include <stdio.h>
2 #include "sample.h" ①
3
4 void sample_init(Sample *s) {
5     s->state.current = ST0; ②
```

```

6     s->attr_a = true;
7     s->attr_b = true;
8 }

```

- ① `sample.h` のインクルード。
- ② 状態の初期値を「ST0」に設定する。

クラスを使う時に必要となるヘッダーファイルは、できるだけヘッダーファイルでインクルードしておくこととします。

3.6.6 状態遷移を担当する関数を作成する

ステートマシン図に書いた状態遷移を担当する操作「play」をplayメソッドとして作成します（リスト 3.9）。ルールの検討用なので、途中には処理の確認用の表示処理を書いておきます。

リスト 3.9 `sample.c`: 「play」メソッドの実装 [C]

```

71 void sample_play(Sample *s, Event evt, int p01) { ①
72     switch (s->state.current) { ②
73     case ST0: ③
74         st0_proc(s, evt, p01); ④
75         break;
76     case ST1:
77         st1_proc(s, evt, p01);
78         break;
79     case ST2:
80         // none
81         break;
82     case ST_INVALID:
83         printf("invalid state: %d.\n", s->state.current);
84         break;
85     }
86 }

```

- ① イベントとイベントの引数を追加したので、`sample01` のときと宣言が変わっていることに注意。
- ② 現在の状態で場合分けする。
- ③ 状態「ST0」の場合。
- ④ その状態の処理を行う関数を呼び出す。

3.6.7 ガード条件の判定用関数を作成する

このサンプルのステートマシン図には、状態遷移のガード条件として「gd1」「gd2」が登場します。これらをC言語の関数として作成します。サンプルとして、属性の値を使った演算を割り当てました（リスト 3.10）。

リスト 3.10 `sample.c`: ガード条件のメソッドを作成する [C]

```

10 static bool gd1(const Sample *s) { ①
11     return s->attr_a && s->attr_b; ②
12 }
13
14 static bool gd2(const Sample *s) { ③
15     return !s->attr_a || s->attr_b;
16 }

```


- ① ガード条件「gd1」判定用の関数。
- ② 属性値を使った条件式の例。
- ③ 「gd2」用の関数。

TIP

C言語での関数のスコープ

C言語では、関数名を **static** キーワードで修飾することで関数のスコープを制限できます。

- **static** 宣言のない関数: 他のファイルから呼び出し可能（public相当）
- **static** 宣言付きの関数: 同じファイル内からのみ呼び出し可能（private相当）

st0_proc などの内部処理関数は、**static** 宣言付きの関数にして、外部から直接呼び出せないようにします。

3.6.8 状態ごとの処理用関数を作成する（1）

状態ごとの処理を受け持つメソッドを作成します。メソッド名は、状態名を小文字して「_proc」をつけるというルールにします。したがって、状態名「ST0」の場合、関数名は「st0_proc」になります（リスト 3.11）。

リスト 3.11 sample.c: 「ST0」の状態遷移の関数を作成する

[C]

```

30 static void st0_proc(Sample *s, Event evt, int p01) {
31     switch (evt) { ①
32         case EV1: ②
33             if (gd1(s)) { ③
34                 act1(evt, p01); ④
35                 s->state.current = ST1; ⑤
36             } else {
37                 printf("Event ignored due to guard condition\n"); ⑥
38             }
39             break;
40         case EV3: ⑦
41             act3(evt, p01);
42             s->state.current = ST0;
43             break;
44         default:
45             /* その他のイベントは無視 */
46             break;
47     }
48 }
```

- ① イベントで場合分けする。
- ② イベントが「ev1」の場合。イベントはenumのシンボルを使って表す。
- ③ ガード条件による判定。
- ④ アクション「act1」の呼び出し。
- ⑤ 次の状態「ST1」への遷移。
- ⑥ ガード条件が偽だったとき。イベントが無視されたことを表示する。
- ⑦ イベントが「ev3」の場合。

改めて、この部分のC言語のコードとステートマシン図の対応を、図で確認してみましょう（図 3.22）。

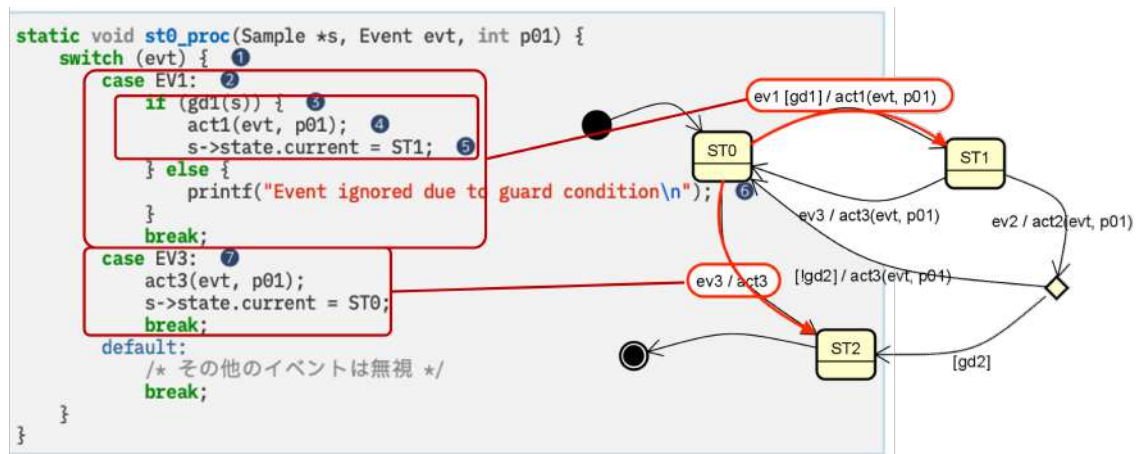


図 3.22 C言語のコードとステートマシン図の対応関係（再掲）

おおむねのことは図 3.22 を読めばわかるでしょうが、対応関係を説明しておきます。

選択疑似状態がある状態遷移をC言語のコードで表すルール

1. 選択疑似状態の前の状態遷移については、図 3.21 のルールで賄う。
2. その後で、選択疑似状態の後の状態遷移のガード条件を評価する。
3. 評価結果によって（アクションがあれば実行してから）、次の状態へ遷移する。
4. 選択疑似状態の後の状態遷移は、ガード条件による場合分けで漏れる場合がないように注意する。

3.6.9 状態遷移を追加する（2）

こんどは、「ST1」からの状態遷移に対する関数「st1_proc」です（リスト 3.12）。

リスト 3.12 sample.c: 「ST1」の状態遷移の関数を作成する

[C]

```

50 static void st1_proc(Sample *s, Event evt, int p01) {
51     switch (evt) { ①
52     case EV2: ②
53         act2(evt, p01); ④
54         if (gd2(s)) { ③
55             act3(evt, p01); ④
56             s->state.current = ST0; ⑤
57         } else {
58             s->state.current = ST2; ⑥
59         }
60         break;
61     case EV3: ⑦
62         act3(evt, p01);
63         s->state.current = ST0;
64         break;
65     default:
66         /* その他のイベントは無視 */
67         break;
68     }

```

69 }

- ① イベントが「ev2」の場合。
- ② アクション「act2」の呼び出し。
- ③ 選択疑似状態におけるガード条件による判定。
- ④ 次の状態「ST2」への遷移。
- ⑤ 選択疑似状態からの遷移におけるアクション「act3」の呼び出し。
- ⑥ 次の状態「ST0」への遷移。
- ⑦ イベントが「ev3」の場合。

この部分のC言語のコードとステートマシン図の対応を図で表してみましょう（図 3.23）。

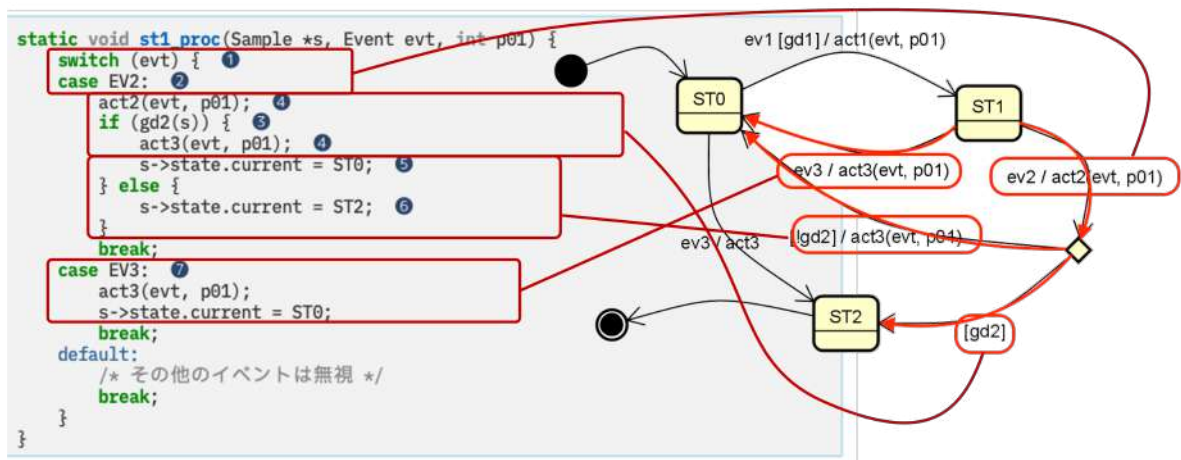


図 3.23 C言語のコードとステートマシン図の対応関係（2）

3.6.10 アクションを処理する関数を追加する

このサンプルでは、アクション「act1」、「act2」、「act3」は、定めたルールを確認しやすいメソッドにしておきます（リスト 3.13）。これらのアクションと、先に挙げたガード条件をチェックするメソッドは、「Sample」クラス内部のメソッドだけが使います。そこで、C言語のコードでもこれらのメソッドのスコープをプライベートにしましょう。C言語では、関数名を「static」で修飾すると、そのファイルの内部では参照できますが、外部からは参照できない関数になります。この方法を使えば、状態遷移の操作をクラスの外から参照できないプライベートな関数として実装できます。

リスト 3.13 sample.c: アクションの処理用の関数を作成する

[C]

```
18 static void act1(Event evt, int prm) { ①
19     printf("    act1: event:%d, param: %d\n", evt, prm);
20 }
21
22 static void act2(Event evt, int prm) { ②
23     printf("    act2: event:%d, param: %d\n", evt, prm);
24 }
25
26 static void act3(Event evt, int prm) { ③
27     printf("    act3: event:%d, param: %d\n", evt, prm);
28 }
```

- ① アクション「act1」の処理用関数。
- ② アクション「act2」の処理用関数。
- ③ アクション「act3」の処理用関数。

これで、ステートマシン図で表したクラスの振る舞いをC言語のコードに対応づけられました。

3.7 追加したアクションをクラス図に反映する

ステートマシン図を作成するときに追加したアクションは、このステートマシン図を割り当てているクラス（ここでは「Sample」クラス）の操作にしておきましょう。

また、ここで、「Sample」クラスをテストするクラス「SampleTest」も追加しておきましょう。

イベントやアクションをクラス図に反映する

1. クラス図を開く。
2. 「Sample」クラスを選択し、プロパティから「操作」タブを表示する。
3. 「+」アイコンを使って操作「act1」を追加する。
4. 可視性を「private」に設定する。
5. 「act2」、「act3」についても同じように設定する（図 3.24）。

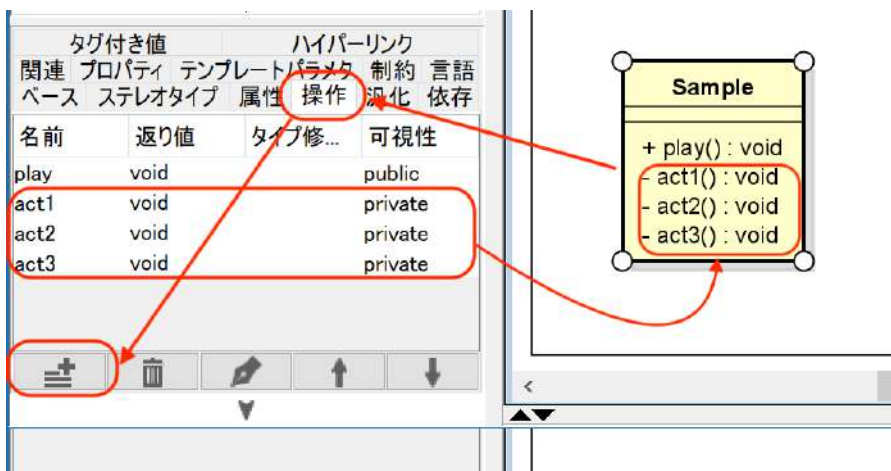


図 3.24 「Sample」クラス内部で使う操作「act1」、「act2」、「act3」を追加する

6. ガード条件に使うメソッド「gd1?」、「gd2?」も追加しておく。
 - C言語の関数名は、gd1、gd2 とする。
7. 「SampleTest」クラスを追加する（図 3.25）。
 - このテスト用クラスには、テストを実行する「run」メソッドを用意しておく。
 - テスト用のメソッドをいくつか追加しておく。
8. 「SampleTest」から「Sample」クラスへ関連を引き、関連端名を「samp01」、多重度を「1」としておく。

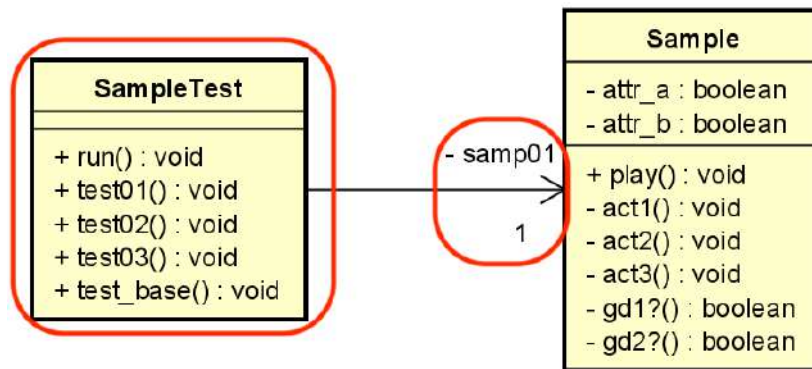


図 3.25 「SampleTest」クラスと関連を追加する

これで、クラス図とステートマシン図とC言語のコードの対応づけができました。実際の問題について、クラス図やステートマシン図を作成するときは、このようなルールを使う前提で作成します。そして、C言語でコードを作成するときは、作成した図と対応づけのルールを使ってコードを作成します。

【コラム5】 列挙型 (enum) をクラス図に反映する方法について【参考】

UMLにはenumのような列挙子を直接定義する記法がないので、このチュートリアルのため、図中の状態やイベントを目視で洗い出してC言語のenumに割り当てました。

astah* は、Java, C#, C++ については、スレオタイプを使ってenumを表現する方法を提案していますので、その方法で代用してもよいでしょう。

詳しい設定方法については、次の記事を参考にしてみてください。

Java、enumの設定

http://astah-users.change-vision.com/ja/modules/xhnewbb/viewtopic.php?topic_id=1249

3.8 変換ルールを使ったコードを確認する

検討用のクラス図とステートマシン図に対応づけたC言語のプログラムの動作を確認してみましょう。

NOTE

この段階のサンプルコードは、`codes/sample03` ディレクトリに用意してあります。

3.8.1 検討用モデルに対応したC言語プログラムの全体

作成したプログラムの全体をリスト 3.14、リスト 3.15、リスト 3.16、リスト 3.17、リスト 3.18 に示します。

これらのソースコードには、「SampleTest」のインスタンスを作成して、テストを起動する処理や、テストメソッドの具体例が含まれています。また、実行結果をみやすくするために、enumの

値を文字列に変換するユーティリティ関数も追加しました。

リスト 3.14 sample.h

[C]

```

1  #ifndef SAMPLE_H
2  #define SAMPLE_H
3
4  #include <stdbool.h>
5
6  /* ----- State ----- */
7
8  typedef enum {
9      ST0,
10     ST1,
11     ST2,
12     ST_INVALID
13 } State;
14
15 /* ----- SampleState ----- */
16
17 typedef struct {
18     State current;
19 } SampleState;
20
21 void sample_state_init(SampleState *ss);
22 State sample_state_transit_to(SampleState *ss, State new_state);
23
24 /* ----- Event ----- */
25
26 typedef enum {
27     EV1,
28     EV2,
29     EV3,
30     EV_UNKNOWN
31 } Event;
32
33 /* ----- Sample ----- */
34
35 typedef struct {
36     bool attr_a;
37     bool attr_b;
38     SampleState state;
39 } Sample;
40
41 void sample_init(Sample *s);
42 void sample_play(Sample *s, Event evt, int p01);
43
44 const char *state_name(State s); ①
45 const char *event_name(Event e); ②
46
47 #endif /* SAMPLE_H */

```

① 状態名を文字列に変換するためのユーティリティ関数。

② イベント名を文字列に変換するためのユーティリティ関数。

リスト 3.15 sample.c

[C]

```

1  #include <stdio.h>
2  #include "sample.h"
3
4  /* ----- State ----- */
5
6  const char *state_name(State s) { ①
7      switch (s) {

```



```

8     case ST0: return "ST0";
9     case ST1: return "ST1";
10    case ST2: return "ST2";
11    default: return "UNKNOWN";
12  }
13 }
14
15 /* ----- SampleState ----- */
16
17 void sample_state_init(SampleState *ss) {
18     ss->current = ST0;
19 }
20
21 State sample_state_transit_to(SampleState *ss, State new_state) {
22     if (new_state < ST_INVALID) {
23         ss->current = new_state;
24     } else {
25         printf("invalid state: %d.\n", new_state);
26     }
27     return ss->current;
28 }
29
30 /* ----- Event ----- */
31
32 const char *event_name(Event e) { ②
33     switch (e) {
34         case EV1: return "ev1";
35         case EV2: return "ev2";
36         case EV3: return "ev3";
37         default: return "unknown";
38     }
39 }
40
41 /* ----- Sample ----- */
42
43 void sample_init(Sample *s) {
44     sample_state_init(&s->state);
45     s->attr_a = true;
46     s->attr_b = true;
47 }
48
49 static bool gd1(const Sample *s) {
50     return s->attr_a && s->attr_b;
51 }
52
53 static bool gd2(const Sample *s) {
54     return !s->attr_a || s->attr_b;
55 }
56
57 static void act1(Event evt, int prm) {
58     printf("    act1: event:%s, param: %d\n", event_name(evt), prm);
59 }
60
61 static void act2(Event evt, int prm) {
62     printf("    act2: event:%s, param: %d\n", event_name(evt), prm);
63 }
64
65 static void act3(Event evt, int prm) {
66     printf("    act3: event:%s, param: %d\n", event_name(evt), prm);
67 }
68
69 static void st0_proc(Sample *s, Event evt, int p01) {
70     switch (evt) {
71         case EV1:
72             if (gd1(s)) {
73                 printf("    gd1: true\n");

```



```

74         act1(evt, p01);
75         sample_state_transit_to(&s->state, ST1);
76     } else {
77         printf("    <<< gd1: false, transition is ignored. >>>\n");
78     }
79     break;
80 case EV3:
81     act3(evt, p01);
82     sample_state_transit_to(&s->state, ST2);
83     break;
84 default:
85     break;
86 }
87 }
88
89 static void st1_proc(Sample *s, Event evt, int p01) {
90     switch (evt) {
91     case EV2:
92         act2(evt, p01);
93         printf("    gd2: %s\n", gd2(s) ? "true" : "false");
94         if (gd2(s)) {
95             sample_state_transit_to(&s->state, ST2);
96         } else {
97             act3(evt, p01);
98             sample_state_transit_to(&s->state, ST0);
99         }
100        break;
101     case EV3:
102         act3(evt, p01);
103         sample_state_transit_to(&s->state, ST0);
104         break;
105     default:
106         break;
107     }
108 }
109
110 void sample_play(Sample *s, Event evt, int p01) {
111     printf("%s ->\n", state_name(s->state.current));
112     printf("    event:%s, param: %d\n", event_name(evt), p01);
113
114     switch (s->state.current) {
115     case ST0: st0_proc(s, evt, p01); break;
116     case ST1: st1_proc(s, evt, p01); break;
117     case ST2: /* none */ break;
118     default: break;
119     }
120
121     printf("    -> %s\n", state_name(s->state.current));
122     if (s->state.current == ST2) {
123         printf("finished.\n");
124     }
125 }

```

- ① 状態名を文字列に変換するためのユーティリティ関数。
- ② イベント名を文字列に変換するためのユーティリティ関数。

リスト 3.16 sample_test.h

[D]

```

1 #ifndef SAMPLE_TEST_H
2 #define SAMPLE_TEST_H
3
4 #include "sample.h"
5
6 void sample_test_run(void);
7

```

```
8 #endif /* SAMPLE_TEST_H */
```

テスト関数の詳細は割愛します。

リスト 3.17 sample_test.c

[C]

```
1 #include <stdio.h>
2 #include <time.h>
3 #include "sample_test.h"
4
5 static void test_base(const Event *events_list[], int list_size,
6                       const int *sizes, bool g0, bool g1) {
7     for (int i = 0; i < list_size; i++) {
8         Sample samp;
9         sample_init(&samp);
10        samp.attr_a = g0;
11        samp.attr_b = g1;
12
13        for (int j = 0; j < sizes[i]; j++) {
14            int p = (int)(clock() % 1000000);
15            sample_play(&samp, events_list[i][j], p);
16        }
17        printf("-----\n");
18    }
19 }
20
21 static void test01(void) {
22     static const Event seq0[] = { EV1, EV2 };
23     static const Event seq1[] = { EV3 };
24     static const Event seq2[] = { EV1, EV3, EV3 };
25
26     static const Event *events_list[] = { seq0, seq1, seq2 };
27     static const int sizes[] = { 2, 1, 3 };
28
29     test_base(events_list, 3, sizes, true, true);
30 }
31
32 static void test02(void) {
33     static const Event seq0[] = { EV1, EV3 };
34     static const Event seq1[] = { EV1, EV2, EV3 };
35
36     static const Event *events_list[] = { seq0, seq1 };
37     static const int sizes[] = { 2, 3 };
38
39     test_base(events_list, 2, sizes, false, false);
40 }
41
42 static void test03(void) {
43     static const Event seq0[] = { EV1, EV2, EV3 };
44     static const Event seq1[] = { EV3 };
45
46     static const Event *events_list[] = { seq0, seq1 };
47     static const int sizes[] = { 3, 1 };
48
49     test_base(events_list, 2, sizes, true, false);
50 }
51
52 void sample_test_run(void) {
53     printf("=====\n");
54     test01();
55     printf("=====\n");
56     test02();
57     printf("=====\n");
58     test03();
59 }
```

リスト 3.18 main.c

[C]

```
1 #include <stdio.h>
2 #include "sample.h"
3 #include "sample_test.h"
4
5 int main(void) {
6     /* SampleState の単体動作確認 */
7     SampleState ss;
8     sample_state_init(&ss);
9     printf("%s\n", state_name(ss.current));
10
11     printf("%s\n", state_name(sample_state_transit_to(&ss, ST2)));
12     printf("%s\n", state_name(sample_state_transit_to(&ss, ST_INVALID))); /* invalid */
13     printf("%s\n", state_name(sample_state_transit_to(&ss, ST1)));
14     printf("- - -\n");
15
16     /* テスト実行 */
17     sample_test_run();
18
19     return 0;
20 }
```

3.8.2 検討用モデルに対応したプログラムを動かす

コマンドプロンプトを起動して（MacやLinuxならターミナルを起動して）、プログラムをビルドしましょう。ビルドに使う Makefile はリスト 3.19 です。

リスト 3.19 sample03 で使用する makefile

[MAKEFILE]

```
CC      = gcc
CFLAGS  = -std=c99 -Wall -Wextra
TARGET  = sample
SRCS    = main.c sample.c sample_test.c
OBJS    = $(SRCS:.c=.o)

.PHONY: all clean

all: $(TARGET)

$(TARGET): $(OBJS)
    $(CC) $(CFLAGS) -o $@ $^

%.o: %.c
    $(CC) $(CFLAGS) -c $< -o $@

main.o:      main.c sample.h sample_test.h
sample.o:    sample.c sample.h
sample_test.o: sample_test.c sample.h sample_test.h

clean:
    $(RM) $(OBJS) $(TARGET)
```

コンパイラのオプションやマクロの意味

- CC — コンパイラの指定（ここでは gcc を使うことを指定）
- CFLAGS — コンパイラオプション用マクロ
 - -std=c99 — C99規格を指定
 - -Wall — 基本的な警告を有効化

- `-Wextra` — 追加の警告を有効化
- `TAEAGET` — 出力ファイル名を指定
- `SRCS` — コンパイル対象のソースコードファイル
- `OBJS` — `SRCS` の拡張子を `.o` に変えたもの

ビルドには `make` コマンドを使います（リスト 3.20）。Windows環境の場合は、異なるビルド手順が必要かもしれません。

リスト 3.20 `sample` をビルドする

[CONSOLE]

```
make
```

実行プログラム `sample` が作成できたら、実行してみましょう（リスト 3.21）。

動作させると、状態遷移やイベントが表示されます。作成したモデル図を比較して期待した動作をしているか確認してみましょう。

リスト 3.21 `sample` を実行する

[CONSOLE]

```
C:\Users\（演習用ディレクトリ）>cd sample03
C:\Users\（演習用ディレクトリ）>\sample03> ./sample ①
ST0
ST2
invalid state: 3.
ST2
ST1
- - - -
=====
ST0 ->
  event:ev1, param: 6642
  gd1: true
  act1: event:ev1, param: 6642
  -> ST1
ST1 ->
  event:ev2, param: 6644
  act2: event:ev2, param: 6644
  gd2: true
  -> ST2
finished.
-----
ST0 ->
  event:ev3, param: 6644
  act3: event:ev3, param: 6644
  -> ST2
finished.
-----
ST0 ->
  event:ev1, param: 6646
  gd1: true
  act1: event:ev1, param: 6646
  -> ST1
ST1 ->
  event:ev3, param: 6647
  act3: event:ev3, param: 6647
  -> ST0
ST0 ->
  event:ev3, param: 6647
  act3: event:ev3, param: 6647
  -> ST2
finished.
```

```

-----
=====
ST0 ->
  event:ev1, param: 6648
  <<< gd1: false, transition is ignored. >>>
  -> ST0
ST0 ->
  event:ev3, param: 6648
  act3: event:ev3, param: 6648
  -> ST2
finished.
-----
ST0 ->
  event:ev1, param: 6649
  <<< gd1: false, transition is ignored. >>>
  -> ST0
ST0 ->
  event:ev2, param: 6649
  -> ST0
ST0 ->
  event:ev3, param: 6650
  act3: event:ev3, param: 6650
  -> ST2
finished.
-----
=====
ST0 ->
  event:ev1, param: 6651
  gd1: true
  act1: event:ev1, param: 6651
  -> ST1
ST1 ->
  event:ev2, param: 6651
  act2: event:ev2, param: 6651
  gd2: false
  act3: event:ev2, param: 6651
  -> ST0
ST0 ->
  event:ev3, param: 6652
  act3: event:ev3, param: 6652
  -> ST2
finished.
-----
ST0 ->
  event:ev3, param: 6652
  act3: event:ev3, param: 6652
  -> ST2
finished.
-----

```

❶ Windows環境では `sample.exe` となっているかもしれない。

3.9 まとめ

クラス図とステートマシン図で動作を表現すれば、C言語のコードに変換できることがわかりました。

3.9.1 構造のモデルを作成した手順

構造のモデルに登場する構成要素や要素間の関連を見つけ出し、スコアシートの構造を整理しました。

スコアシートの構造のモデルを作成した手順

1. スコアシートに関する構成要素の発見
 - 具体的なスコアシートを観察して、そこにある要素を洗い出してオブジェクト図で表した。
2. スコアシートに関するクラス図の作成
 - オブジェクト図に登場したオブジェクトからクラスを、オブジェクト同士のリンクから関連を見出し、クラス図に表した。

3.9.2 構造のモデルだけではプログラムは作れない

構造のモデルを作ったことで、プログラムを構成する要素と、要素同士の関係（関連）を表せるようになりました。また、クラスのインスタンスをオブジェクト図に見合うように作成すれば、実際に観察したスコアシートと同じようなデータ構造を作成できることもわかりました。

ところで、作りたかったのは、ボウリングのゲームスコアをつけるプログラムでした。つまり、プレイヤーがフレームごとに交代して投球し、そのたびにスコアを更新する処理です。ですが、これまでに作成した構造のモデルでは、ゲームの進行に応じてどんなことを処理するか表せていません。そのためには、処理の中身を表す「振る舞いのモデル」を作成する必要があります。次の章では、ステートマシン図を使って、フレームやスコアの振る舞いを表すモデルを作成します。

第4章 スコアやフレームの状態を調べる

設計で作成するモデルと実装に使うプログラムの対応づけができました。この対応づけを前提に、ボウリングスコアのモデルの作成を進めましょう。

4.1 フレームの状態について検討する

図 2.1 を見ると、フレームの表示は、ゲームの進行状況によって変わっています。

ゲームの進行状況によってフレームの表示は異なる

- まだプレーしていないフレーム
- 1投目の投球を待っているフレーム（現在のプレーヤーの現在のフレームの1投目の前）
- 2投目の投球を待っているフレーム（現在のプレーヤーの現在のフレームの2投目の前）
- 2投目が投球されて獲得ピン数が確定したフレーム（1,2投目のピン数とトータルスコアが表示されている）
- ストライクのボーナスが確定しないフレーム（ストライクの記録だけでトータルスコアは表示されていない）
- スペアのボーナスが確定しないフレーム（1投目とスペアの記録だけでトータルスコアは表示されていない）
- スペアまたはストライクのボーナスが確定したフレーム（1,2投目のピン数とトータルスコアが表示されている）

それぞれについて、もう少し詳しく見てみましょう。

4.1.1 まだプレーしていないフレーム

まだプレーしていないフレームは、フレームの最初の状態です（図 4.1）。このフレームは、ピン数の入力をしていません。

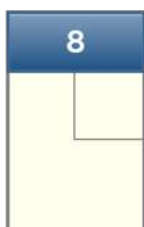


図 4.1 まだプレーしていないフレーム

4.1.2 1投目の投球を待っているフレーム

現在プレー中のフレームで、まだ1投目が投球されていない状態のフレームです（図 4.2）。次に

ピン数を受け取ると、このフレームの1投目に記録されます。



図 4.2 1投目の投球を待っているフレーム

4.1.3 2投目の投球を待っているフレーム

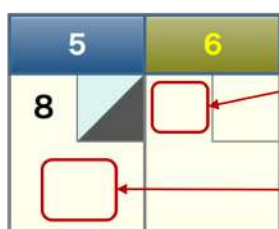
現在プレー中のフレームで、1投目がストライクでなかったときに2投目を待っている状態のフレームです（図 4.3）。次にピン数を受け取ると、このフレームの2投目に記録されます。



図 4.3 2投目の投球を待っているフレーム

4.1.4 スペアのボーナスの確定待ちのフレーム

現在プレー中のフレームの前のフレームがスペアで、現在のフレームが1投目の投球を待っていると、前のフレームはスペアボーナスの確定待ちの状態です（図 4.4）。次にピン数を受け取ると、現在のフレームの1投目に記録されるとともに、前のフレームのスペアボーナスが確定します。



この投球が終わるまで、
前のフレームのボーナス
は確定しない
ボーナスが確定しないと、
トータルが出せない

図 4.4 スペアのボーナスの確定待ちのフレーム

4.1.5 ストライクのボーナスの確定待ちのフレーム

ストライクボーナスの確定待ちには2通りの場合があります。

1つ目は、現在プレー中のフレームの前のフレームがストライクで、現在のフレームが2投目の投球を待っているときです。このとき、前のフレームはストライクボーナスの確定待ちの状態です（図

4.5)。次にピン数を受け取ると、現在のフレームの2投目に記録されるとともに、前のフレームのストライクボーナスが確定します。

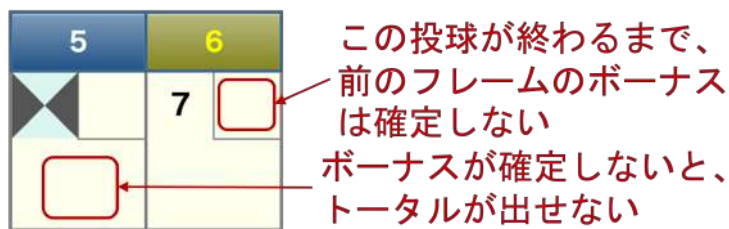


図 4.5 ストライクのボーナスの確定待ちのフレーム（次がストライクでない）

2つ目は、現在プレー中のフレームが1投目の投球を待っていて、前のフレームと前の前のフレームがともにストライクであったとき（ダブルのとき）ときです。このとき、前の前のフレームはストライクボーナスの確定待ちの状態です（図 4.6）。次にピン数を受け取ると、現在のフレームの1投目に記録されるとともに、前の前のフレームのストライクボーナスが確定します。

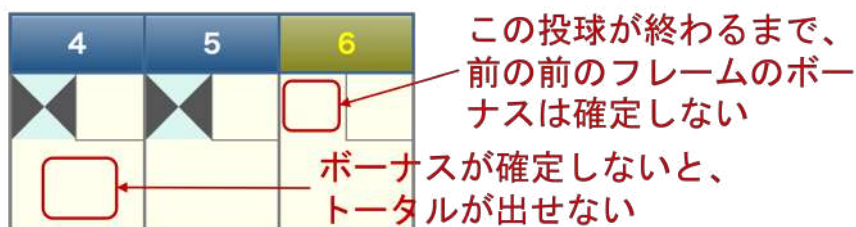


図 4.6 ストライクのボーナスの確定待ちのフレーム（次がストライク）

4.1.6 獲得ピン数とボーナスが確定したフレーム

現在のフレームがスペアやストライクにならなかったとき、そのフレームはボーナスの確定待ちにならず、この段階でそのフレームのトータル（ボーナスなしの獲得ピン数）が確定します。

スペアボーナスの確定待ち、またはストライクボーナスの確定待ちのフレームは、後のフレームの投球によってボーナスが確定すると、フレームのトータルが確定します。このとき、確定待ちのフレームでは、そのフレームの獲得ピン数と確定したボーナスの合計がそのフレームのトータルです。

フレームのトータルが求められると、それ以前のフレームまでの「のべのトータル」にそのフレームのトータルを加算して、のべのトータルを更新します。そして、更新したのべのトータルがフレームの下部に記録されます（図 4.7）。このとき、確定待ちになっていたフレームものべのトータルが更新され、その段階のゲームのトータルも更新されます。

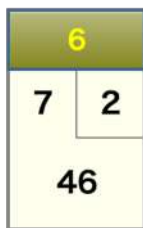


図 4.7 2投目を投球して、ピン数が確定したフレーム（のべのトータルが求められている）

4.2 フレームの状態をステートマシン図で表す

フレームの状態を検討した結果、フレームはゲームの状況によって変わる複数の状態を持つことがわかりました。また、フレームが表示できる情報も、状態によって異なることがわかりました。このことを、モデル図を使って表してみましょう。状態とその推移（状態遷移と呼びます）を表すには、ステートマシン図を使います。

4.2.1 「Frame」クラスにステートマシン図を追加する

「Frame」クラスの状態を表す図を描きたいので、「Frame」クラスに図を追加しましょう。

「Frame」クラスにステートマシン図を追加する

1. 「Frame」クラスにステートマシン図を追加する
 - ・ 構造ツリーから「Frame」クラスを選択し、右クリックしてポップアップメニューを表示する（図 4.8）。
 - ・ 「図の追加>ステートマシン図」でステートマシン図が追加される。

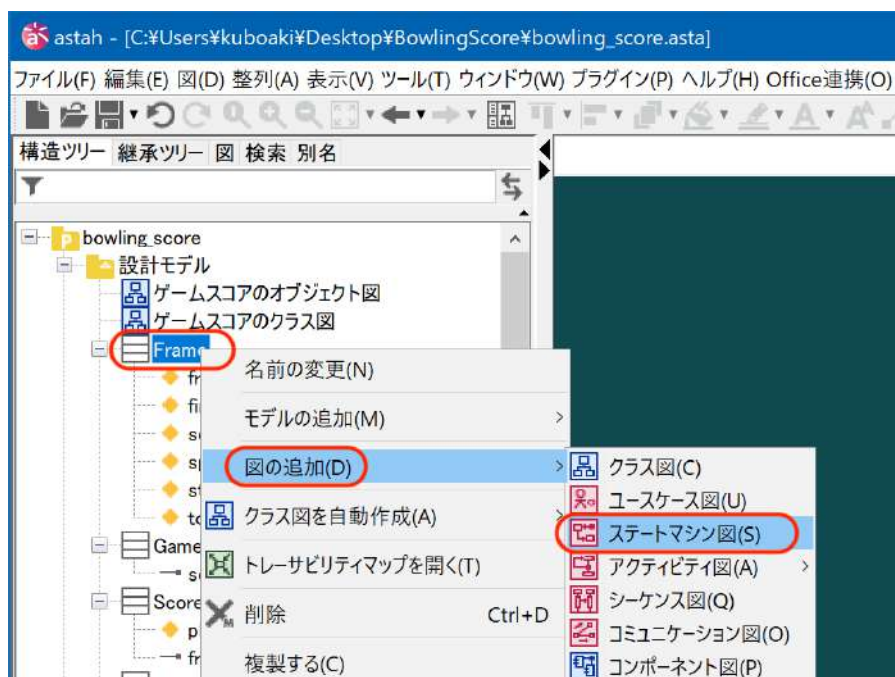


図 4.8 「Frame」クラスにステートマシン図を追加する

2. ステートマシン図に名前をつける

- 追加したステートマシン図のプロパティの「ベース」を開く。
- 名前を編集して「Frameクラスのステートマシン図」とする。
- ダイアグラムエディタのタイトルやタブにも反映される。

4.2.2 フレームのステートマシン図を作成する

ステートマシン図に、フレームの状態と状態遷移を追加しましょう。

ステートマシン図に「Frame」クラスの状態遷移を作成する

1. 「Frame」クラスの最初の状態を作成する（図 4.9）。
 - パレットから「開始疑似状態」を選択し、ステートマシン図に追加する。
 - パレットから「状態」を選択し、ステートマシン図に追加する。
 - 「まだプレーしていない」状態を「RESERVED」という名前にする。
 - パレットから「遷移」を選択して、「開始疑似状態」から「RESERVED」へ遷移を引く。

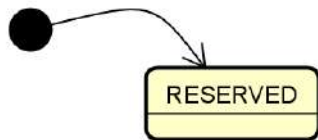


図 4.9 「Frame」クラスのステートマシン図に最初の状態を追加する

2. 最初の状態遷移とイベントを追加する（図 4.10）。
 - 「1投目の投球を待っている」状態「BEFORE_1ST」を追加する。
 - 「RESERVED」から「BEFORE_1ST」へ状態遷移を引き、イベント「SETUP」を割り当てる。

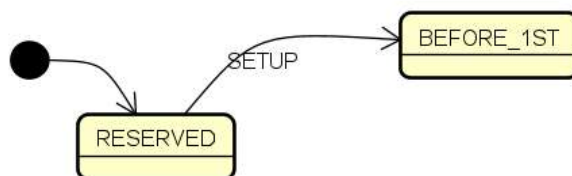


図 4.10 「Frame」クラスのステートマシン図に1投目の前の状態遷移とイベントを追加する

3. 1投目の後の状態遷移とイベントを追加する（図 4.11）。1投目の後は、受け取ったピン数によって遷移先は2通りある（ストライクか否か）ので「選択疑似状態」を使って遷移先を分ける。

- 「2投目の投球を待っている」状態を「BEFORE_2ND」として追加する。
- 「スペアのボーナスの確定待ち」と「ストライクのボーナスの確定待ち」を「PENDING」として追加する。
- 「選択疑似状態」を追加する。
- ピン数を受け取るイベント「PINS」（パラメーターとしてピン数 pins を持つ）を「BEFORE_1ST」から「選択疑似状態」への遷移に割り当てる。アクションとして、イベントで受け取ったピン数を1投目のピン数に保存する。

4. 選択疑似状態からの遷移を追加する。

- 1投目のピン数がストライクであれば「PENDING」へ遷移する。このとき2投目のピン数は「0」にする。
- 1投目のピン数がストライクでなければ「BEFORE_2ND」へ遷移する。

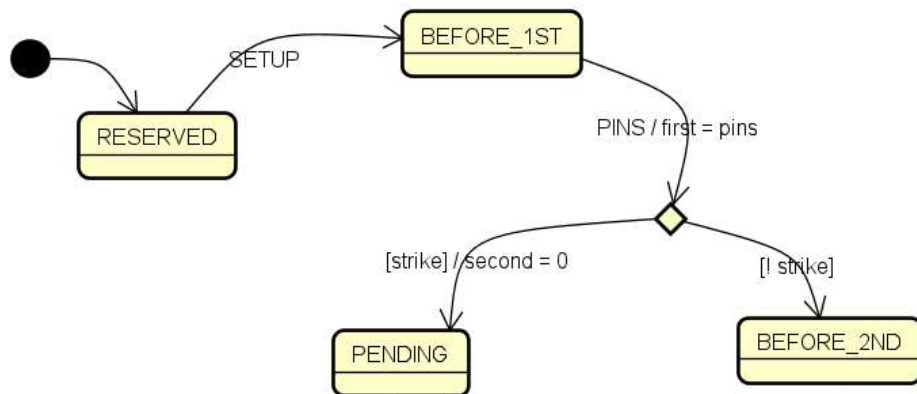


図 4.11 「Frame」クラスのステートマシン図に1投目の後の状態遷移とイベントを追加する

- 2投目の後の状態遷移とイベントを追加する（図 4.12）。2投目の後は、受け取ったピン数によって遷移先は2通りある（スペアか否か）ので「選択疑似状態」を使って遷移先を分ける。
 - 「獲得ピン数とボーナスが確定した」状態を表す「FIXED」として追加する。
 - 2投目の後の処理の選択のために「選択疑似状態」を追加する。
 - 「BEFORE_2ND」でイベント「PINS」を受け取ると、選択疑似状態へ遷移する。アクションとして、受け取ったピン数を2投目のピン数に保存する。
- 選択疑似状態からの遷移を追加する。
 - 2投目のピン数がスペアであれば「PENDING」へ遷移する。
 - 2投目のピン数がスペアでなければ「FIXED」へ遷移する。
- 「PENDING」からの遷移を追加する。
 - 「PENDING」中のフレームで、イベント「DETERMINE」を受け取ったらトータルが確定したとし、「FIXED」へ遷移する。

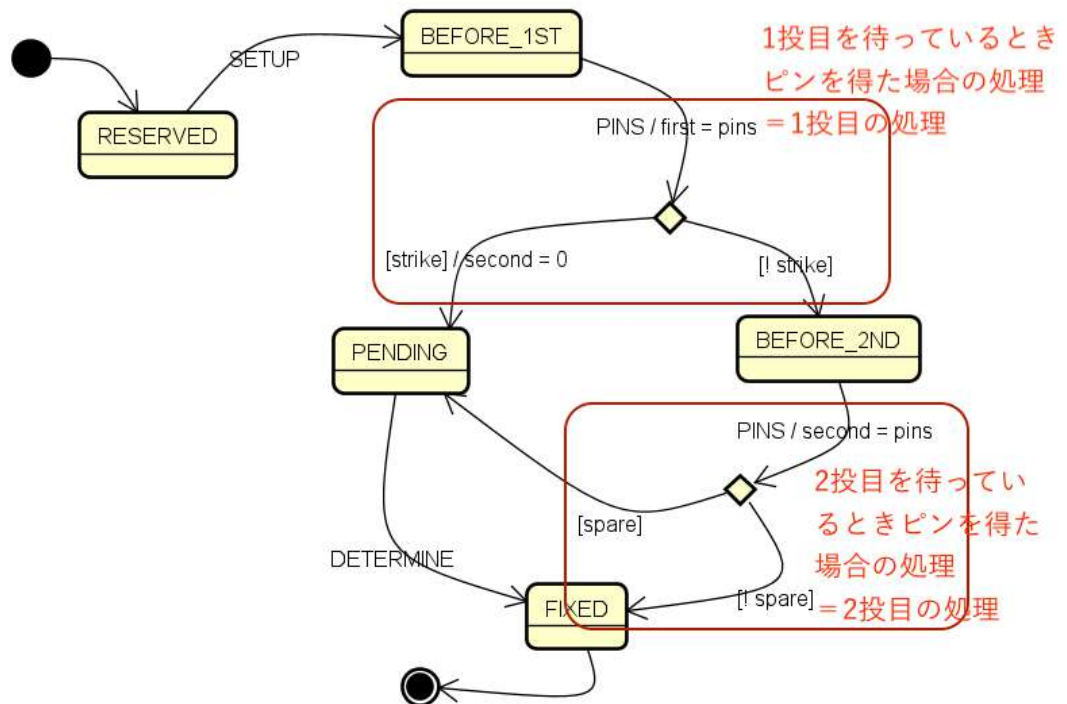


図 4.12 「Frame」クラスのステートマシン図に2投目の後の状態遷移とイベントを追加する

これで、フレームのステートマシン図が作成できました。

4.2.3 フレームのクラス図を更新する

ステートマシン図で状態遷移のために追加したアクションやガード条件用の処理を「Frame」クラスのメソッドに追加しておきましょう（図 4.13）。

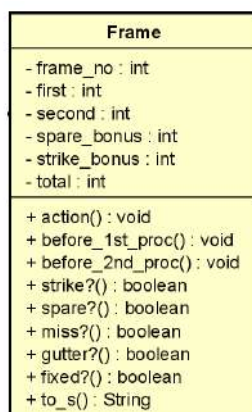


図 4.13 ステートマシン図に合わせて「Frame」クラスを更新する

4.2.4 フレームの処理をプログラムに変換する

フレームのクラスとステートマシン図が作成できたので、C言語のプログラムに変換してみましょう。「第3章」で決めたルールにしたがって、モデルからコードへ変換します。

まず、クラス「Frame」の部分は、リスト 4.1、リスト 4.2 のようになるでしょう。

リスト 4.1 bowling.h (1)

[C]

```

1  /* bowling.h - ボーリングスコア管理システム */
2  #ifndef BOWLING_H
3  #define BOWLING_H
4
5  #include <stdio.h>
6  #include <stdlib.h>
7  #include <string.h>
8  #include <stdbool.h>
9  #include <time.h>
10
11 /* 定数定義 */
12 #define MAX_FRAMES 15      /* ダミーフレーム含む配列サイズ */
13 #define MAX_PLAYERS 10    /* 最大プレイヤー数 */
14 #define MAX_GAMES 20      /* 最大ゲーム数 */
15 #define MAX_NAME_LEN 64   /* プレイヤー名の最大長 */
16 #define ID_LEN 12         /* ID文字列の長さ */
17
18 /* フレームの状態 */
19 typedef enum {
20     RESERVED,
21     BEFORE_1ST,
22     BEFORE_2ND,
23     PENDING,
24     FIXED
25 } FrameState;
26
27 /* イベントタイプ */
28 typedef enum {
29     SETUP,
30     PINS,
31     DETERMINE
32 } EventType;
33
34 /* スコアの状態 */
35 typedef enum {
36     WAIT_FOR_1ST,
37     WAIT_FOR_2ND,
38     FINISHED
39 } ScoreState;
40
41 /* Frame構造体: 1フレーム分のピン数やボーナスを記録 */
42 typedef struct {
43     int frame_no;
44     int first;
45     int second;
46     int spare_bonus;
47     int strike_bonus;
48     int total;
49     FrameState state;
50 } Frame;
51
52 /* Frame関数 */
53 void frame_init(Frame *frame, int frame_no);
54 void frame_action(Frame *frame, EventType event, int pins);
55 int frame_score(const Frame *frame);
56 bool frame_is_strike(const Frame *frame);
57 bool frame_is_spare(const Frame *frame);
58 bool frame_is_miss(const Frame *frame);
59 bool frame_is_gutter(const Frame *frame);
60 bool frame_is_fixed(const Frame *frame);
61 void frame_print(const Frame *frame);

```

リスト 4.2 bowling.c (1)

[C]

```

1  /* bowling.c - ボーリングスコア管理システムの実装 */
2  #include "bowling.h"
3
4  void generate_id(char *id) { ①
5      static int counter = 0;
6      snprintf(id, ID_LEN, "ID%08d", counter++);
7  }
8
9  const char* frame_state_to_string(FrameState state) { ②
10     switch (state) {
11         case RESERVED:    return "RESERVED";
12         case BEFORE_1ST:  return "BEFORE_1ST";
13         case BEFORE_2ND:  return "BEFORE_2ND";
14         case PENDING:     return "PENDING";
15         case FIXED:       return "FIXED";
16         default:          return "UNKNOWN";
17     }
18 }
19
20 /* =====
21  * Frame実装
22  * ===== */
23
24 void frame_init(Frame *frame, int frame_no) { ③
25     frame->frame_no = frame_no;
26     frame->first = 0;
27     frame->second = 0;
28     frame->spare_bonus = 0;
29     frame->strike_bonus = 0;
30     frame->total = 0;
31     frame->state = RESERVED; ④
32 }

```

- ① 簡易的なIDを生成するユーティリティ関数。
- ② FrameStateを文字列に変換するユーティリティ関数。
- ③ フレームの初期化処理。
- ④ フレームの初期状態は「RESERVED」とする。

フレームごとのアクションの振り分けは、リスト 4.3 のようになるでしょう。

リスト 4.3 bowling.c (2)

[C]

```

69 void frame_action(Frame *frame, EventType event, int pins) {
70     switch (frame->state) { ①
71         case RESERVED: ②
72             if (event == SETUP) {
73                 frame->state = BEFORE_1ST;
74             } else {
75                 printf("invalid event is ignored.\n");
76             }
77             break;
78         case BEFORE_1ST: ③
79             frame_before_1st_proc(frame, event, pins);
80             break;
81         case BEFORE_2ND: ④
82             frame_before_2nd_proc(frame, event, pins);
83             break;
84         case PENDING: ⑤
85             if (event == DETERMINE) {
86                 frame->state = FIXED;

```

```

87     }
88     break;
89     case FIXED:
90         printf("fixed.\n");
91         break;
92     }
93 }

```

- ① フレームの状態に応じて処理を分ける。
- ② 「RESERVED」状態では「SETUP」イベントを受け取り、「BEFORE_1ST」状態へ遷移する。他のイベントが来たら無視する。
- ③ 「BEFORE_1ST」状態では「PINS」イベントを待つ。詳細な処理は「before_1st_porc」関数に記載する。
- ④ 「BEFORE_2ND」状態では「PINS」イベントを待つ。詳細な処理は「before_2nd_porc」関数に記載する。
- ⑤ 「PENDING」状態では「DETERMINE」イベントを受け取り、「FIXED」状態へ遷移する。

アクションやガード条件判定用の関数メソッドは、リスト 4.4、リスト 4.5 のようになるでしょう。

リスト 4.4 bowling.c (3)

[C]

```

34 static void frame_before_1st_proc(Frame *frame, EventType event, int pins) {
35     if (event == PINS) {
36         if (pins < 0 || pins > 10) {
37             printf("invalid pins: %d\n", pins);
38             return;
39         }
40         frame->first = pins;
41         if (frame_is_strike(frame)) {
42             frame->second = 0;
43             frame->state = PENDING;
44         } else {
45             frame->state = BEFORE_2ND;
46         }
47     } else {
48         printf("invalid event on %s.\n", frame_state_to_string(frame->state));
49     }
50 }
51
52 static void frame_before_2nd_proc(Frame *frame, EventType event, int pins) {
53     if (event == PINS) {
54         if (pins < 0 || pins > (10 - frame->first)) {
55             printf("invalid pins: %d\n", pins);
56             return;
57         }
58         frame->second = pins;
59         if (frame_is_spare(frame)) {
60             frame->state = PENDING;
61         } else {
62             frame->state = FIXED;
63         }
64     } else {
65         printf("invalid event on %s.\n", frame_state_to_string(frame->state));
66     }
67 }

```

リスト 4.5 bowling.c (4)

[C]

```

95 int frame_score(const Frame *frame) {
96     return frame->first + frame->second + frame->spare_bonus + frame->strike_bonus;
97 }
98
99 bool frame_is_strike(const Frame *frame) {
100     return frame->first == 10;
101 }
102
103 bool frame_is_spare(const Frame *frame) {
104     return frame->first < 10 && (frame->first + frame->second) == 10;
105 }
106
107 bool frame_is_miss(const Frame *frame) {
108     return frame->first < 10 && frame->second == 0 && frame->state == FIXED;
109 }
110
111 bool frame_is_gutter(const Frame *frame) {
112     return frame->first == 0;
113 }
114
115 bool frame_is_fixed(const Frame *frame) {
116     return frame->state == FIXED;
117 }
118
119 void frame_print(const Frame *frame) {
120     char total_str[8];
121     if (frame->state == FIXED) {
122         snprintf(total_str, sizeof(total_str), "%5d", frame->total);
123     } else {
124         snprintf(total_str, sizeof(total_str), "  .");
125     }
126     printf("|%2d|%3d|%3d|%5s|%11d|%11d|%12d|%11s|\n",
127         frame->frame_no, frame->first, frame->second, total_str,
128         frame_score(frame), frame->spare_bonus, frame->strike_bonus,
129         frame_state_to_string(frame->state));
130 }

```

4.3 サービスフレームの扱いについて検討する

「クラシックスコアリング」の場合、第10フレームが他のフレームとは異なります。サービスフレームという考え方があり、ストライクやスペアの場合に追加で投球できます。

このサービスフレームの扱い方について整理しておきましょう。

4.3.1 第10フレームがストライクもスペアもない場合

第10フレームがストライクでもスペアもない場合、サービスフレームは提供されません。第9フレームまでの通常のフレームと同様、1投目と2投目を記録するだけです（図 4.14）。

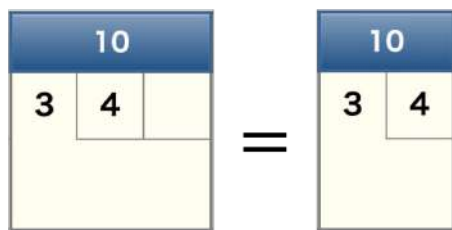


図 4.14 第10フレームがストライクもスペアもない場合

4.3.2 第10フレームがスペアの場合

第10フレームの2投目でスペアになった場合、1投目と2投目を通常のフレームと同様に記録したのち、もう1投追加されます。これを、第10フレームに加えて、第11フレームの1投目が追加されたものとみなします。つまり、ゲームスコアのデータを「図 4.15」のように構成します。

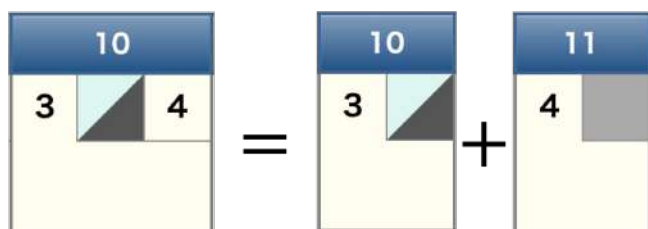


図 4.15 第10フレームがスペアの場合（第11フレームの2投目はない）

スコアをこのように構成しておくと、第10フレームの場合も、通常フレームの組み合わせによってボーナスが計算できます。ゲーム終了時の第10フレームのスコア（ピン数に以後の投球によるボーナスを加えたスコア）を取得すると、それがゲームのスコアです。

4.3.3 第10フレームの1投目がストライクの場合

第10フレームの1投目がストライクの場合、第10フレームをストライクとした上で、もう2投追加されます。これを、第10フレームに加えて、第11フレームの1投目と2投目が追加されたものとみなします。つまり、ゲームスコアのデータを「図 4.16」のように構成します。ただし、第11フレームの1投目がストライクの場合は、同じ2投追加でも次の「2投目もストライクの場合」の考え方を使います。

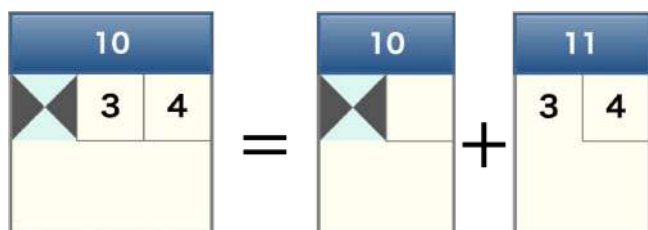


図 4.16 第10フレームの1投目がストライクの場合

スコアをこのように構成しておくと、第10フレームの場合も、通常フレームの組み合わせによってボーナスが計算できます。ゲーム終了時の10フレームのスコア（ピン数に以後の投球によるボーナ

スを加えたスコア)を取得すると、それがゲームのスコアです。

4.3.4 第10フレームの2投目もストライクの場合

第10フレームの2投目もストライクの場合、第10フレーム、第11フレームをストライクとした上で、もう1投追加されます。これを、第10フレーム、第11フレームに加えて、第12フレーム目の1投目が追加されたとみなします。つまり、ゲームスコアのデータを「図 4.17」のように構成します。このように構成すれば、通常のフレームでダブルをとったときの計算方法（ストライクが続いたときはさらに次のフレームの1投目が2投目として加算される）のままで第10フレームのボーナスが計算できます。

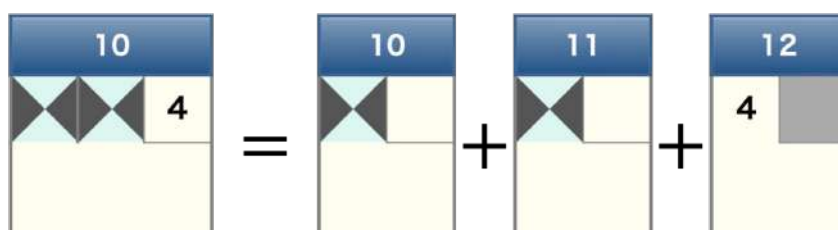


図 4.17 第10フレームの2投目もストライクの場合

スコアをこのように構成しておくと、第10フレームの場合も、通常フレームの組み合わせによってボーナスが計算できます。ゲーム終了時の10フレームのスコア（ピン数に以後の投球によるボーナスを加えたスコア）を取得すると、それがゲームのスコアです。

4.4 スコアの状態をステートマシン図で表す

サービスフレームの扱い方を検討した結果、工夫すれば通常のフレームと同じように扱えることがわかりました。それでは、フレームの集まりであるスコアについて、どのような処理をすればよいのか検討しましょう。

4.4.1 Scoreクラスにステートマシン図を追加する

「Frame」クラスにステートマシン図を追加したのと同じ手順で、「Score」クラスにステートマシン図を追加します（図 4.18）。

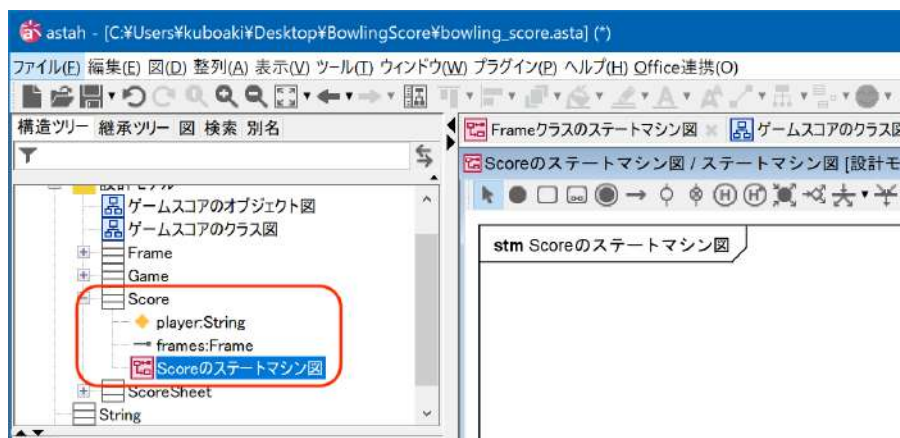


図 4.18 「Score」クラスにステートマシン図を追加する

4.4.2 スコアのステートマシン図を作成する

ステートマシン図に、検討した結果を使って、スコアの状態と状態遷移を追加しましょう。

ステートマシン図に「Score」クラスの状態遷移を作成する

1. 「Score」クラスの状態を追加する（図 4.19）。
 - パレットから「開始疑似状態」をステートマシン図に追加する。
 - パレットから「状態」を選択し、ステートマシン図に追加する。
 - 「1投目待ち」の状態として状態名を「WAIT_FOR_1ST」に設定する。
 - 「2投目待ち」の状態として状態名を「WAIT_FOR_2ND」に設定する。
 - 「ゲームの終了」の状態として状態名を「FINISHED」に設定する。
 - パレットから「終了疑似状態」をステートマシン図に追加する。

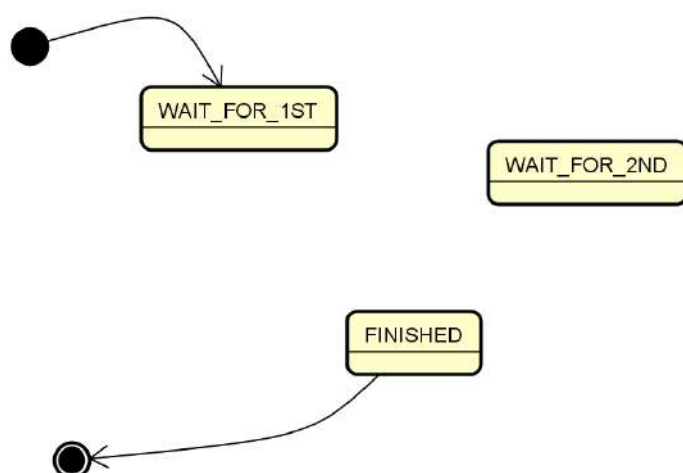


図 4.19 ステートマシン図に「Score」クラスの状態を追加する

2. 「Score」クラスのイベントとアクションを作成する（図 4.20）。
3. 「WAIT_FOR_1ST」からの遷移には、ストライクの場合、ストライクでない場合、ゲームが終了の場合があるので、「選択疑似状態」を追加する。

- 「WAIT_FOR_1ST」から「選択疑似状態」への遷移では、ピン数を受け取るのを待っている。受け取ったときには、現在にフレームへピン数のイベントを送る。その後、1投目の後のスベアとストライクのボーナスを計算し、のべのトータルを更新する。
 - 「選択疑似状態」から「WAIT_FOR_1ST」への遷移のガード条件は「ストライクである」こと。このときはアクションとして「次のフレームへ進む」。
 - 「選択疑似状態」から「WAIT_FOR_2ND」への遷移のガード条件は「ストライクでない（かつゲーム終了ではない）」こと。
 - 「選択疑似状態」から「FINISHED」への遷移のガード条件は「ゲーム終了である」であること。
4. 「WAIT_FOR_2ND」からの遷移には、ゲーム終了の場合とそうでない場合があるので、「選択疑似状態」を追加する。
- 「WAIT_FOR_1ST」から「選択疑似状態」への遷移では、ピン数を受け取るのを待っている。受け取ったときには、現在にフレームへピン数のイベントを送る。そして、2投目の後のスベアとストライクのボーナスを計算し、のべのトータルを更新する。
 - 「選択疑似状態」から「WAIT_FOR_1ST」への遷移のガード条件は「ゲーム終了ではない」こと。このときはアクションとして「次のフレームへ進む」。
 - 「選択疑似状態」から「FINISHED」への遷移のガード条件は「ゲーム終了である」であること。ゲーム終了は、10フレーム目の状態が「FIXED」になったことで判定できる（そのようなメソッドを用意する）。

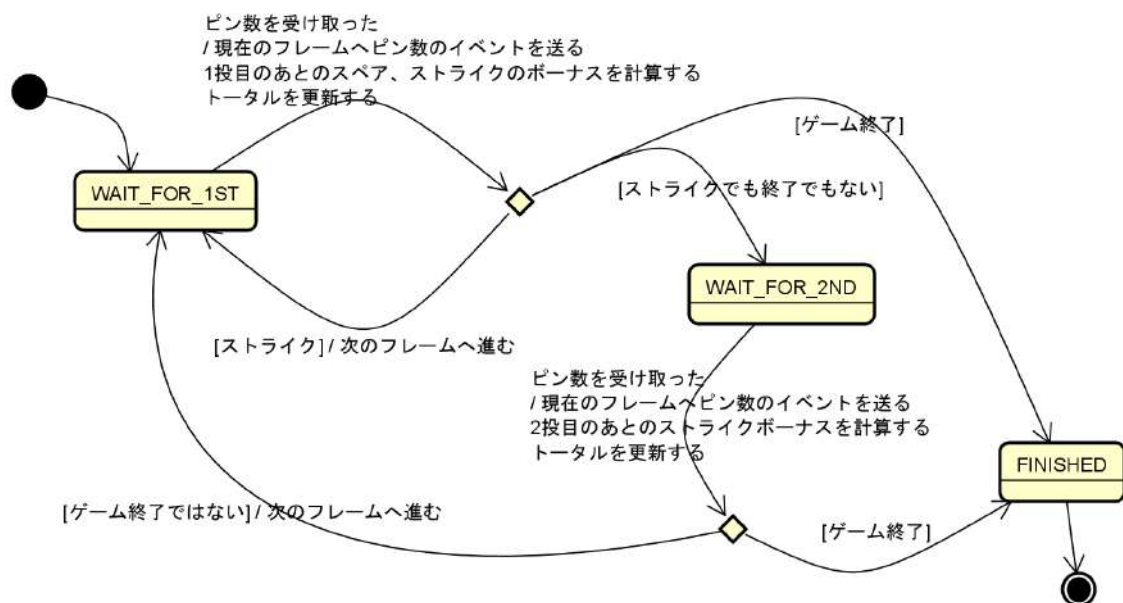


図 4.20 ステートマシン図に「Score」クラスのイベントとアクションを追加する

4.4.3 クラス図に検討結果を反映する

第10フレームの場合にもサービスフレームのために追加のフレームを用意することで、通常フレームと同じようにピン数やボーナスを扱えるようになりました。

クラス図にこの結果を反映しましょう。

ステートマシン図に合わせてクラス図を更新する（図 4.21）

1. 「Frame」クラス側の関連端の多重度を「10」から「12」に変更する。
 - これは、通常のフレームを追加する方法でサービスフレームを処理するための追加。
2. 関連にノートをつけて説明をつけておく。
 - パレットからノートを選択し、クラス図に追加する。
 - ノートに「Frame側の多重度は、サービスフレーム用に必要なフレーム分だけ追加してある」という説明を追加する。
 - ノートから関連の線に向かってアンカーを引く。
 - パレットからノートのアンカーを選択し、ノートにマウスカーソルを移動して青枠が表示されるのを待つ。
 - マウスのボタンを押したまま、マウスカーソルをドラッグし、関連の線に近づけ青枠が表示されるのを待つ。
3. 「Score」クラスにステートマシン図で作成したメソッドを追加しておく。
 - 「次のフレームへ進む」メソッド「go_next_frame」を追加する。
 - 「ゲーム終了か判定する」メソッド「finished?」を追加する。
 - 「1投目の後のスペアのボーナスを計算する」メソッド「calc_spare_bonus_after_1st」を追加する。
 - 「1投目の後のストライクのボーナスを計算する」メソッド「calc_strike_bonus_after_1st」を追加する。
 - 「2投目の後のストライクのボーナスを計算する」メソッド「calc_strike_bonus_after_2st」を追加する。
 - 「のべのトータルを更新する」メソッド「update_total」を追加する。
 - スコアを記録するメソッド（ステートマシン図の処理を担当する）メソッド「scoring」を追加する。
 - ステートマシン図の状態ごとの処理を担当するメソッド「wait_for_1st_proc」と「wait_for_2nd_proc」を追加する。
 - スコアの記録を文字列化するメソッド「to_s」を追加する。
4. 関連を追加する
 - 「Score」クラスから「Frame」クラスへ、「現在のフレーム」を指す関連「current」を追加する。

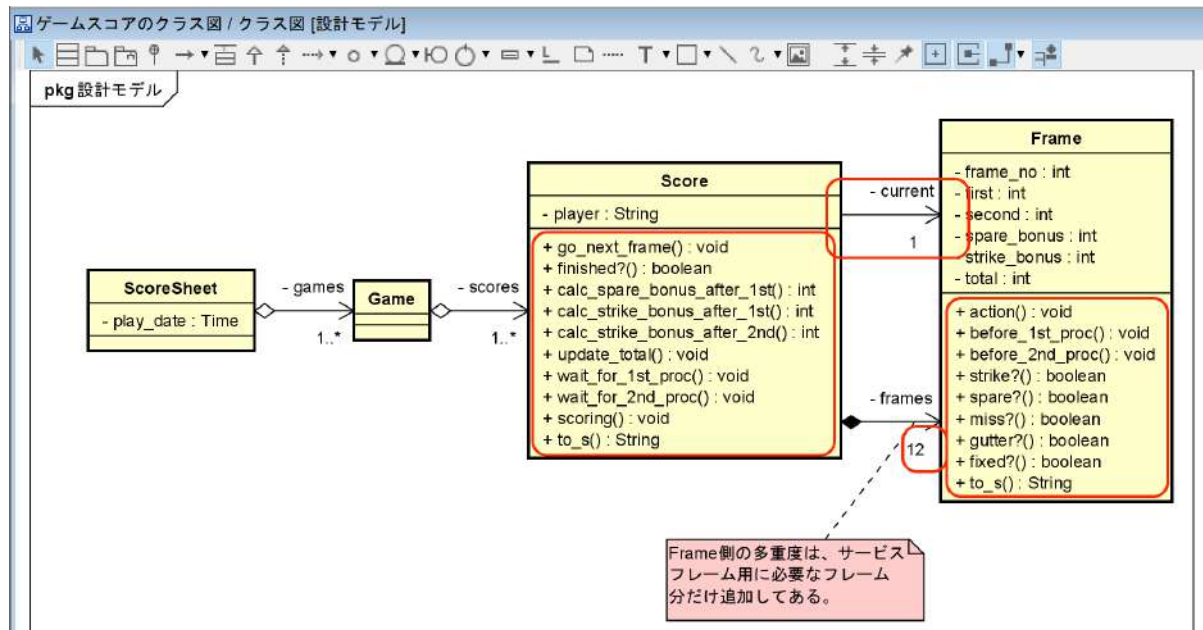


図 4.21 ステートマシン図に合わせてクラス図を更新する

TIP

ノート内の文章を編集するときは、ノートを選択した状態でプロパティを使って編集すると、改行が入力しやすくなります。

4.4.4 スコアの処理をプログラムに変換する

スコアのクラスとステートマシン図が作成できたので、C言語のプログラムに変換してみましょう。変換したプログラムの初期化やユーティリティメソッドの部分は、リスト 4.6、リスト 4.7 のようになるでしょう。

リスト 4.6 bowling.h (2)

[C]

```

63 /* Score構造体: 各人の10フレーム分のスコアを記録 */
64 typedef struct {
65     char id[ID_LEN];
66     char player[MAX_NAME_LEN];
67     int fno;
68     Frame frames[MAX_FRAMES];
69     ScoreState state;
70 } Score;
71
72 /* Score関数 */
73 void score_init(Score *score, const char *name);
74 int score_fno2idx(int fno);
75 Frame* score_frame(Score *score, int fno);
76 void score_go_next_frame(Score *score);
77 Frame* score_current(Score *score);
78 Frame* score_prev(Score *score);
79 Frame* score_pprev(Score *score);
80 void score_calc_spare_bonus_after_1st(Score *score);
81 void score_calc_strike_bonus_after_1st(Score *score);
82 void score_calc_strike_bonus_after_2nd(Score *score);
83 void score_update_total(Score *score);
84 bool score_is_finished(const Score *score);
85 void score_scoring(Score *score, int pins);
86 void score_print(const Score *score);
  
```

リスト 4.7 bowling.c (5)

[C]

```

136 void score_init(Score *score, const char *name) { ①
137     generate_id(score->id);
138     strncpy(score->player, name, MAX_NAME_LEN - 1);
139     score->player[MAX_NAME_LEN - 1] = '\0';
140     score->fno = 1;
141     score->state = WAIT_FOR_1ST; ②
142
143     /* ダミーフレーム含めて初期化 (-1, 0はダミー) */
144     for (int i = 0; i < MAX_FRAMES; i++) {
145         frame_init(&score->frames[i], i - 1);
146     }
147
148     frame_action(score_frame(score, score->fno), SETUP, 0); ③
149 }
150
151 int score_fno2idx(int fno) { ④
152     return fno + 1; /* frame number 1 => array index 2 (0 origin) */
153 }
154
155 Frame* score_frame(Score *score, int fno) {
156     return &score->frames[score_fno2idx(fno)];
157 }
158
159 void score_go_next_frame(Score *score) { ⑤
160     score->fno += 1;
161     frame_action(score_frame(score, score->fno), SETUP, 0);
162 }
163
164 Frame* score_current(Score *score) { ⑥
165     return score_frame(score, score->fno);
166 }
167
168 Frame* score_prev(Score *score) { ⑦
169     return score_frame(score, score->fno - 1);
170 }
171
172 Frame* score_pprev(Score *score) { ⑧
173     return score_frame(score, score->fno - 2);
174 }
175
176 void score_calc_spare_bonus_after_1st(Score *score) { ⑨
177     Frame *prev = score_prev(score);
178     Frame *current = score_current(score);
179
180     if (!frame_is_spare(prev)) {
181         return;
182     }
183
184     prev->spare_bonus = current->first;
185     frame_action(prev, DETERMINE, 0);
186 }
187
188 void score_calc_strike_bonus_after_1st(Score *score) { ⑩
189     Frame *prev = score_prev(score);
190     Frame *pprev = score_pprev(score);
191     Frame *current = score_current(score);
192
193     if (!(frame_is_strike(prev) && frame_is_strike(pprev))) {
194         return;
195     }
196
197     pprev->strike_bonus = prev->first + current->first;
198     frame_action(pprev, DETERMINE, 0);
199 }

```

```

200
201 void score_calc_strike_bonus_after_2nd(Score *score) { ⑪
202     Frame *prev = score_prev(score);
203     Frame *current = score_current(score);
204
205     if (!frame_is_strike(prev)) {
206         return;
207     }
208
209     prev->strike_bonus = current->first + current->second;
210     frame_action(prev, DETERMINE, 0);
211 }
212
213 void score_update_total(Score *score) { ⑫
214     for (int i = 1; i < MAX_FRAMES - 1; i++) {
215         score->frames[i + 1].total =
216             score->frames[i].total + frame_score(&score->frames[i + 1]);
217     }
218 }
219
220 bool score_is_finished(const Score *score) { ⑬
221     return frame_is_fixed(score_frame((Score*)score, 10));

```

- ① 「Frame」のインスタンスの作成。クラス図では、「Score」から「Frame」へのコンポジションとして表されている部分。設計上は12個だが、サービスフレームで「次」を参照する場面、第1フレームで「前のフレーム」「前の前のフレーム」を参照する場面で参照するダミーのフレームを追加している。
- ② スコアの最初の状態を「WAIT_FOR_1ST」にする。
- ③ 第1フレームへ「SETUP」イベントを送る。
- ④ フレーム番号とフレームの配列のインデックスを対応づける関数（ダミーのフレームを第1フレームの前に追加したためのオフセット）。
- ⑤ 現在のフレームを「次のフレームへ進める」関数。この関数では、次のフレームに「SETUP」イベントを送る。
- ⑥ 「現在のフレーム」を参照するための関数。クラス図では関連端名が「current」の「Frame」クラスへの関連で表されている。
- ⑦ ボーナス計算で使う、「前のフレーム」を得る関数。
- ⑧ ボーナス計算で使う、「前の前のフレーム」を得る関数。
- ⑨ 1投目の後のスペアボーナスを計算する関数。前のフレームのスコアが確定するので、前のフレームへ「DETERMINE」イベントを送る。
- ⑩ 1投目の後のストライクボーナスを計算する関数。前の前のフレームからストライクが続いていた場合、ここでスコアが確定するので、前の前のフレームへ「DETERMINE」イベントを送る。
- ⑪ 2投目の後のストライクボーナスを計算する関数。前のフレームのスコアが確定するので、前のフレームへ「DETERMINE」イベントを送る。
- ⑫ 「のべのトータル」を更新する関数。each_consは、配列から指定した数ずつ要素を取り出す関数。
- ⑬ 「ゲーム終了」の判定用の関数。サービスフレームの検討結果から、第10フレームが「FIXED」になれば、そのゲームは終了とみなせる。

ボーナス計算やトータル計算のメソッドはリスト 4.8 のようになるでしょう。

リスト 4.8 bowling.c (6)

[C]

```

224 static void score_wait_for_1st_proc(Score *score, int pins) { ❶
225     Frame *current = score_current(score);
226
227     frame_action(current, PINS, pins); ❷
228     score_calc_spare_bonus_after_1st(score);
229     score_calc_strike_bonus_after_1st(score);
230     score_update_total(score);
231
232     if (score_is_finished(score)) { ❸
233         score->state = FINISHED;
234     } else if (frame_is_strike(current)) { ❹
235         score->state = WAIT_FOR_1ST;
236         score_go_next_frame(score);
237     } else {
238         score->state = WAIT_FOR_2ND; ❺
239     }
240 }
241
242 static void score_wait_for_2nd_proc(Score *score, int pins) { ❻
243     Frame *current = score_current(score);
244
245     frame_action(current, PINS, pins); ❼
246     score_calc_strike_bonus_after_2nd(score);
247     score_update_total(score);
248
249     if (score_is_finished(score)) { ❽
250         score->state = FINISHED;
251     } else {
252         score->state = WAIT_FOR_1ST; ❾
253         score_go_next_frame(score);
254     }
255 }
256
257 void score_scoring(Score *score, int pins) { ❿
258     switch (score->state) {
259         case WAIT_FOR_1ST:
260             score_wait_for_1st_proc(score, pins);
261             break;
262         case WAIT_FOR_2ND:
263             score_wait_for_2nd_proc(score, pins);
264             break;
265         case FINISHED:
266             printf("finished\n");
267             break;
268     }
269 }
270
271 void score_print(const Score *score) { ⓫
272     printf("Player:%s, Score(id: %s), Frame:%d\n",
273           score->player, score->id, score->fno);
274     printf("|No|1st|2nd|Total|Frame Score|Spare Bonus|Strike Bonus|Frame State|\n");
275     for (int i = 0; i < MAX_FRAMES; i++) {
276         frame_print(&score->frames[i]);
277     }
278 }

```

- ❶ 「WAIT_FOR_1ST」状態のときの状態遷移のメソッド。
- ❷ 現在のフレームについて、ピン数を更新し、ボーナスを計算し、のべのトータルを更新する。
- ❸ 「選択疑似状態」での遷移先の分岐処理。「ゲーム終了」の場合は「FINISHED」へ遷移する。
- ❹ 「ストライクだった」の場合は、次のフレームへ進んで「WAIT_FOR_1ST」へ遷移する。
- ❺ それ以外のときは2投目を待つので、「WAIT_FOR_2ND」へ遷移する。
- ❻ 「WAIT_FOR_2ND」状態のときの状態遷移のメソッド。

- ⑦ 現在のフレームについて、ピン数を更新し、ボーナスを計算し、のべのトータルを更新する。
- ⑧ 「選択疑似状態」での遷移先の分岐処理。「ゲーム終了」の場合は「FINISHED」へ遷移する。
- ⑨ それ以外のときは、次のフレームへ進んで「WAIT_FOR_1ST」へ遷移する。
- ⑩ スコアを記録するステートマシン図の振る舞いを担当するメソッド。状態に応じてそれぞれの状態用のメソッドを呼び出す。
- ⑪ 「スコア」クラスのインスタンスの内容（内包するフレームも含む）を文字列化するメソッド。

これで、フレームとスコアを、それぞれのステートマシン図で表した振る舞いに合わせて動作するプログラムに変換できました。

4.5 ゲームの進行について検討する

残るは、複数名のスコアのセットで構成される「Game」と、複数の「Game」を記録する「ScoreSheet」クラスです。

4.5.1 ヨーロピアン方式とアメリカン方式

ボウリングを複数名で楽しむとき、みなさんはたいいてい、1組のチームで1つのレーンを使って、1フレームごとにプレイヤーが交代しながらプレイします。このようなゲームの方式は、「ヨーロピアン方式」と呼ばれています。

別の方式として、ボールラック（ボールが返ってくるラック）を挟んだ2つのレーンを、1フレーム交代で使ってプレイする方式があります。このようなゲームの方式は「アメリカン方式」と呼ばれています。

このチュートリアルでは、ヨーロピアン方式を使うことにします。

4.5.2 Gameクラスの処理

ほとんどのみなさんがボウリングをやるときにゲームを進行する手順は、「ヨーロピアン方式」と呼ばれています。「ヨーロピアン方式」による進行を想定して、「Game」クラスはどのような手順で動作させるべきか整理しましょう。

ボウリングのゲームを進める手順（ヨーロピアン方式）

1. スコアシートに参加するプレイヤー名を書く（エントリーする）。
2. 書いたプレイヤーの順に1フレーム分プレイする（各自のターン）。
3. 次のプレイヤーと交代する（次のターンへ進む）。
4. 全員がゲーム終了するまで手順を繰り返す。

この方式に合わせてスコアを記録する処理をする「Game」クラスは、リスト 4.9、リスト 4.10 のようになるでしょう。

リスト 4.9 bowling.h (3)

[C]

```

88 /* Game構造体: 複数名の1ゲーム分のスコアのセット */
89 typedef struct {
90     char id[ID_LEN];
91     int turn;
92     Score scores[MAX_PLAYERS];
93     int player_count;
94 } Game;
95
96 /* Game関数 */
97 void game_init(Game *game);
98 void game_entry(Game *game, const char *name);
99 const char* game_turn_player_name(const Game *game);
100 void game_go_next_turn(Game *game);
101 void game_playing(Game *game, int score_index, int pins);
102 bool game_is_finished(const Game *game);
103 void game_print(const Game *game);

```

リスト 4.10 bowling.c (7)

[C]

```

284 void game_init(Game *game) {
285     generate_id(game->id); ①
286     game->turn = 0;
287     game->player_count = 0;
288 }
289
290 void game_entry(Game *game, const char *name) { ②
291     if (game->player_count >= MAX_PLAYERS) {
292         printf("Maximum number of players reached.\n");
293         return;
294     }
295     score_init(&game->scores[game->player_count], name);
296     game->player_count++;
297 }
298
299 const char* game_turn_player_name(const Game *game) { ③
300     return game->scores[game->turn].player;
301 }
302
303 void game_go_next_turn(Game *game) { ④
304     game->turn = (game->turn + 1) % game->player_count;
305 }
306
307 void game_playing(Game *game, int score_index, int pins) { ⑤
308     Score *score = &game->scores[score_index];
309     score_scoring(score, pins);
310
311     if (score->fno > 10) {
312         if (score_is_finished(score)) { ⑥
313             game_go_next_turn(game);
314         }
315     } else if (score_current(score)->state == BEFORE_1ST) { ⑦
316         game_go_next_turn(game);
317     }
318 }
319
320 bool game_is_finished(const Game *game) { ⑧
321     for (int i = 0; i < game->player_count; i++) {
322         if (!score_is_finished(&game->scores[i])) {
323             return false;
324         }
325     }
326     return true;
327 }

```

```

328
329 void game_print(const Game *game) { ⑨
330     printf("Game(id:%s)\n", game->id);
331     for (int i = 0; i < game->player_count; i++) {
332         score_print(&game->scores[i]);
333     }
334 }

```

- ① ゲームごとにユニークなIDをつけておく。
- ② ゲームに参加するプレイヤーを登録する関数。そのプレイヤーの今回のゲーム分のスコアも用意する。
- ③ 現在プレー中のプレイヤー名を取得する関数。
- ④ 次のプレイヤーに交代する関数。登録したプレイヤーの順に進み、最後まで来たら最初のプレイヤーへ戻る。
- ⑤ ゲームを実行する関数。現在のプレイヤーのスコアクラスの `scoring` 関数を呼び出して1フレーム分のプレーを記録する。
- ⑥ サービスフレームでは、2フレーム以上プレーする場合があるので、そのときは交代しない。
- ⑦ 現在のプレイヤーの現在の状態が「BEFORE_1ST」なら、次のフレームの投球待ちになっているので、プレイヤーを交代する。
- ⑧ すべてのプレイヤーがゲーム終了かどうか調べる関数。
- ⑨ ゲームの状況を文字列化する関数。

クラス図にもこの結果を反映しておきましょう（図 4.22）。現在のプレイヤーは、関連端名が「turn」の関連によって参照しているScoreを使っています。

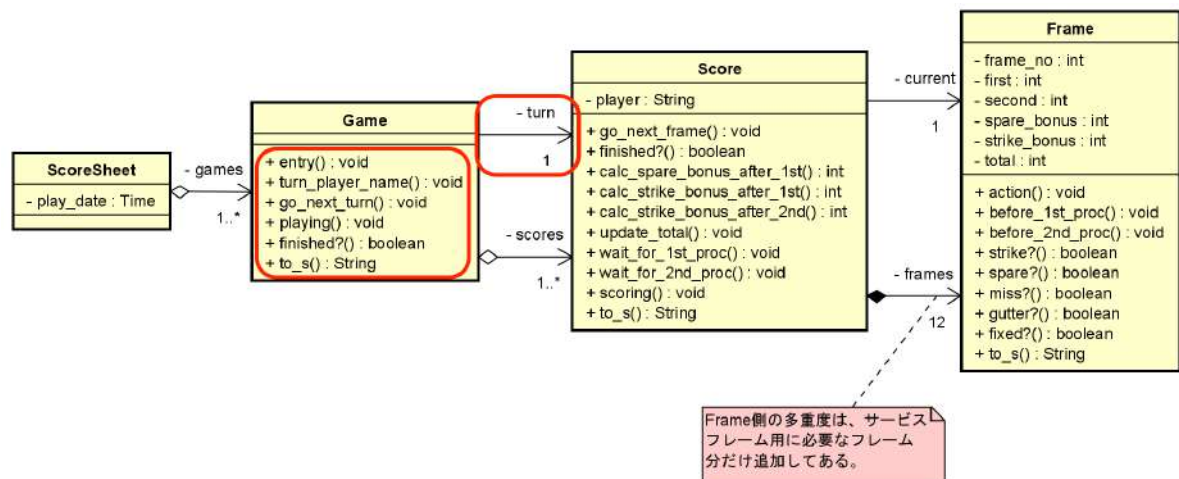


図 4.22 「Game」クラスを更新したクラス図

4.5.3 ScoreSheetクラスの処理

「Game」クラスが作成できたので、スコアシートを扱う「ScoreSheet」クラスも作成できそうですね。新しいスコアシートを作成して、そこに必要な数のゲームを追加すれば済みそうです。

この方式に合わせてスコアを記録する処理をする「ScoreSheet」クラスは、リスト 4.11、リスト 4.12 のようになるでしょう。

リスト 4.11 bowling.h (4)

[C]

```

113 /* ScoreSheet関数 */
114 void scoresheet_init(ScoreSheet *sheet, time_t date);
115 void scoresheet_add_game(ScoreSheet *sheet, int games);
116 void scoresheet_print(const ScoreSheet *sheet);
117
118 /* ユーティリティ関数 */
119 void generate_id(char *id);
120 const char* frame_state_to_string(FrameState state);
121
122 #endif /* BOWLING_H */

```

リスト 4.12 bowling.c (8)

[C]

```

340 void scoresheet_init(ScoreSheet *sheet, time_t date) { ①
341     generate_id(sheet->id); ②
342     sheet->play_date = date;
343     sheet->game_count = 0;
344 }
345
346 void scoresheet_add_game(ScoreSheet *sheet, int games) { ③
347     for (int i = 0; i < games && sheet->game_count < MAX_GAMES; i++) {
348         game_init(&sheet->games[sheet->game_count]);
349         sheet->game_count++;
350     }
351 }
352
353 void scoresheet_print(const ScoreSheet *sheet) { ④
354     char date_str[64];
355     struct tm *tm_info = localtime(&sheet->play_date);
356     strftime(date_str, sizeof(date_str), "%Y-%m-%d %H:%M:%S", tm_info);
357     printf("Score Sheet Date: %s (id:%s)\n", date_str, sheet->id);
358 }

```

- ① スコアシートを作成するときは、作成時の日時を控えておく。
- ② スコアシートごとにユニークなIDをつけておく。
- ③ 希望する数のゲームをシートに追加するメソッド。引数がないときは1組だけ追加する。
- ④ スコアシートの状況を文字列化するメソッド。

クラス図にもこの結果を反映しておきましょう（図 4.23）。現在のプレイヤーは、関連端名が「turn」の関連によって参照しているScoreを使っています。

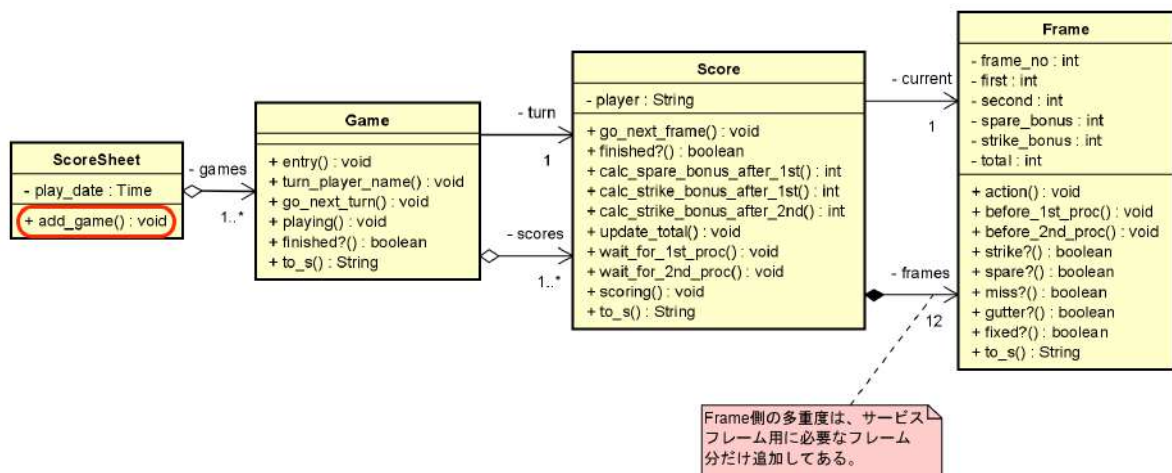


図 4.23 「ScoreSheet」クラスを更新したクラス図

4.6 まとめ

ボウリングのゲームスコアを記録するスコアシートの振る舞いを検討しました。

4.6.1 振る舞いのモデルを作成した手順

振る舞いのモデルに登場する構成要素（状態、イベント、アクション）を洗い出し、ゲームの手順を整理しました。

スコアシートの振る舞いのモデルを作成した手順

1. スコアに関する状態の発見
 - スコアが記録されときのフレームの変化を観察して、フレームにどのような状態があるか洗い出した。
2. スコアに関するステートマシン図の作成
 - 洗い出したフレームの状態に、関連するイベントやアクションを考えてステートマシン図に表した。
3. フレームスコアに関する状態の発見
 - ゲームを進めるときのスコアの変化を観察して、スコアにどのような状態があるか洗い出した。
4. フレームに関するステートマシン図の作成
 - サービスフレームの動作を観察して、通常のフレームを追加してスコアを記録する方法を発見した。
 - 洗い出したスコアの状態に関連するイベントやアクションを考えてステートマシン図に表した。

4.6.2 モデルとコードの対応づけは振る舞い設計の前に

ステートマシン図を使うことで、フレームやスコアの動作を表せました。そして、あらかじめモデルとコードを対応づけておけば、それを前提として（モデルやコードの構成方式として）振る舞いの設計に活かせます。

モデルとコードが対応づけられていれば、C言語のプログラムはステートマシン図から変換するように作成できます。つまり、解決すべき課題があったとき、その課題を構造のモデルと振る舞いのモデルで表すことができれば、そのモデルなりのプログラムが作成できるわけです。一方、このような方法で作成したプログラムが期待した動作をしない場合には、モデルが誤っているか、対応づけのルールに不備があるということです。

【コラム 6】 モデル変換とモデル駆動開発（MDD）【参考】

ここで示したようなモデルとコードの対応づけのほかに、モデルから別のモデルへ、あるいはコードから別のコードへといった対応づけも考えられます。モデルからコードを生成する

方法には、このチュートリアルのように図の要素とコードの要素を対応させるルールを決めて手で変換する方法以外にもあります。例えば、コード片のテンプレート（スニペットなどと呼ばれます）にモデルのデータベースへの問い合わせを埋め込んだスクリプトを使って生成する方法はよく使われています

これらは、ソフトウェア設計における「モデル変換」と呼ばれています（コードも一種のモデル表現とみなせます）。そして、モデル変換を用いて開発プロセスにおける各工程間をモデルで接続して開発する方法のことを「モデル駆動開発（MDD: Model Driven Development）」と呼びます。

モデル駆動開発を紹介している記事として次の記事があります。

モデル駆動開発におけるモデル変換の役割

<https://codezine.jp/article/detail/10597>

この記事では、モデル変換の繰り返しによる開発方法であることや、その実施例を紹介しています。

第5章 できあがったプログラムを試す

対応づけルールを使って、モデルから変換して作成したプログラムができあがりました。できあがったプログラムを確認し、実際に動かしてみましょう。

5.1 プログラムを完成させる

まず、プログラムをテストできるよう、完成させましょう。

5.1.1 作成したプログラムを確認する

作成したプログラムの全体を リスト 5.1、 リスト 5.2 に示します。みなさんも作成したプログラムの全体を確認しましょう。

リスト 5.1 bowling.h

[C]

```
1 /* bowling.h - ボーリングスコア管理システム */
2 #ifndef BOWLING_H
3 #define BOWLING_H
4
5 #include <stdio.h>
6 #include <stdlib.h>
7 #include <string.h>
8 #include <stdbool.h>
9 #include <time.h>
10
11 /* 定数定義 */
12 #define MAX_FRAMES 15          /* ダミーフレーム含む配列サイズ */
13 #define MAX_PLAYERS 10        /* 最大プレイヤー数 */
14 #define MAX_GAMES 20          /* 最大ゲーム数 */
15 #define MAX_NAME_LEN 64       /* プレイヤー名の最大長 */
16 #define ID_LEN 12             /* ID文字列の長さ */
17
18 /* フレームの状態 */
19 typedef enum {
20     RESERVED,
21     BEFORE_1ST,
22     BEFORE_2ND,
23     PENDING,
24     FIXED
25 } FrameState;
26
27 /* イベントタイプ */
28 typedef enum {
29     SETUP,
30     PINS,
31     DETERMINE
32 } EventType;
33
34 /* スコアの状態 */
35 typedef enum {
36     WAIT_FOR_1ST,
37     WAIT_FOR_2ND,
38     FINISHED
39 } ScoreState;
40
41 /* Frame構造体: 1フレーム分のピン数やボーナスを記録 */
42 typedef struct {
```

```

43     int frame_no;
44     int first;
45     int second;
46     int spare_bonus;
47     int strike_bonus;
48     int total;
49     FrameState state;
50 } Frame;
51
52 /* Frame関数 */
53 void frame_init(Frame *frame, int frame_no);
54 void frame_action(Frame *frame, EventType event, int pins);
55 int frame_score(const Frame *frame);
56 bool frame_is_strike(const Frame *frame);
57 bool frame_is_spare(const Frame *frame);
58 bool frame_is_miss(const Frame *frame);
59 bool frame_is_gutter(const Frame *frame);
60 bool frame_is_fixed(const Frame *frame);
61 void frame_print(const Frame *frame);
62
63 /* Score構造体: 各人の10フレーム分のスコアを記録 */
64 typedef struct {
65     char id[ID_LEN];
66     char player[MAX_NAME_LEN];
67     int fno;
68     Frame frames[MAX_FRAMES];
69     ScoreState state;
70 } Score;
71
72 /* Score関数 */
73 void score_init(Score *score, const char *name);
74 int score_fno2idx(int fno);
75 Frame* score_frame(Score *score, int fno);
76 void score_go_next_frame(Score *score);
77 Frame* score_current(Score *score);
78 Frame* score_prev(Score *score);
79 Frame* score_pprev(Score *score);
80 void score_calc_spare_bonus_after_1st(Score *score);
81 void score_calc_strike_bonus_after_1st(Score *score);
82 void score_calc_strike_bonus_after_2nd(Score *score);
83 void score_update_total(Score *score);
84 bool score_is_finished(const Score *score);
85 void score_scoring(Score *score, int pins);
86 void score_print(const Score *score);
87
88 /* Game構造体: 複数名の1ゲーム分のスコアのセット */
89 typedef struct {
90     char id[ID_LEN];
91     int turn;
92     Score scores[MAX_PLAYERS];
93     int player_count;
94 } Game;
95
96 /* Game関数 */
97 void game_init(Game *game);
98 void game_entry(Game *game, const char *name);
99 const char* game_turn_player_name(const Game *game);
100 void game_go_next_turn(Game *game);
101 void game_playing(Game *game, int score_index, int pins);
102 bool game_is_finished(const Game *game);
103 void game_print(const Game *game);
104
105 /* ScoreSheet構造体: 複数名の複数回のGameを記録 */
106 typedef struct {
107     char id[ID_LEN];
108     time_t play_date;

```



```

109     Game games[MAX_GAMES];
110     int game_count;
111 } ScoreSheet;
112
113 /* ScoreSheet関数 */
114 void scoresheet_init(ScoreSheet *sheet, time_t date);
115 void scoresheet_add_game(ScoreSheet *sheet, int games);
116 void scoresheet_print(const ScoreSheet *sheet);
117
118 /* ユーティリティ関数 */
119 void generate_id(char *id);
120 const char* frame_state_to_string(FrameState state);
121
122 #endif /* BOWLING_H */

```

リスト 5.2 bowling.c

[C]

```

1 /* bowling.c - ボーリングスコア管理システムの実装 */
2 #include "bowling.h"
3
4 void generate_id(char *id) { ①
5     static int counter = 0;
6     snprintf(id, ID_LEN, "ID%08d", counter++);
7 }
8
9 const char* frame_state_to_string(FrameState state) { ②
10     switch (state) {
11         case RESERVED: return "RESERVED";
12         case BEFORE_1ST: return "BEFORE_1ST";
13         case BEFORE_2ND: return "BEFORE_2ND";
14         case PENDING: return "PENDING";
15         case FIXED: return "FIXED";
16         default: return "UNKNOWN";
17     }
18 }
19
20 /* =====
21 * Frame実装
22 * ===== */
23
24 void frame_init(Frame *frame, int frame_no) { ③
25     frame->frame_no = frame_no;
26     frame->first = 0;
27     frame->second = 0;
28     frame->spare_bonus = 0;
29     frame->strike_bonus = 0;
30     frame->total = 0;
31     frame->state = RESERVED; ④
32 }
33
34 static void frame_before_1st_proc(Frame *frame, EventType event, int pins) {
35     if (event == PINS) {
36         if (pins < 0 || pins > 10) {
37             printf("invalid pins: %d\n", pins);
38             return;
39         }
40         frame->first = pins;
41         if (frame_is_strike(frame)) {
42             frame->second = 0;
43             frame->state = PENDING;
44         } else {
45             frame->state = BEFORE_2ND;
46         }
47     } else {
48         printf("invalid event on %s.\n", frame_state_to_string(frame->state));
49     }
50 }
51
52 static void frame_before_2nd_proc(Frame *frame, EventType event, int pins) {

```

```

53     if (event == PINS) {
54         if (pins < 0 || pins > (10 - frame->first)) {
55             printf("invalid pins: %d\n", pins);
56             return;
57         }
58         frame->second = pins;
59         if (frame_is_spare(frame)) {
60             frame->state = PENDING;
61         } else {
62             frame->state = FIXED;
63         }
64     } else {
65         printf("invalid event on %s.\n", frame_state_to_string(frame->state));
66     }
67 }
68
69 void frame_action(Frame *frame, EventType event, int pins) {
70     switch (frame->state) { ①
71     case RESERVED: ②
72         if (event == SETUP) {
73             frame->state = BEFORE_1ST;
74         } else {
75             printf("invalid event is ignored.\n");
76         }
77         break;
78     case BEFORE_1ST: ③
79         frame_before_1st_proc(frame, event, pins);
80         break;
81     case BEFORE_2ND: ④
82         frame_before_2nd_proc(frame, event, pins);
83         break;
84     case PENDING: ⑤
85         if (event == DETERMINE) {
86             frame->state = FIXED;
87         }
88         break;
89     case FIXED:
90         printf("fixed.\n");
91         break;
92     }
93 }
94
95 int frame_score(const Frame *frame) {
96     return frame->first + frame->second + frame->spare_bonus + frame->strike_bonus;
97 }
98
99 bool frame_is_strike(const Frame *frame) {
100     return frame->first == 10;
101 }
102
103 bool frame_is_spare(const Frame *frame) {
104     return frame->first < 10 && (frame->first + frame->second) == 10;
105 }
106
107 bool frame_is_miss(const Frame *frame) {
108     return frame->first < 10 && frame->second == 0 && frame->state == FIXED;
109 }
110
111 bool frame_is_gutter(const Frame *frame) {
112     return frame->first == 0;
113 }
114
115 bool frame_is_fixed(const Frame *frame) {
116     return frame->state == FIXED;
117 }
118
119 void frame_print(const Frame *frame) {
120     char total_str[8];
121     if (frame->state == FIXED) {
122         snprintf(total_str, sizeof(total_str), "%5d", frame->total);

```

```

123     } else {
124         snprintf(total_str, sizeof(total_str), "  .");
125     }
126     printf("%2d|%3d|%3d|%5s|%11d|%11d|%12d|%11s|\n",
127         frame->frame_no, frame->first, frame->second, total_str,
128         frame_score(frame), frame->spare_bonus, frame->strike_bonus,
129         frame_state_to_string(frame->state));
130 }
131
132 /* =====
133  * Score実装
134  * ===== */
135
136 void score_init(Score *score, const char *name) { ①
137     generate_id(score->id);
138     strncpy(score->player, name, MAX_NAME_LEN - 1);
139     score->player[MAX_NAME_LEN - 1] = '\0';
140     score->fno = 1;
141     score->state = WAIT_FOR_1ST; ②
142
143     /* ダミーフレーム含めて初期化 (-1, 0はダミー) */
144     for (int i = 0; i < MAX_FRAMES; i++) {
145         frame_init(&score->frames[i], i - 1);
146     }
147
148     frame_action(score_frame(score, score->fno), SETUP, 0); ③
149 }
150
151 int score_fno2idx(int fno) { ④
152     return fno + 1; /* frame number 1 => array index 2 (0 origin) */
153 }
154
155 Frame* score_frame(Score *score, int fno) {
156     return &score->frames[score_fno2idx(fno)];
157 }
158
159 void score_go_next_frame(Score *score) { ⑤
160     score->fno += 1;
161     frame_action(score_frame(score, score->fno), SETUP, 0);
162 }
163
164 Frame* score_current(Score *score) { ⑥
165     return score_frame(score, score->fno);
166 }
167
168 Frame* score_prev(Score *score) { ⑦
169     return score_frame(score, score->fno - 1);
170 }
171
172 Frame* score_pprev(Score *score) { ⑧
173     return score_frame(score, score->fno - 2);
174 }
175
176 void score_calc_spare_bonus_after_1st(Score *score) { ⑨
177     Frame *prev = score_prev(score);
178     Frame *current = score_current(score);
179
180     if (!frame_is_spare(prev)) {
181         return;
182     }
183
184     prev->spare_bonus = current->first;
185     frame_action(prev, DETERMINE, 0);
186 }
187
188 void score_calc_strike_bonus_after_1st(Score *score) { ⑩
189     Frame *prev = score_prev(score);
190     Frame *pprev = score_pprev(score);
191     Frame *current = score_current(score);
192

```

```

193     if (!(frame_is_strike(prev) && frame_is_strike(pprev))) {
194         return;
195     }
196
197     pprev->strike_bonus = prev->first + current->first;
198     frame_action(pprev, DETERMINE, 0);
199 }
200
201 void score_calc_strike_bonus_after_2nd(Score *score) { ⑪
202     Frame *prev = score_prev(score);
203     Frame *current = score_current(score);
204
205     if (!frame_is_strike(prev)) {
206         return;
207     }
208
209     prev->strike_bonus = current->first + current->second;
210     frame_action(prev, DETERMINE, 0);
211 }
212
213 void score_update_total(Score *score) { ⑫
214     for (int i = 1; i < MAX_FRAMES - 1; i++) {
215         score->frames[i + 1].total =
216             score->frames[i].total + frame_score(&score->frames[i + 1]);
217     }
218 }
219
220 bool score_is_finished(const Score *score) { ⑬
221     return frame_is_fixed(score_frame((Score*)score, 10));
222 }
223
224 static void score_wait_for_1st_proc(Score *score, int pins) { ①
225     Frame *current = score_current(score);
226
227     frame_action(current, PINS, pins); ②
228     score_calc_spare_bonus_after_1st(score);
229     score_calc_strike_bonus_after_1st(score);
230     score_update_total(score);
231
232     if (score_is_finished(score)) { ③
233         score->state = FINISHED;
234     } else if (frame_is_strike(current)) { ④
235         score->state = WAIT_FOR_1ST;
236         score_go_next_frame(score);
237     } else {
238         score->state = WAIT_FOR_2ND; ⑤
239     }
240 }
241
242 static void score_wait_for_2nd_proc(Score *score, int pins) { ⑥
243     Frame *current = score_current(score);
244
245     frame_action(current, PINS, pins); ⑦
246     score_calc_strike_bonus_after_2nd(score);
247     score_update_total(score);
248
249     if (score_is_finished(score)) { ⑧
250         score->state = FINISHED;
251     } else {
252         score->state = WAIT_FOR_1ST; ⑨
253         score_go_next_frame(score);
254     }
255 }
256
257 void score_scoring(Score *score, int pins) { ⑩
258     switch (score->state) {
259         case WAIT_FOR_1ST:
260             score_wait_for_1st_proc(score, pins);
261             break;
262         case WAIT_FOR_2ND:

```

```

263         score_wait_for_2nd_proc(score, pins);
264         break;
265     case FINISHED:
266         printf("finished\n");
267         break;
268     }
269 }
270
271 void score_print(const Score *score) { ⑪
272     printf("Player:%s, Score(id: %s), Frame:%d\n",
273           score->player, score->id, score->fno);
274     printf("|No|1st|2nd|Total|Frame Score|Spare Bonus|Strike Bonus|Frame State|\n");
275     for (int i = 0; i < MAX_FRAMES; i++) {
276         frame_print(&score->frames[i]);
277     }
278 }
279
280 /* =====
281  * Game実装
282  * ===== */
283
284 void game_init(Game *game) {
285     generate_id(game->id); ①
286     game->turn = 0;
287     game->player_count = 0;
288 }
289
290 void game_entry(Game *game, const char *name) { ②
291     if (game->player_count >= MAX_PLAYERS) {
292         printf("Maximum number of players reached.\n");
293         return;
294     }
295     score_init(&game->scores[game->player_count], name);
296     game->player_count++;
297 }
298
299 const char* game_turn_player_name(const Game *game) { ③
300     return game->scores[game->turn].player;
301 }
302
303 void game_go_next_turn(Game *game) { ④
304     game->turn = (game->turn + 1) % game->player_count;
305 }
306
307 void game_playing(Game *game, int score_index, int pins) { ⑤
308     Score *score = &game->scores[score_index];
309     score_scoring(score, pins);
310
311     if (score->fno > 10) {
312         if (score_is_finished(score)) { ⑥
313             game_go_next_turn(game);
314         }
315     } else if (score_current(score)->state == BEFORE_1ST) { ⑦
316         game_go_next_turn(game);
317     }
318 }
319
320 bool game_is_finished(const Game *game) { ⑧
321     for (int i = 0; i < game->player_count; i++) {
322         if (!score_is_finished(&game->scores[i])) {
323             return false;
324         }
325     }
326     return true;
327 }
328
329 void game_print(const Game *game) { ⑨
330     printf("Game(id:%s)\n", game->id);
331     for (int i = 0; i < game->player_count; i++) {
332         score_print(&game->scores[i]);

```

```

333     }
334 }
335
336 /* =====
337  * ScoreSheet実装
338  * ===== */
339
340 void scoresheet_init(ScoreSheet *sheet, time_t date) { ①
341     generate_id(sheet->id); ②
342     sheet->play_date = date;
343     sheet->game_count = 0;
344 }
345
346 void scoresheet_add_game(ScoreSheet *sheet, int games) { ③
347     for (int i = 0; i < games && sheet->game_count < MAX_GAMES; i++) {
348         game_init(&sheet->games[sheet->game_count]);
349         sheet->game_count++;
350     }
351 }
352
353 void scoresheet_print(const ScoreSheet *sheet) { ④
354     char date_str[64];
355     struct tm *tm_info = localtime(&sheet->play_date);
356     strftime(date_str, sizeof(date_str), "%Y-%m-%d %H:%M:%S", tm_info);
357     printf("Score Sheet Date: %s (id:%s)\n", date_str, sheet->id);
358 }

```

リスト 5.3 Makefile

[MAKEFILE]

```

1 # Makefile for Bowling Score System
2
3 CC = gcc
4 CFLAGS = -Wall -Wextra -std=c99 -pedantic -O2
5 TARGET = bowling
6 OBJS = main.o bowling.o
7
8 .PHONY: all clean run
9
10 all: $(TARGET)
11
12 $(TARGET): $(OBJS)
13     $(CC) $(CFLAGS) -o $@ $^
14
15 main.o: main.c bowling.h
16     $(CC) $(CFLAGS) -c main.c
17
18 bowling.o: bowling.c bowling.h
19     $(CC) $(CFLAGS) -c bowling.c
20
21 clean:
22     rm -f $(TARGET) $(OBJS)
23
24 run: $(TARGET)
25     ./$(TARGET)

```

5.1.2 テスト用のプログラムを追加する

作成したプログラムの末尾に、テストするためのプログラムを追加します（リスト 5.4）。実際にピン数を与えてスコアを作成しているのは「game_playing」の呼び出しです。このテストコードでは、ピン数のデータは、1ゲーム分のピン数をまとめています。しかし、スコアシートのプログラム本体はピン数を受け取るたびに動作するように作られているので、ピン数を1投球ずつ与える方法でも動かします。

リスト 5.4 main.c

[C]

```
1 /* main.c - ボーリングスコアシステムのメインプログラム */
2 #include "bowling.h"
3
4 int main(void) {
5     ScoreSheet sheet;
6     scoresheet_init(&sheet, time(NULL)); ①
7
8     scoresheet_add_game(&sheet, 1); ②
9     Game *game = &sheet.games[sheet.game_count - 1]; ③
10
11     game_entry(game, "くぼあき"); ④
12     game_entry(game, "うえはら");
13
14     scoresheet_print(&sheet); ⑤
15     printf("\n");
16
17     int game_records[2][30] = { ⑥
18         // {7, 0, 5, 5, 10, 10, 5, 4, 10, 7, 3, 5, 4, 7, 3, 7, 3, 4},
19         // {6, 3, 9, 0, 0, 3, 8, 2, 7, 3, 10, 9, 1, 8, 0, 10, 6, 3},
20         {7, 0, 5, 5, 10, 10, 5, 4, 10, 7, 3, 5, 4, 7, 3, 7, 3, 4},
21         {10, 10, 10, 10, 10, 10, 10, 10, 10, 10, 10, 10, 10, 10, 10},
22     };
23     int record_index[2] = {0, 0}; ⑦
24
25     while (!game_is_finished(game)) { ⑧
26         printf("-----\n");
27         printf("turn: %s\n", game_turn_player_name(game));
28
29         int score_index = game->turn; ⑨
30         int pins = game_records[score_index][record_index[score_index]]; ⑩
31         record_index[score_index]++;
32
33         printf("input pins: %d\n", pins);
34         game_playing(game, score_index, pins); ⑪
35
36         game_print(game); ⑫
37
38         printf("Press Enter to continue..."); ⑬
39         getchar();
40     }
41
42     printf("Game(id:%s) is finished.\n", game->id);
43     return 0;
44 }
```

- ① 新しいスコアシートを作成する。
- ② 作成したスコアシートにゲームを追加する。
- ③ 追加したゲームを選ぶ（ここでは最後のゲームを選ぶ方法をとった）。
- ④ ゲームプレイヤーがエントリーする。このプレイヤーのこのゲーム用のスコアが用意される。

- ⑤ スコアシートの作成日付とIDの表示。
- ⑥ テスト用データ。1行が1人の1ゲーム分のピン数のデータ。
- ⑦ 各プレイヤーの現在の投球インデックス。
- ⑧ ゲームの終了まで繰り返し。
- ⑨ turnは、そのゲーム内の何番目のプレイヤーか（ゲーム中の何番目のスコアか）を表している。
このテストコードでは2人がエントリしているので、0→1→0→1 と繰り返す。
- ⑩ 現在のturnのプレイヤーのピン数のテストデータを先頭から1つずつ取り出す。
- ⑪ 現在のturnのプレイヤーのスコアにピン数を渡して、スコアへ反映する。
- ⑫ 現在のゲームの全体像を取得して表示する。
- ⑬ ここで空の改行キーを入力してテストを進める。テストを1回ずつ止めて実行できる。この入力待ちをやめれば一括して実行できる。

5.2 プログラムをテストする

テスト用のプログラムも作成できたので、実行してみましょう。

コマンドプロンプトを起動して（MacやLinuxならターミナルを起動して）、プログラムをビルドします（リスト 5.5）。

リスト 5.5 bowling プログラムをビルドする

[CONSOLE]

```
C:\Users\（演習用ディレクトリ）>cd score
C:\Users\（演習用ディレクトリ）>\score> make
```

ビルドできたら、実行しましょう（リスト 5.6）。

出力フォーマットは、左から「フレーム番号」「1投目のピン数」「2投目のピン数」「のべのトータル」「フレームのトータル」「スペアボーナス」「ストライクボーナス」「フレームの状態」です。

フレームの状態の変化を追ってみて、フレームのステートマシン図と対応していることを確認してみましょう。

リスト 5.6 bowling プログラムを実行する

[CONSOLE]

```
C:\Users\（演習用ディレクトリ）>\score> ./bowling
Score Sheet Date: 2022-02-26 02:08:21 +0900(id:0GG70gNRWtk) ①
-----
turn: くぼあき ②
input pins: 7 ③
Game(id:zwBxlorJlog), ④
Player:<くぼあき, Score(id: q5rZ-vnFM1o), Frame:1, ⑤
|No|1st|2nd|Total|Frame Score|Spare Bonus|Strike Bonus|Frame State|
|-1| 0| 0| .| 0| 0| 0| RESERVED| ⑥
| 0| 0| 0| .| 0| 0| 0| RESERVED|
| 1| 7| 0| .| 7| 0| 0| BEFORE_2ND| ⑦
| 2| 0| 0| .| 0| 0| 0| RESERVED|
| 3| 0| 0| .| 0| 0| 0| RESERVED|
| 4| 0| 0| .| 0| 0| 0| RESERVED|
| 5| 0| 0| .| 0| 0| 0| RESERVED|
```

6	0	0	.		0	0	0	RESERVED
7	0	0	.		0	0	0	RESERVED
8	0	0	.		0	0	0	RESERVED
9	0	0	.		0	0	0	RESERVED
10	0	0	.		0	0	0	RESERVED
11	0	0	.		0	0	0	RESERVED
12	0	0	.		0	0	0	RESERVED
13	0	0	.		0	0	0	RESERVED

Player:うえはら, Score(id: plq9pFD6nSA), Frame:1,

No	1st	2nd	Total	Frame	Score	Spare	Bonus	Strike	Bonus	Frame	State
-1	0	0	.		0		0		0		RESERVED
0	0	0	.		0		0		0		RESERVED
1	0	0	.		0		0		0		BEFORE_1ST
2	0	0	.		0		0		0		RESERVED
3	0	0	.		0		0		0		RESERVED
4	0	0	.		0		0		0		RESERVED
5	0	0	.		0		0		0		RESERVED
6	0	0	.		0		0		0		RESERVED
7	0	0	.		0		0		0		RESERVED
8	0	0	.		0		0		0		RESERVED
9	0	0	.		0		0		0		RESERVED
10	0	0	.		0		0		0		RESERVED
11	0	0	.		0		0		0		RESERVED
12	0	0	.		0		0		0		RESERVED
13	0	0	.		0		0		0		RESERVED

(略)

turn: うえはら

input pins: 3

Game(id:zwBxlorJlog),

Player:くぼあき, Score(id: q5rZ-vnFM1o), Frame:11,

No	1st	2nd	Total	Frame	Score	Spare	Bonus	Strike	Bonus	Frame	State
-1	0	0	.		0		0		0		RESERVED
0	0	0	.		0		0		0		RESERVED
1	7	0	7		7		0		0		FIXED
2	5	5	27		20		10		0		FIXED
3	10	0	52		25		0		15		FIXED
4	10	0	71		19		0		9		FIXED
5	5	4	80		9		0		0		FIXED
6	10	0	100		20		0		10		FIXED
7	7	3	115		15		5		0		FIXED
8	5	4	124		9		0		0		FIXED
9	7	3	141		17		7		0		FIXED
10	7	3	155		14		4		0		FIXED
11	4	0	.		4		0		0		BEFORE_2ND
12	0	0	.		0		0		0		RESERVED
13	0	0	.		0		0		0		RESERVED

8

Player:うえはら, Score(id: plq9pFD6nSA), Frame:10,

No	1st	2nd	Total	Frame	Score	Spare Bonus	Strike Bonus	Frame	State
-1	0	0	.		0	0	0		RESERVED
0	0	0	.		0	0	0		RESERVED
1	6	3	9		9	0	0		FIXED
2	9	0	18		9	0	0		FIXED
3	0	3	21		3	0	0		FIXED
4	8	2	38		17	7	0		FIXED
5	7	3	58		20	10	0		FIXED
6	10	0	78		20	0	10		FIXED
7	9	1	96		18	8	0		FIXED
8	8	0	104		8	0	0		FIXED
9	10	0	123		19	0	9		FIXED
10	6	3	132		9	0	0		FIXED
11	0	0	.		0	0	0		RESERVED
12	0	0	.		0	0	0		RESERVED
13	0	0	.		0	0	0		RESERVED

```
Game(id:zwBxlorJlog) is finished.  
C:\Users\（演習用ディレクトリ）>\score>
```

- ❶ スコアシートの作成日付とIDの表示。
- ❷ どのプレイヤーのターンなのかを表示した。
- ❸ プログラムが受け取ったピン数。
- ❹ 現在のゲームのID。
- ❺ プレーヤーとそのプレイヤーのスコアのID、現在のフレーム番号。
- ❻ 前の前のフレームを無条件に扱うためのダミーフレーム。
- ❼ 1投目が終わったので、フレームは2投目の投球前の状態。
- ❽ ここは2投目の投球前の状態だが、第10フレームの状態が「FIXED」なのでゲームは終了。

動作を確認できるよう、テストプログラムを実行した様子を収めた動画を用意しました。このテキストと一緒に配布している資料として `videos` フォルダに次のファイルを置いてあります。

実行の様子を収めたビデオ（Ruby版のもの。 `videos` フォルダにあります）

- `score_rb_demo_win.mp4`
- `score_rb_demo_mac.mov`

5.3 まとめ

作成したプログラムを、テストコードを使って動かしてみました。テストした結果、作成したプログラムが、モデル（クラス図やステートマシン図）に従って動作していることを確認できました。

第6章 まとめ

このチュートリアルでは、ボーリングスコアのモデルを作ることを通して、ソフトウェアの開発にモデルを使うことについて学びました。また、UMLが、そのようなときに役立つ記法であることも実感できたのではないのでしょうか。ソフトウェアの開発にモデルを活用するときは、記法やツールも欠かせません。このチュートリアルでは、モデル図を描くのに astah* を使いました。ですが、このチュートリアルをやったみなさんは、記法やツールがあればモデルが作れるわけではないことに気づいたのではないのでしょうか。

まず、このチュートリアルでは、ボーリングスコアの構造のモデルや振る舞いのモデルをいきなり作るのではなく、実際のスコアシートを観察してオブジェクト図を作るところから始めました。ボーリングスコアのような自明なよく知られた問題であれば、このような手間をかけなくてもモデルを作成できる人もいるでしょう。しかし、システム開発案件の多くは、どのように対処すればよいのかわかっていない問題を抱えています（だからこそ、そこが開発案件になるわけですね）。このチュートリアルでは、そのようなときに問題を表すモデルを作るのと同じように、ボーリングスコアの記録に関わる領域をモデルで表すところから始めました。

みなさんのなかには、UMLの考え方や記法に慣れていないだけで、モデル化して考えることには慣れている人もいるでしょう。そのような人は、こんどは自分たちがすでにモデル化している図や文書を、UMLを使って表してみるとよいでしょう。

あるいは、すでに作りたいもの（あるいは解決したい課題）はわかってきているものの、それがどのような課題でどのように解決すればよいのかといったことを、うまく表せずに困っている人もいます。そのような人は、みなさんの課題（を含む現状）をモデル図で表してみるとよいでしょう。これを「As Isのモデル」と呼びます。そして、浮き彫りになった問題を、そのモデルを出発点として解消したモデルを作ります。これを「To Beのモデル」と呼びます。課題を解決したシステムは、To Be のモデルを元に設計、実装します。このような方法で、問題解決にモデルを活用してみることにチャレンジしてみてください。

みなさんがこれまで以上にモデルを活用できるようになることを期待しています。

第7章 他の図、他の機能など

この演習で作成したのは、クラス図（オブジェクト図も含む）とステートマシン図でした。UMLには他にも多くの図の記法が提供されています。ほかによく使われる図としては、ユースケース図、シーケンス図、アクティビティ図があります。

付録A: モデルやプログラムの作成例

ボウリングのゲームスコアのモデル図 (`models` フォルダにあります)

- `bowling_score.asta`

C言語で実装したプログラム (`codes` フォルダにあります)

- `score/bowling.c`
- `score/bowling.h`
- `score/main.c`
- `score/Makefile`

実行している様子の動画 (`videos` フォルダにあります)

- `score_rb_demo_win.mp4`
- `score_rb_demo_mac.mov`

参考文献

- [UMLSPEC] OMG 統一モデリング言語 (UNIFIED MODELING LANGUAGE) .
 - <https://www.omg.org/spec/UML/>
- [robustness01] ワークブック形式で学ぶUMLオブジェクトモデリング.
 - ローゼンバーグ, スコット. ソフトバンク. 2002.
- [robustness02] ユースケース駆動開発実践ガイド.
 - ローゼンバーグ, ステファン. 翔泳社. 2007.
- [rulebook] ボーリング競技規則.
 - <https://jbc-iwate.com/service/message/201012101.pdf>
- [kuboaki01] 作って身につくC言語入門.
 - 久保秋. ソシム. 2018.
- [shibata01] 新・明解C言語 入門編.
 - 柴田. ソフトバンク. 2021.
- [learnc] オンライン学習サイト
 - <https://www.learn-c.org/>
- [embtdd] テスト駆動開発による組み込みプログラミングーC言語とオブジェクト指向で学ぶアジャイルな設計 --.
 - Grenning, 笹井 (訳) , 蛸島 (監修) . オライリー. 2013.

モデルを使ってソフトウェアを開発しよう

UML初学者向けチュートリアル (C言語版)

発行日 : 2026-04-01

バージョン : pdf 0596

作成者 : 株式会社チェンジビジョン

本書の内容に関する質問等がありましたら、作成者までお知らせください。